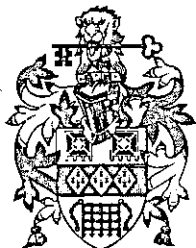


VECTOR

"L'une des toutes meilleures publications APL existantes"

- Arthur Whitney on K 74
 - Windows .BMP files and APL 80
 - Pearson on Dyalog $\square$ NA 117
 - Day and Langlet on "MAGIC" 125
- ... plus**
- 16-page Educational Supplement 25



***The Journal of the
British APL Association***

A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.10 No.1 July 1993

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible (b/w or colour prints welcome). Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The Editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, in workspaces from I-APL, APL*PLUS/PC, APL*PLUS II, IBM APL2/PC or Dyalog APL/W, or in documents from Windows Write (use the Vector TrueType font, available free from Vector Production).

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the Editor of your intention to re-use material from VECTOR.

Membership Rates 1993-94

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£20	1	1
(Supplement for Airmail, not needed for Europe)	£8		
UK Corporate Membership	£100	10	5
Overseas Corporate	£155	10	
Sustaining	£430	50	5
Non-voting Member (Student, OAP, unemployed)	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready format (A4 or A5) with a 20mm blank border after reduction. Illustrations should be photographs (b/w or colour prints) or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

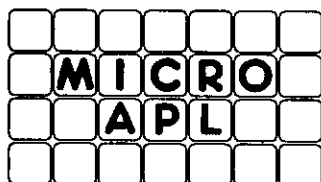
Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to: Gill Smith, Brook House, Gilling East, YORK YO6 4JJ. Tel: 04393-385.

CONTENTS

Editorial: APL in Business	Anthony Camacho	Page 3
APL NEWS		
Quick Reference Diary		5
General Correspondence		9
British APL Association News:		
BAA-GUI Workshop	Duncan Pearson	11
"APL in Business" Conference Prospectus		12
Annual Accounts	Nicholas Small	14
AGM Report	David Eastwood	15
News from Sustaining Members	Gill Smith	19
The Education Vector	Alan Mayer	25
REVIEWS SECTION		
APL Product Guide	Gill Smith	41
The Sharp APL Reference Manual	Jonathan Barman	53
Update on ISI APLIWIN	Jonathan Barman	55
Dyalog APL: Programming for Windows	Jonathan Barman	59
RECENT MEETINGS:		
Business Graphics — A Second Look	George MacLeod	70
GENERAL ARTICLES		
K	Arthur Whitney	74
Windows BMP Bitmap Files and APL	Ray Cannon	80
Coast-to-Coast: Managing with Objects	Adrian Smith	97
TECHNICAL SECTION		
Technical Correspondence		114
A Standard Font Dialogue		
Box from Dyalog APL/W using □NA	Duncan Pearson	117
The MAGIC goes away — Opening the Boxes	Mike Day	125
The Fractal MAGIC Universe	Gérard Langlet	137
Index to Advertisers		143

— APL Preferences ▾

APL.68000



Platforms

- ☒ RISC System/6000
- ☒ Apple Macintosh
- ☒ Commodore Amiga
- ☒ Atari ST

Versions

- ☐ Level I
- ☒ Level II

APL for GUIs

MicroAPL Ltd., South Bank Technopark, 90 London Road,
LONDON SE1 6LN, UK

Tel: 071 922 8866 Fax: 071 928 1006

Applelink: microapl Internet: microapl@applelink.apple.com

Editorial: APL in Business

by Anthony Camacho

Swansea University is host to the conference *APL in Business* (18-22 July 1994) which is to be sponsored by the European Business Management School with the support of the British APL Association and the British Computer Society.

What are we Trying to Do?

We discussed at length how best to organise and sell a conference. The miracle at St Petersburg; APL 92 and the superlative programme for APL 93 are hard acts to follow. Nevertheless these are our suggestions:

- Improved value for companies sending staff and for vendors
- Promote good uses of APL more effectively
- Make profits to give strength to APL organisations for years ahead.

Value for Money

Too many advertisements for conferences begin with the attractions of the location, with the tours and with the entertainment. Anyone would think that the most important people to attract were the accompanying persons. It is overwhelmingly companies who send people to conferences. A few people will come to every conference they can afford at private expense, but not many. It is companies we have to persuade that they will get value for money by sending their staff. If it looks like a holiday then only generous companies who want to reward their staff may send people — only may.

An APL conference is not a holiday camp and should not look like one. Persuade the employers that APL conferences are harder work than any course and will improve their programmers' performance better than any course and we will be able to raise the price and have to turn away the late-comers. Conferences must be good value for the vendors too. It is ridiculous to prohibit sales at conferences. Vendors should be encouraged to sell books, disks, programs, workspaces of utilities, interpreters, interfaces between different flavours of APL and the major spreadsheets, databases, word processors and other packages. Why not persuade companies that there will be products on sale that they cannot afford to be without?

Promoting Good Use of APL

A conference which promises to teach its delegates how to use APL well starts off on the right foot. A conference which promises to teach techniques which other disciplines can use with advantage will also spread the word about APL. We ought to offer to teach people who use a technique how they can do it easier with APL than any other way. There are many techniques where APL has much to offer. Hard-headed people choose APL and some are willing to say why. The best way to promote something is to show it to the people who would benefit from it. We are surrounded by examples of APL's advantages and we scarcely use them.

Making a Profit

To make a profit from running a conference we must persuade people to come. The best way is to make it irresistible to the people who spend company training budgets. To stand a credible chance of delivering what we promise the conference program should be planned. Only part of the conference should depend on what papers arrive in the post unasked for. The plenary speakers should be acknowledged experts in their fields with reputations as excellent teachers, commissioned to produce the material required.

The atmosphere should be of work too. More sessions should be spent sitting at a computer per delegate. There should be competitions and workshops. Sessions should be long enough for something to be learned. Many sessions are too short for the presenter to do more than tell the audience what a wonderful product he has to sell. Good planning of the accommodation and conference facilities is the other secret. The conference should break even with a pessimist's view of the number of delegates and should have as much headroom as possible where every additional delegate adds ninety percent of his fee to the profit and ten percent of it to the costs.

APL is Still a Success

The whole thing depends on the continued success of APL. There are more APLers in the world today than ever. APL innovation is far from dead. There are now at least four GUI APLs. You can now write Windows applications in APL as well as, but faster than you can in Visual Basic. You can link APL to other systems and share data, and with Object Linking and Embedding the data can be updated dynamically by APL running in the background. APL still delivers working systems faster; they are still more reliable and they look as good as those produced any other way.

Quick Reference Diary 1993

Date	Venue	Event
August 15-19 1993	Toronto, Canada	APL93
17 Sept	IEE, London	Vendor Forum. Come and see all the newly announced APL systems from APL93!
26 November	To be announced (near London)	BAA-GUI Workshop: This will be an all-day event (lunch is included) - please use the booking form on page 51 to reserve your place.
March 6-8 1994	Phoenix, Arizona	Symposium on Applied Computing APL-Track Chair: Chris Lee (Manugistics) Papers and panel proposals by Sept 1st 93 Contact Chris Lee at (301) 984-5117

British APL Association meetings are normally held in the IEE, Savoy Place.
Nearest tube outlets: Temple or Embankment.

APL Training Courses for 1994

If you would like to have your courses or seminars listed in Vector, please contact Gill Smith with the details.

Dates for Future Issues of VECTOR

	Vol.10 No.2	Vol.10 No.3	Vol.10 No.4
Copy date	10th Sept 93	3rd Dec 93	4th March 94
Ad booking	17th Sept 93	10th Dec 93	11th March 94
Ad Copy	24th Sept 93	17th Dec 93	18th March 94
Distribution	October 93	January 94	April 94

BAA-GUI

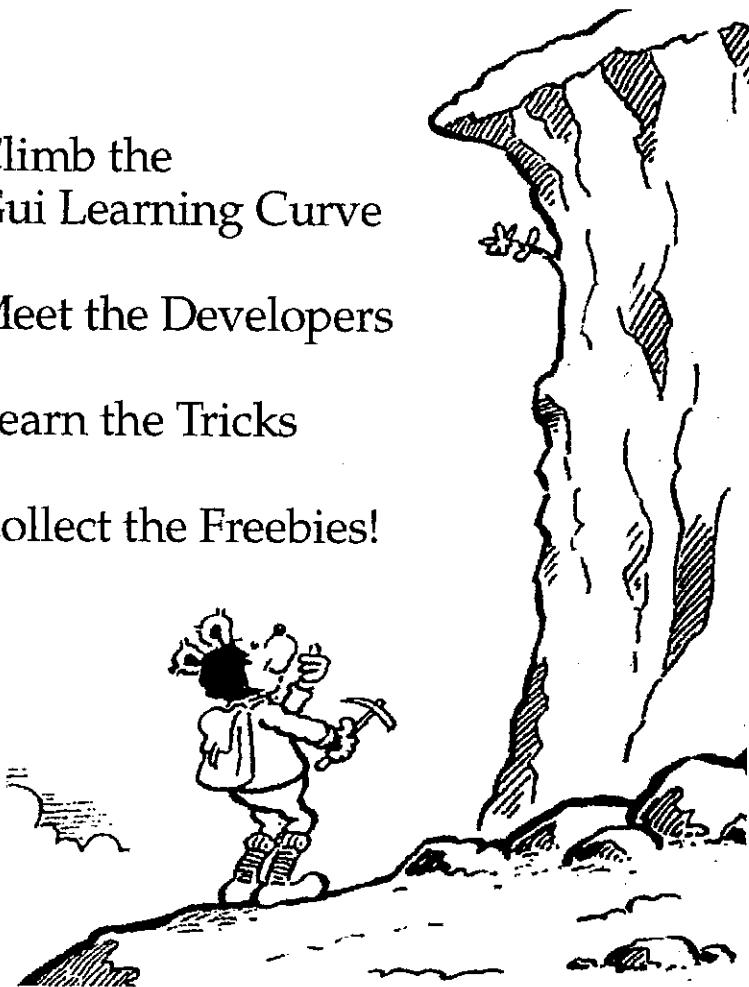
The British APL Association
presents ...

Climb the
Gui Learning Curve

Meet the Developers

Learn the Tricks

Collect the Freebies!



London, November 26th

Learn from the Developers

Speakers from each supplier will tell you how they meant their products to be used.

Manugistics: we have asked Patrick Parks, who designed the APL*PLUS II Gui

Peter Donnelly (Dyadic Systems)

David Eastwood (MicroAPL)



Influence the Designers

Discuss your gripes (or bright ideas) with the designers of the next release. They really value your opinions!

Get Hands-On Experience

Use the machines (one for every two delegates) to build your own systems, and try what you see.

See how Major UK Sites are Managing

Developing large-scale Gui systems presents new and different problems. David Piper (Eagle Star) and Adrian Smith (Nestlé) will explain what problems they had, and how they overcame them.

Take away the FREE programs!

Every delegate gets a disk with many of the tools and utilities used during the day.



The Need for Training

The latest release of Dyadic Systems' APL/W has introduced an APL that can be used to build Windows applications without having to buy the Windows Developer's Toolkit and without having to give up APL and use a language like Basic or C.

Unfortunately that is not the end of the matter. To benefit from the facilities provided by the GUI interface and, at the same time, be able to manage the effects that are created, requires knowledge. Dyalog provides access to 31 objects, each having some of 78 different properties and generating some of 31 different events. Learning how to deal with these objects, properties and events takes time and effort. This time and effort can be considerably reduced by attending a three day Grey mantle Training Class on your own site.

George MacLeod, of Grey mantle Associates, has been working continuously with Dyalog APL/W since its original release and has worked with one alpha and several beta test releases of the current Release 6.3. The purpose of the three day class is to transfer the knowledge acquired.

Class Topics.

- The Objects, Properties and Events provided by Dyalog APL/W

- The use of □DQ and □NQ to control event handling.

- An outline of the Customer User Interface (the standard to which Windows applications should be built).

- Introduction to □NA for accessing Dynamic Link Libraries (DLLs).

- Introduction to the use of Direct Data Exchange (DDE) to enable Windows applications to share data.

- Introduction to Working with a Windows Help Compiler and the Windows Help Engine.

Class Style

The class is a mixture of lectures using overhead projector foils and hands on practical sessions with a PC. Part of each practical session is involved in the building of the student's first Windows application. At the end of the three days the application will be complete, will use many of the GUI objects with their properties and events, and will be a well-behaved Windows application. (The content will of course be small.)

Summary

To benefit from the class the student should be proficient in APL.

The class is charged for at consulting rates plus expenses and materials, so a class can be as small as one student. The maximum number for a class is six.

Telephone, or write, to Grey mantle Associates Limited for more details.

Bartrum House, Ravens Lane, Berkhamsted, Herts. HP4 2DY Tel. 0442 878065

GENERAL CORRESPONDENCE

Review of APL*PLUS II v4.0

From: James G Wheeler

1 February 1993

We at Manugistics appreciate the effort lavished by Ray Cannon in preparing his review of the MS Window interface in version 4 of APL*PLUS II for Vector 9.3. The article contains useful technical information and insights on Windows programming. The issues that he raises about the performance and ease of using our first Windows interface are fair, and we have been working very hard for the past fourteen months to address them.

We believe that we succeeded on this front when we completed version 5 in November 1992, and the praise we have been receiving from version 5 users seems to bear this out. As Vector 9.3 arrived in the U.S., we had nearly completed version 5.1, a significant improvement on 5.0 with important new features.

In this context, it came as a bit of a disappointment to be reading of shortcomings of version 4 that are long forgotten here. I recognize the difficulties of a quarterly publication in keeping pace with rapidly evolving software. But if there is any way the we can assist Vector in providing more up-to-the-minute information, we will do our best to oblige.

James G Wheeler
Manugistics, Inc.
2115 East Jefferson Street
Rockville MD 20852

A Comment on "Announcing The APL Review"

From: Martin Gfeller

7 June 1993

I do think a new, refereed APL Journal will be more harmful than beneficial to the APL community, for the following reasons:

- There are some good APL newsletters such as Quote Quad, Vector and local publications that need material of high quality. Since the APL community is not growing, introducing yet another journal would lower the quantity and quality of material available to the established publications.
- Quote Quad, being an ACM SIG newsletter, has the largest visibility outside the APL community. To strengthen the outreach of APL into the computer community at-large, we need to strengthen Quote Quad.
- More reports on applications of APL should be directed to non-APLers, in publications that are not specific to APL. The yearly APL conferences suffice to report APL stories to APLers; but we need to tell more about our applications to application users and to the software community at large.
- Formal papers about the APL language and system could reach a wider audience by publication in other ACM newsletters and journals, such as SIGPLAN Notices, LOPLAS, or TOPLAS. These communities hardly know that APL is alive and evolving; yet they have a great decision power in education and industry.
- The quality of APL specific material geared to APL users could as well be enhanced by introducing a reviewing process for APL journals such as Vector; many articles could be edited and condensed to serve the readers better. I know and appreciate that the work of volunteers cannot match the professionalism and resources of major trade publications such as Byte; but this would presumably be the case for a new journal as well.

In publications, as in APL coding, less can be more!

Martin Gfeller
Reuters SA Risk Management Software
Kleinstrasse 6 CH-8008
Zürich Switzerland

British APL Association News

BAA-GUI Workshop: November 26th 1993

In order to learn effectively to program for a graphical user interface, with APL or any other language, you need to have a certain level of knowledge and experience, to say nothing of confidence. The November meeting of the BAA will be an all day workshop designed to give you that experience and confidence. You will learn enough about GUI programming to go away and develop an application yourself. You will then be able to work more quickly at solving application problems without getting bogged down in irrelevant technical details.

Part of the 'hump' that most users have to get over is the change from controlling the user to being controlled by him. This has significant implications for system and program design. The UK users will be showing what those implications are and explaining how they have adapted to them.

The major interpreters for personal computers will be covered. There will be parallel streams covering APL*Plus II v5/6, Dyalog APL/W and APL68000 with:

- Patrick Parkes (the man who wrote the GUI interface for APL*Plus II v5) or an alternative speaker from Manugistics.
- Peter Donnelly. A director of Dyadic and a designer of Dyalog APL/W.
- David Eastwood. Managing director of MicroAPL, and an experienced teacher of APL programming techniques.

To show how the interfaces actually are being used and to give some examples from real systems we will have:

- David Piper from Eagle Star. David is a regular contributor to Vector on system design topics.
- Adrian Smith from Nestlé. Adrian is the author of 'APL, A Design Handbook for Commercial Systems' and has worked hard over the years to better understand the relationship between people and computers.

The workshop will be based on tutorials in which the speaker will present methods to the workshop and then the delegates will try them out on their computers. Each session will be about three hours long, broken at convenient times by coffee or tea breaks. Lunch will be provided and there will be ample

opportunity for the delegates to mingle and discuss any aspects of the work with each other and with the speakers.

The venue is not yet fixed but will be within striking distance of central London. There will be a charge of £75 per head to cover the cost of machine hire and catering. In order to plan effectively we need to know the number of people who intend to come, and the streams in which they are interested. Please use the booking form on page 51.

APL in Business 18-22 July 1994 University College of Swansea

The European Business Management School in conjunction with the British APL Association is to host an international conference in Swansea next year. The title 'APL in Business' is an umbrella term to signify the inter-related themes of Finance, Operational Research and Statistics which can work together with the aid of modern computing power to enable more effective management. The future for APL lies in an effective synthesis of technical expertise applied by means of APL in co-operation with other software. The conference aims to bring these application areas into contact with the technical developments in operating systems, managing technologies and hardware.

Consequently we would like to take this opportunity to encourage participation from as many of our readers as possible in the following areas:

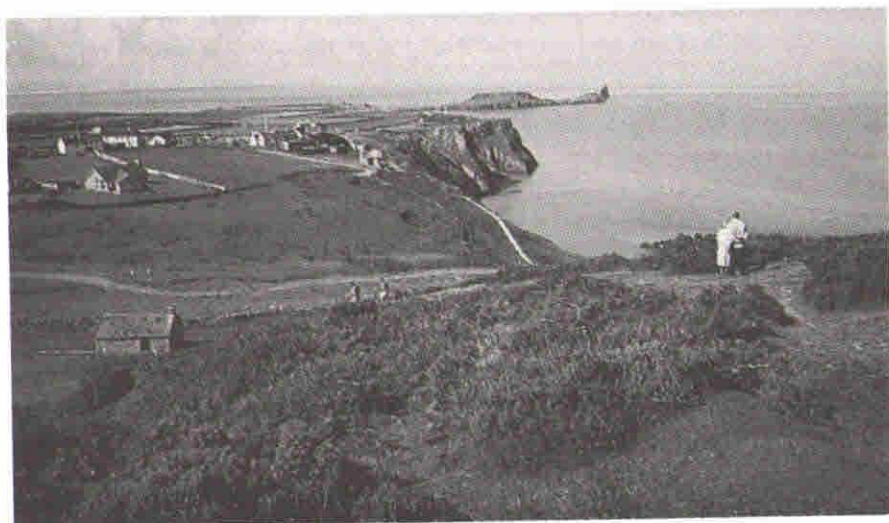
Finance. Trading systems, actuarial methods, credit scoring.

Operational Research. Rule-based systems, scheduling and rostering, management information and control systems, genetic algorithms.

Statistics. Statistical modelling in general, experimental design and Taguchi methods. Forecasting techniques, Econometrics, Bayesian methods, Reliability analysis.

Technical. Designing fully-functional Windows applications covering the GUI objects, the Windows DLL's and the Windows Help engine, Working with messages, co-operative processing (using APL as one element in a suite of co-operating applications).

All papers presented at the conference will be published in a conference proceedings. In addition, selected papers will collectively form the first volume of *The APL Review*, a new international refereed journal to be launched by the British APL Association.



Sessions will take place in the Swansea Conference Centre based on the **Taliesin Arts Centre**, which houses a 350-seat auditorium, an art gallery, bookshop and bank. All the facilities of the European Business Management School will be available to conference members throughout. The College has its own sports centre including squash, badminton and tennis courts and a heated indoor swimming pool adjacent to the campus, making the college a most attractive conference centre. The organizers are committed to making this a most enjoyable and fruitful conference — all you have to do is to commit yourself to come and participate!

Further details will shortly be available in a 'Call for Papers'. If you would like to help with the organization of the conference please contact one of us as soon as possible.

Adrian Smith
Brook House, Gilling East, YORK YO6 4JJ Tel: 04393-385

Alan Sykes,
European Business Management School,
Singleton Park, Swansea SA2 8PP
Tel: 0792-295296, Fax: 0792-295626.

British APL Association - Summary of Annual Accounts 1992/93

Summary of income and expenditure/receipts and payments:

	1992/93 (R&P) £	1992/93 (I&E) £	1991/92 (I&E) £
Income/Receipts			
Subscriptions	14710	14979	15303
BCS services	1152	1152	596
Bank interest	4584	4584	7583
Vector advertising (incl. VAT)	4031	4056	5935
Software and publications	1285	1285	372
	-----	-----	-----
Total receipts	25762	26056	29789
	-----	-----	-----
Expenditure/Payments			
Meetings	2598	2526	2774
Administration	2077	2959	3962
BCS services	1152	1152	596
Outstanding Achievement Award	500	500	500
APL Statistical Library	-	-	2155
Vector production and despatch	11259	13815	14549
Education Vector supplement	2658	3396	3434
Conferences and seminars	-	-	1128
Projects	921	1023	4937
Other	1299	1173	1937
	-----	-----	-----
Total payments	22463	26544	35972
	-----	-----	-----
Assets summary:			
Bank and other balances		74072	70773
Debtors		5510	5100
Creditors		(6393)	(2196)
		-----	-----
Net assets		73188	73677
		-----	-----

Notes

1. Pence figures have been omitted, so columns may not add exactly.
2. The value of stocks of Vector and the Software Library have not been assessed, nor has the value of the Association's computing hardware and software.

Minutes of the Annual General Meeting at the IEE on the 11th June 1993

The meeting began at 14:06. The minutes of the 1992 AGM were approved.

Election of Officers

The chairman reported that, as one of the two candidates for the only contested post, that of recruitment, had decided instead to stand for the vacant post of activities officer, there was no need for a ballot. He therefore presented the new committee to the meeting.

Peter Donnelly is standing down as activities officer due to the pressure of his work and the chairman expressed the thanks of the BAA for the work he has done on the committee over the past five years. Peter Branson is also unable to continue in the post of recruitment officer. Thanks were extended to him for his valuable service, both as recruitment and as activities officer before that. Jonathan Barman having been the editor of Vector for three years is to become the technical officer. It was noted that during his time as editor Vector had been interesting, timely and professionally produced. Anthony Camacho was moving after three years as secretary to be the new editor of Vector and is replaced as secretary by Duncan Pearson. The new activities officer is Richard Weber, a lecturer at Doncaster College of Further Education, and the new recruitment Officer is Mark Harris of Kestrel Consulting. Other officers continuing in their posts are David Eastwood — Chairman, Nicholas Small — Treasurer, Misha Jovanovic — Publicity and George MacLeod — Projects.

Treasurer's Report

For 1992/93, figures are shown both as income and expenditure, i.e. revenues strictly relating to the activities of that year, and as receipts and payments, i.e. what goes in and out of our bank account. The comparative figures for 1991/92 relate to income and expenditure (which are as presented at the AGM last year, being a revised version of what was published in Vector 9.1).

The three main net adjustments made to the receipts and payments figures to arrive at those for income and expenditure related to the cost of producing and distributing Vector 9.4, the associated Education Vector, and late claims for expenses by committee members.

Chairman's Report

The chairman reported that the year 1992/3 had seen the bid to hold APL94 in York rejected by the board of SIGAPL in favour of a conference in Antwerp. It was thought that this was not due to any demerits of the bid but rather that there had been two major APL conferences in the UK in the last decade.

The BAA had given financial support (£1000) to a project by Ian Clarke on the Apple Mackintosh to write and market an educational software building tool using I-APL as a scripting language. This tool, APLomb, received its UK launch at the meeting which followed the AGM. The organisers of APL93 in Toronto have set up a fund to finance the travel to Toronto of Russian APLers who would otherwise find it impossible to attend. The BAA have contributed £1000 to this fund.

In the coming year the Chairman highlighted the proposed change in the format of meetings with the all-day GUI workshop in November and with support for the business and statistics conference to be held by the European Business Management School at Swansea from the 18th to the 22nd July 1994. The BAA is also launching The APL Review, an academic refereed APL journal about which the chairman hoped he would have some more news at the next annual general meeting.

Audit Arrangements

The audit arrangements were approved.

Outstanding Achievement Award

The award was presented to Roger Hui, represented in his absence by Paul Chapman, for his work on the implementation of J. Paul read the following short message of thanks from Roger, "I am honored and gratified to receive the award. Thank you."

The meeting closed at 14:20.

APL93 Russian Fund — Further Funding

Anthony Camacho read a letter from Robert Bernecky and Benjamin Best of the APL93 organising committee saying that the Russian Fund had grown to nearly US\$3,000 by the end of February but that there had been no further contributions by the end of April, and explaining the aims of the fund. Part of the letter is reproduced below. The sum did not include the £1000 from the BAA. He then proposed two alternative means for supporting the fund further, that had been discussed by the committee.

1. That the letter below be sent to all members of the BAA with payment details to enable them to contribute personally.
2. That the BAA make a further contribution of £500.

He pointed out that the expenses involved in the first option (postage and exchange costs on individual payments) might exceed £400. The committee had discussed the two options and was in favour of the second. Anthony took a vote to gauge the feelings of those members present at the meeting. The second option was preferred by a large majority.

Part of the letter from Robert Bernecky and Benjamin Best follows.

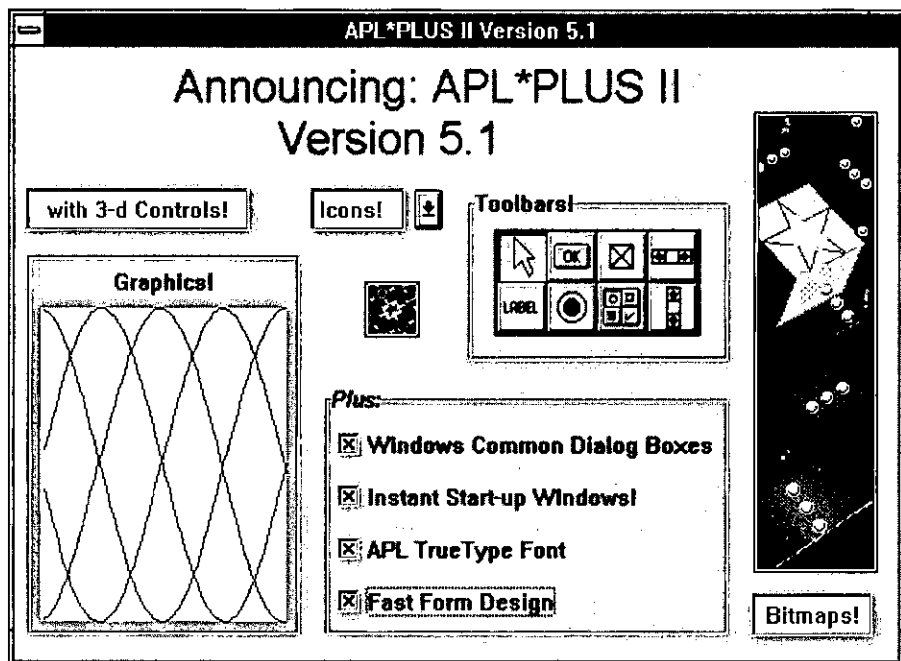
The APL93 Program Chair has now accepted papers from four Russians for the APL93 Proceedings, three of whom cannot attend APL93 without financial support (the fourth is living and working in the United States). The first priority of the APL93 Russian Fund is to bring these three contributors, plus one of the APL92 organisers, to Toronto. Another group of Russians have written an expert system in APL, which they would like to demonstrate. It would also be nice to bring some of the SovAPL founders as well as APLers from the other former Soviet Republics. We would like to bring 8 to 10 if possible. But this would mean much more money — perhaps US\$20,000 — and fairly soon since the getting of Visas can be time-consuming.

The average Russian income is about US\$25, but Russia is not just another Third World country. It is a country of many highly educated people and very many skilled scientists. In a country where chess is the national sport and computers must be adapted to a different alphabetic system, APL has a chance to grow and flourish. By helping Russian APLers to be a part of our community, they can acquire the resources to promote the use of APL as part of their new, emerging society.

Some people question why there should be a fund for Russians specifically, rather than for Eastern Europeans, students, APLers from other poor countries, or APLers under economic hardship. This fund was an act of gratitude for the APL92 conference in Russia — and for the groundbreaking efforts to make that conference a reality. The fund is a recognition that the new connections have been made and should not be lost.

The original objectives of the APL93 Russian Fund were modest. In fact there were moments in the Fall of 1992 when one of us (Ben Best) had fears that he would have to top the fund with money from his own pocket to pay for even one Russian to attend APL93 (after having already spent hundreds of dollars on postage). If this fund is successful, however, it can be the precedent for a "World APL Conference Attendance Fund". It may be that we have colleagues in Eastern Europe, India, China, South America etc. who would submit papers and attend our conferences if they were not subject to such severe economic constraints.

We just couldn't leave well enough alone!



APL*PLUS® II for DOS and Windows™ just got better! It's now even easier (and a lot faster!) to build applications using *all* of the features of Windows 3.1. There's even a Software Developer's Kit that shows you how to *easily* add third-party custom controls!

And don't forget that **APL*PLUS II** is *the only* APL system that provides you with a fully-integrated Debugger, Paradox Interface, full DDE access, Lotus

and dBase Import/Export, optimized Assembler functions, the User Command Processor, a Numeric Data Editor and the most powerful session manager available anywhere!

Get GUIing today! In the UK, phone Cocking & Drury at 071-436-9481. Elsewhere, call your local dealer or contact Manugistics, Inc. in the U.S. at (301) 984-5412, Fax (301) 984-5094 for the name of a dealer near you.

APL*PLUS is a registered trademark of Manugistics, Inc., 2115 E. Jefferson Street, Rockville MD 20852.
Other trademarks are the property of their respective owners.

News from Sustaining Members

Compiled by Gill Smith

Kestrel Consulting

Our recruiting side is very active; handling both permanent and contract placements. We are always pleased to hear from people with strong technical APL skills, particularly when they are combined with sound business awareness and a grasp of the broader technical context within which APL exists. At the present moment we would be particularly interested to hear from people with up to five years of solid APL2; we are also placing people in areas other than APL, particularly the SAP package.

Consulting activities during the last quarter have included participation in a large-scale audit of APL use at a major British insurance company (more than 2000 workspaces on the mainframe). We have also been evaluating a comprehensive Decision Support System written in APL, and would be happy to discuss requirements with anyone having a particular interest in this sort of software.

The Kestrel Software Library was announced in the last issue and we have been (slowly) collecting software; anyone out there with APL tools and utilities which they would like to see distributed at low cost please contact us. Once again — the idea here is virtually a shareware one with Kestrel acting only as a central library. We are happy to distribute full-fledged commercial software as well of course, on a fully professional basis quite distinct from the Software Library..

Manugistics Inc

Manugistics has had a busy few months since the release of APL*PLUS II/386 Version 5.0 and APL*PLUS PC Version 11.

APL*PLUS II/386 Version 5.1 is now available and is generating a great deal of enthusiasm. Reviews and articles on application building have already started appearing in the US. Our customers are telling us that the improved developer tools and dramatic speedups are just what the doctor ordered.

On the UNIX front, Version 5 development is proceeding apace and we expect to be shipping well before the next issue of Vector reaches you. This release emphasizes code compatibility with APL*PLUS II/386, including the User Command Processor and the APL2-style language model.

Patrick Parks has certainly had a busy time recently. Not satisfied with being the chief developer of Version 5.1, he found time to make two trips to Europe. On the first one he presented a seminar to almost 200 people in Paris on APL*PLUS as a development language, emphasizing GUI and DDE. The next day offered an APLGUI internals class to a smaller, more technical audience. Less than a month later he was back, demonstrating the power of APLGUI to groups in Finland, Germany and the Netherlands. Chris Lee also made a similar presentation in Toronto, Canada in May.

As the APL*PLUS II release work quietens down, more and more of our attention is being devoted to the design and development of "INCA". We have just finished a week of intensive design meetings which have given the developers plenty to do over the coming months. The most exciting thing about INCA is the frequent realization that programmers won't have to resort to a lot of the convoluted coding tricks they had to use in a pure APL environment. The incessant chanting by the non-APL community is going to fade away once and for all!

As well as participating in APL93, Manugistics is holding its own User Conference here in Rockville. The conference will be on September 20th and 21st and features presentations, workshops and panels hosted by Manugistics developers and APL programmers as well as a special presentation by Dr Ben Shneiderman, a leading authority on User Design Interface Design. We hope to see some of you there. Immediately following the conference will be a three-day training course on building professional applications with APLGUI and APLDDE.

Something else we're really excited about — the first APL*PLUS PC SE (Special Edition) book has been published. If you didn't know, we introduced the SE as a way for authors and educators to provide a fully working version of APL*PLUS PC with their book or course work at no charge. Maurice Delois of Montreal, Canada has published *"Introduction to APL*PLUS PC"* which is available from EducAPL Inc., 1120 av du Parc, Quebec, Canada G1S 2W7. The price is \$30 for 1-5 copies and \$20 each for more than 5 (plus shipping). Each copy includes a disk with tutorial materials and a fully working copy of APL*PLUS PC. This has to provide the best available way for newcomers to learn APL. We invite others to take advantage of the SE program. Please contact us if you are interested.

MicroAPL Ltd

We are currently finishing off work on an upgrade to APL.68000, which should be complete by the time this issue of Vector appears. The upgrade is mainly

devoted to removing those few bugs that have been reported in our Level II product — either in the original 68000-based version or some that were generated by the port to the RISC System/6000. Full details of the upgrade will be issued directly to registered users.

Our applications consultancy work continues to be almost entirely non-APL and, as mentioned in the last issue of Vector, we held a seminar in London at the end of April entitled 'Developing Windows Applications'. This was held jointly with Maddox Ford Ltd and the principal speakers were Richard Nabavi of MicroAPL and Brian Drury of Maddox Ford.

Richard Nabavi opened the seminar by discussing the convergence of Client-Server computing with Graphical User Interfaces and outlined the various types of GUI development tools that are available — ranging from end-user enquiry applications through to simple tools like Visual BASIC and then on to the professional programming tools such as PowerBuilder. He illustrated his talk with various case studies that MicroAPL and Maddox Ford had been involved with. Brian Drury addressed the issue of managing GUI development projects and the lack of fully thought-out methodologies for GUI development.

The seminar concluded with the presentation of a checklist for the selection of GUI development products.

Dyadic Systems Ltd

Dyadic is delighted to announce excellent progress with the new Motif version of Dyalog APL (ingeniously code-named Dyalog APL/M) which, together with the latest release of Dyalog APL/W for MS-Windows will be the focus of the company's exhibit at APL93 in Toronto. As many of you know, this "news" is written far ahead of publication date, so it is difficult to say how much of the port will be completed by August. However, by mid-June many of the GUI objects (FORMS, BUTTONS, etc. and even some graphical objects) were working. When complete, Dyalog APL/W workspaces developed under Windows on a PC will run unchanged under Motif on a UNIX workstation, such as the IBM RS/6000. This cross-platform portability will appeal to software developers wishing to market products to a wide cross-section of end-users.

At APL93, Dyadic will for the first time be hosting a number of third-party products developed for Dyalog APL/W. There will be several complete applications on show. The first of these is FREGI, an extremely powerful multi-dimensional planning and control system from Kunzle Planning Systems. The second application is JERRY, a decision-support system similar in concept to

ADI, developed by Adviseurs in France. Finally, Dyadic will show an early version of a business graphics application produced by Greymantle Associates which aims to provide very high-quality hard-copy output in addition to the usual attractive screen presentation. All these products take full advantage of the Dyalog APL GUI support and are excellent examples of professional APL applications.

There will also be several add-on products that will appeal to the Dyalog APL developer community. GDDME from Lingo Allegro is a complete emulation of GDDM for Dyalog APL/W including full text and graphics support. It runs as a separate invisible task and is driven from APL by shared variables, just like AP126. This product enables you to port mainframe AP126 applications to Dyalog APL/W with only a few minor changes to the screen driving code. SQAPL is an extremely exciting product developed by Insight Systems in Denmark. It provides the mechanism for Dyalog APL applications to function as both clients and servers using SequeLink Client/Server Middleware from TechGnosis. In practical terms this allows your APL/W application to talk to most SQL databases running on any computer system to which your PC is networked. SQAPL also provided the capability for APL/W workspaces to offload computational and/or data retrieval tasks to Dyalog APL running on a more powerful UNIX server. In short — true client/server computing for APL. Finally, Dyadic will be demonstrating Adrian Smith's superb presentation graphics package which has already been adopted for a Dyalog APL/W Executive Information System developed in Finland.

If you will not have the opportunity to view these products yourself at APL93, please call Dyadic for details.

Dyadic is also pleased to announce an implementation of Dyalog APL for the DEC Alpha AXP running OSF/1. With a SPECmark92 rating of up to 160 (Intel claims 64.5 for a 66MHz Pentium) the DEC Alpha promises to push the level of performance of an APL workstation even further towards mainframe capability, and perhaps even beyond.

Soliton Associates Limited

Soliton Associates Limited is a newly established software company which has acquired assets related to the APL software business from Reuters Information Services (Canada) Limited. Soliton Associates now owns the right to develop, market, sell and support SHARP APL and the application software products: VIEWPOINT, an end-user reporting facility; MAILBOX, an electronic mail system; and LOGOS, a development environment for APL applications.

SHARP APL was originally developed in the 1970s by I.P. Sharp Associates. It is both a powerful computer language and a high-performance computing environment, supporting high-availability, multi-user, shared-data applications on mainframes and workstations using client-server technology.

Soliton Associates supplies its products and services worldwide and has assumed full responsibility for existing international SHARP APL customers, which include Reuters. President, Clarke Bruce, says "Soliton Associates is focused on continued development of SHARP APL and on the development of new products and new markets that will allow it to become a more broadly based software company."

Soliton Associates employees represent collectively over 250 years of SHARP APL business and technical experience. Clarke Bruce says "The company is committed to continuing the heritage of quality products and services from Reuters and I.P. Sharp Associates." The Soliton Associates management team managed the APL business within Reuters, and includes: President, Clarke Bruce; Marketing and Sales, Nancy Lamb; Research and Development, Mike Symes; Software Manufacturing and Support, Margot Tate; and Finance and Accounting, Robert Duncan. The company is headquartered in Toronto, Canada, with offices in Rochester, USA; Coventry, England; and Amsterdam, the Netherlands.

Product announcements from Soliton Associates for the 3rd and 4th quarters 1993 include new releases of SHARP APL for UNIX, MVS-to-UNIX connectivity products and interfaces for the MVS and UNIX environments. SHARP APL Version 21 for MVS is scheduled for general distribution in the 1st quarter 1994.

APL93, the 1993 International Conference on APL, will be held in Toronto on August 15-19, 1993. Soliton Associates is active in the organisation of the conference, with Elena Anzalone as Proceedings Editor, Kirk Iverson as Software Exchange Coordinator, and Larry Moore as Conference Chairman. Don't miss Soliton Associates' booth and forum at APL93.

The Soliton Associates head office will relocate in Toronto in July 1993. New telephone numbers were not available at the time of writing. Mainframe development activities were recently relocated from the Reuters Toronto Data Centre to a new site in the Toronto area.

For information about Soliton Associates products and services contact Laurie Howard, Soliton Associates Ltd, Amsterdam, Tel +31 20 570 8733, Fax +31 20 570 8758, or Nancy Lamb, Soliton Associates Incorporated, Rochester, USA, Tel +1 716 256 6466, Fax +1 716 256 6469.



General Application Consulting
Migration and Downsizing Products
Decades of APL Experience
Fortune 500 Clients
Dyalog APL/W Dealer

LINGO ALLEGRO U.S.A., INC.

203 N. LaSalle St. Suite 2100
Chicago, IL 60601
Phone: +1 312 558-1342
FAX: +1 312 346-9603

THE EDUCATION VECTOR

July 1993

Editor Alan Mayer

This Education Vector has been reprinted from VECTOR Vol.10 No.1. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 11 Auburn Rd, Redland, BRISTOL, BS6 6LS Tel: 0272-730036.

Contents

Editorial	Alan Mayer	26
Step by Step Analysis of Variance	Alan Mayer	27
Backtracking, Queens and Permutations!	Ted Emms	34
Cows and Bulls	Alan Mayer	39

Dr. Alan Mayer
European Business Management School
University College of Swansea
Singleton Park
Swansea SA2 8PP Wales, UK

Ysgol Rheolaeth Busnes Ewropeaidd
Parc Singleton,
Abertawe SA2 8PP

Tel: 0792-205678 Ext.4274 Fax: 0792-295626 JANET: MAYER@UK.AC.SWAN.MS

Editorial

Welcome to the APL93 edition of Education Vector. This year's International Conference on APL is at the University of Toronto, Ontario, Canada, August 15-19, 1993. The British APL Association Journal, Vector (which includes Education Vector), will be available at the Conference, and the Association will be well represented both by speakers and by other delegates. Members of the Association will already have seen in the last edition of Vector the list of Abstracts of Accepted Papers, covering a very large selection of topics, and showing that APL is alive and well throughout the world.

The Guide to Low-Cost APLs in the last edition produced quite a lively response from various readers. If there is sufficient interest, I shall try to compile some of the ideas into a follow-up article. Dick Holt, who wrote the original Guide, now has a new address: HRH Systems, 3802 N.Richmond St, Arlington VA 22207, USA. Also the BBS telephone number, from which APL*PLUS demo and TryAPL2 can be downloaded, is now 703-528-7617 (prefixed by US code). To use this line, set your communications package to 2400 baud, no parity, 8 data bits, 1 stop bit and full duplex.

An addition to the "low cost" list is a new 300-page tutorial "Introduction to APL*PLUS PC" by Maurice Dalois of Quebec. The book costs \$30 and is accompanied by a disk with an "educational" version of APL*PLUS called APLSE which appears to be an almost complete APL*PLUS version 10. As I write this, I have just received a copy, which I shall review for the next edition. It is available from EducAPL Inc, 1120 av du Parc, Quebec, Que, Canada G1S 2W7.

In this edition we have another contribution from Ted Emms, taking an APLer's look at the well-known Chess Queens problem, with a challenge to readers at the end. I have added a note about the game of Cows and Bulls, which I hope may tempt someone to write another solution program — programming the games themselves is a lot easier than programming the solutions!

As always, we are keen to hear from anyone who has new ideas in APL and would like to tell us about their latest projects. We are also looking for new members for the British APL Association.

Application forms are available from Rowena Small, 8 Cardigan Road, London, E3 5HU. Tel: 081-980 7870. UK subscription is currently £12 (students £6).

Step by Step Analysis of Variance

by Alan Mayer

The statistical procedure called Analysis of Variance (or ANOVA for short) was designed by statisticians with the specific purpose of striking terror into the hearts of all who dare to practise the Black Art (hereafter called Stats, or, for our friends over The Pond, Stat) without consulting the gurus. It is full of terms like Error Sum of Squares, Adjustment for the Mean, Degrees of Freedom... Of course, lots of people do their Stats these days with the help of big Statistics Software Packages, which do their best to hide the complicated bits and present neat little tables of results. This is very satisfactory for the Average Person — but we're not Average Persons, we're APLers and we want to find out what's going on inside the packages! And with a little bit of APL that's exactly what we'll do.

First let's have a look at a typical ANOVA problem. Suppose a gardener has four propagators, with six plants in each propagator, and he wants to try out four fertilisers, A, B, C and D. On a certain day, he treats all the plants in the first propagator with fertiliser A, all the plants in the second propagator with fertiliser B, and so on. After a week he measures the growth of each plant in mm. His results may look something like this:

	Growth (mm)					
Fertiliser A	8	6	6	5	4	7
Fertiliser B	2	3	3	1	7	5
Fertiliser C	5	3	1	4	4	3
Fertiliser D	7	7	15	7	11	8

The sort of question we might want to ask now is "Have the different fertilisers caused different amounts of growth?" Let's get the data into APL and experiment a bit.

Enter the data into a table:

```
TAB←4 6p8 6 6 5 4 7 2 3 3 1 7 5 5 3 1 4 4 3 7 7 15 7 11 8
TAB
8 6 6 5 4 7
2 3 3 1 7 5
5 3 1 4 4 3
7 7 15 7 11 8
```

Are the rows different? Let's try adding them up:

$$\begin{array}{r} + / TAB \\ 36 \ 21 \ 20 \ 55 \end{array}$$

Certainly the totals seem to be different — Fertiliser D has produced a total growth of 55 mm, while fertilisers B and C have produced only 21 and 20 respectively. But does this mean that fertiliser D is better than the others? After all, we would be most surprised if the totals were all exactly the same, so some sort of difference is to be expected. How big does the difference have to be for us to consider it meaningful or, as statisticians say, *significant*?

The answer to this problem depends on the amount of variation in each of the rows of figures. If all the values in the first row (i.e. the plants in the first propagator, treated with fertiliser A) are close to the average value of $36 \div 6 = 6\text{mm}$, and all the values in the second row are close to $21 \div 6 = 3.5\text{mm}$ and so on, then these differences will be significant. On the other hand, if the figures vary a lot, the differences may be put down to "random variation". This is where the idea of *Analysis of Variance* comes in.

We need a different approach — one in which we measure how much variation there is in the figures altogether, and then measure how much of that variation can be explained by the differences between the averages of the rows, and how much of it is due to the variation inside the rows themselves.

We will start by working out the Grand Mean (the average of all the figures) and subtracting it from the whole table:

$$\begin{array}{r} GM \leftarrow (+ / , TAB) \div p , TAB \\ GM \\ 5.5 \end{array}$$

So the average growth over all the plants is 5.5 mm.

$$\begin{array}{r} TAB2 \leftarrow TAB - GM \\ TAB2 \\ \begin{array}{rrrrrr} 2.5 & 0.5 & 0.5 & -0.5 & -1.5 & 1.5 \\ -3.5 & -2.5 & -2.5 & -4.5 & 1.5 & -0.5 \\ -0.5 & -2.5 & -4.5 & -1.5 & -1.5 & -2.5 \\ 1.5 & 1.5 & 9.5 & 1.5 & 5.5 & 2.5 \end{array} \end{array}$$

With the removal of the mean, we are left with a table representing the total variation about that mean, but what is needed is a single measure.

Let us experiment:

```

      +/, TAB2
0

```

Of course, adding the values together achieves nothing, as the positive numbers cancel out the negative ones.

```

      +/, |TAB2
57

```

Our second attempt, adding together the absolute values of the entries in the table is more hopeful. In fact, for mathematical reasons, a more suitable measure is obtained by summing the squares of the entries:

```

      TOTSS+/, TAB2*2
      TOTSS
230

```

This value, the *Total Sum of Squares*, is the quantity which must be apportioned between the two causes previously described (between-row variation and within-row variation).

Next we turn our attention to the means of the four rows:

```

      +/TAB2
3  -12 -13 22

```

These are the row totals, which now sum to 0. The means may be found by dividing each row total by 6 (i.e. $(\rho TAB2) [2]$):

```

      ROWMEAN+ ( +/TAB2 ) ÷ ( ρTAB2 ) [2]
      ROWMEAN
0.5  -2  -2.166666667  3.666666667

```

To isolate that part of *TOTSS* which is attributable to the difference between these row means, we must subtract 0.5 from the first row of *TAB2*, -2 from the second row, and so on.

To do this we create a matrix by reshaping *ROWMEAN*:

```

□PP←6
6 4ρROWMEAN
0.5 -2 -2.16667 3.66667
0.5 -2 -2.16667 3.66667
0.5 -2 -2.16667 3.66667
0.5 -2 -2.16667 3.66667
0.5 -2 -2.16667 3.66667
0.5 -2 -2.16667 3.66667

```

This first effort is in fact the transpose of the required matrix:

```

⊆6 4ρROWMEAN
0.5      0.5      0.5      0.5      0.5      0.5
-2      -2      -2      -2      -2      -2
-2.16667 -2.16667 -2.16667 -2.16667 -2.16667 -2.16667
3.66667 3.66667 3.66667 3.66667 3.66667 3.66667

```

So our third table is obtained by

```

TAB3←TAB2-⊆6 4ρROWMEAN
TAB3
2      0      0      -1      -2      1
-1.5    -0.5    -0.5    -2.5    3.5    1.5
1.66667 -0.333333 -2.33333 0.666667 0.666667 -0.333333
-2.16667 -2.16667 5.83333 -2.16667 1.83333 -1.16667

```

This rather cumbersome method of creating the matrix of row means can be avoided by use of the more powerful (and perhaps more APL-like) dyadic transpose. Consider the three-dimensional matrix *TAB2*°. -*ROWMEAN*. To produce the (i,j)th component of *TAB3* we need the (i,j)th component of *TAB2* minus the ith component of *ROWMEAN*. In other words, the (i,j,i)th component of *TAB2*°. -*ROWMEAN*. Hence the required left argument for the dyadic transpose is 1 2 1:

```

1 2 1⊆TAB2°. -ROWMEAN
2      0      0      -1      -2      1
-1.5    -0.5    -0.5    -2.5    3.5    1.5
1.66667 -0.333333 -2.33333 0.666667 0.666667 -0.333333
-2.16667 -2.16667 5.83333 -2.16667 1.83333 -1.16667

```

TAB3 represents the variation which cannot be explained in terms of the differences between the row means.

The corresponding sum of squares is therefore called the *Error Sum of Squares*:

$$\begin{aligned} ESS &= + / , TAB3 * 2 \\ ESS \\ 95.6667 \end{aligned}$$

Of the original *Total Sum of Squares* (230), we have therefore attributed all but the remaining *Error Sum of Squares* (95.6667) to the effect of the different fertilisers. This amount of variation explained by the effects of the particular treatments given to the different plants is called the *Treatment Sum of Squares*:

$$\begin{aligned} TSS &= (+ / , TAB2 * 2) - + / , TAB3 * 2 \\ TSS \\ 134.333 \end{aligned}$$

Rather more than half of the original variation has been accounted for — but is this enough for us to decide that the differences between the fertilisers is significant? To complete our task, we must now perform a statistical test called an *F-test*. This involves dividing the Treatment and Error Sums of Squares by magic numbers called *Degrees of Freedom*, which are very simple to calculate, but much less simple to justify.

If there are R rows (in our case 4) and a grand total of N observations (our N is 24) then the Treatment Sum of Squares will have $R-1$ degrees of freedom, and the Error Sum of Squares will have $N-R$. Our values are respectively 3 and 20. Using these values, we work out the *Mean Square* values by dividing the degrees of freedom into the sums of squares. So the *Mean Square for Treatment* is

$$\begin{aligned} MST &= TSS \div 3 \\ MST \\ 44.7778 \end{aligned}$$

And the *Mean Square Error* is

$$\begin{aligned} MSE &= ESS \div 20 \\ MSE \\ 4.78333 \end{aligned}$$

The *F statistic* is simply the ratio between these two values:

$$\begin{aligned} F &= MST \div MSE \\ F \\ 9.36121 \end{aligned}$$

This is the ratio that really matters — not so much the fact that “rather more than half of the original variation has been accounted for”. We now ask whether this is the sort of F -value we might expect if all the variations between the row means were due to chance (not to the effect of different fertilisers). To check this, we can either turn to a book of statistical tables, and look up the relevant critical value (with 3 and 20 degrees of freedom) or we can use APL.

The function *FTAIL*, listed at the end of this article, calculates the probability that we would observe a value as extreme as F if there were really no differences between the fertilisers. The left argument is a vector containing the two degrees of freedom. If this probability is small (less than 0.05 say) then we can conclude that the fertilisers are really different.

```
      3 20 FTAIL F
0.000452101
```

In this case, we have a probability of about 0.00045. In other words, there is less than one chance in 2000 that these results could happen by chance. It is reasonable to deduce that there are significant differences between at least some of the fertilisers.

This particular version of *FTAIL* loops until a sufficiently accurate value has been achieved. The algorithm is good, and produces accurate results, but the looping approach is not really in the best traditions of APL. Perhaps some of our readers would like to offer something better. We will publish anything that works!

```
[0] B←K FTAIL Y;X;P;Q;PQ;PP;QQ;CX;TM;A;S;R;TP;I;G;XX
[1] X←K[2]÷K[2]+K[1]×Y
[2] P←K[2]×0.5
[3] Q←K[1]×0.5
[4] PQ←P+Q
[5] +(P<PQ×X)/S1
[6] XX←X
[7] PP←P
[8] QQ←Q
[9] CX←1-X
[10] +CT
[11] S1:XX←1-X
[12] CX←X
[13] PP←Q
[14] QQ←P
[15] CT:TM←1
[16] A←1
[17] B←1
[18] S←⌊QQ+CX×PQ
```

```

[19]  R←XX÷CX
[20]  STO:TP+QQ-A
[21]  →(S≠0)/CT2
[22]  R←XX
[23]  CT2:TM←TM×TP×R÷PP+A
[24]  B←B+TM
[25]  TP←|TM
[26]  →((TP≤0.0000001)^(TP≤B×0.0000001))/SEO
[27]  A←A+1
[28]  S←S-1
[29]  →(S≥0)/STO
[30]  TP←PQ
[31]  PQ←PQ+1
[32]  →CT2
[33]  SEO:G←LOGGAMMA +/K
[34]  G←G-LOGGAMMA K[1]
[35]  G←G-LOGGAMMA K[2]
[36]  B←B×*(G+(PP×●XX)+(QQ-1)×●CX)
[37]  B←B÷PP
[38]  →(P<X×P+Q)/CT3
[39]  B←1-B
[40]  CT3:B←1-B

```

Note that *FTAIL* calls the function *LOGGAMMA*, listed below:

```

[0]  S←LOGGAMMA K;II
[1]  S←0
[2]  I←K
[3]  →(K≤2)/ST
[4]  LB:I←I-2
[5]  S←S+●I×0.5
[6]  →(I>2)/LB
[7]  ST:→(0=2|K)/0
[8]  S←S+0.57236494

```

Backtracking, Queens and Permutations!

by Ted Emms

We are all familiar with the concept of backtracking in everyday life. When presented with a choice we make a selection, and pursue that until we are successful in our aim, or we reach an impasse. If we 'get stuck' we retrace our chosen path and select one of the other alternatives. If this process is carried out in a methodical fashion, we will (if it is possible) achieve our aim.

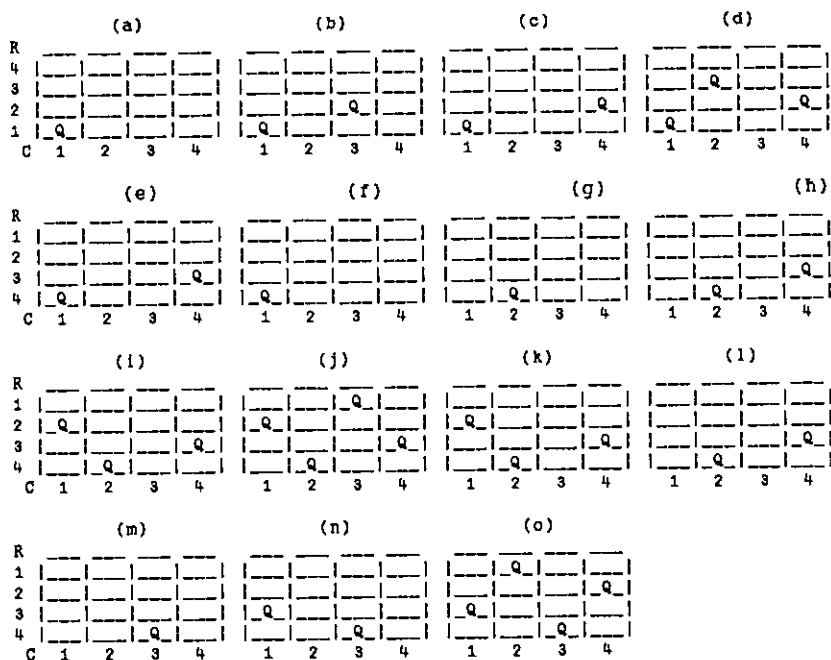
A classic recreational problem is to place 8 queens on an 8×8 chessboard so that no queen attacks any other queen. A queen attacks any piece which is in the same row or column or on one of the diagonals through its square. Backtracking provides a methodical way of solving such a problem. We will choose the simpler problem of placing 4 queens on a 4×4 board to explain the method.

In Figure 1a we have placed a queen on the first row ($R=1$) and the first column ($C=1$). We cannot place another queen in that row so we consider row 2. We consider column 1 ($C=1$) but a queen at that square would violate the conditions. Similarly for $C=2$, but we can place a queen at $C=3$ (Figure 1b). We now move to row 3. We try $C=1$ through 4 but in every case the conditions are violated. We are 'stuck'. So we backtrack. We return to row 2 to find the queen at $C=3$. We try the square to the right ($C=4$) and this is acceptable (Figure 1c). We now move once again to $R=3$ and this time we find that it is possible to place a queen at $C=2$ (Figure 1d). On to row 4, but once again trying $C=1, 2, 3, 4$ we get stuck.

Backtrack to row 3 and try moving the queen at $C=2$ to the right. It is not possible, so we continue to backtrack to row 2 (Figure 1e). We cannot move the queen at $C=4$ to the right, so we backtrack to $R=1$ (Figure 1f). At last! We can move the queen at $C=1$ to $C=2$ (Figure 1g). We can fit in a queen at $C4$ in row 2 (Figure 1h), then a queen at $C=1$ in $R=3$ (Figure 1i), and finally a queen at $C=3$ in row 4. Success!

We have a solution (Figure 1j). But we haven't finished, because we can backtrack again to find further solutions. Figure 1k through to Figure 1o take us to a further solution at Figure 1o. If we again carry out the backtracking routine we find eventually that we are trying to place a queen in row 0 ($R=0$) which does not exist, so we must have found all solutions (2 in this case).

Figure 1



We now have to use this methodology in writing a program in I-APL to find solutions. Experience suggests that we can break the problem up into four routines:

- (1) A routine for placing queens moving up the board (R increasing). This we shall call *QUP*.
- (2) A routine for removing queens and moving down the board (R decreasing). This we shall call *QDN*.
- (3) A routine which acts as the master, calling *QUP* and *QDN* as necessary. This routine can also contain any initialization routines. We shall call it *QMAIN*.
- (4) A routine which prints out the results in a desired form. This is called *QPR*.

We can represent the state of the board by a vector *A* which describes how the queens are placed. Thus *A* ← 2 4 1 3 would represent the position in Figure 1j. An empty row may have a zero so that the position in Figure 1n would be *A* ← 3 1 0 0.

```

[0]  QMAIN N;A;B;Z;R;C;SS
[1]  A←Nρ0
[2]  B←(2×N)ρ' : '
[3]  'Solutions as vectors (0) or as a board (1)'
[4]  Z←□
[5]  R←1
[6]  C←0
[7]  SS←0
[8]  LOOP:QUP
[9]  →(R=0)/OVER
[10]  ±(R=(N+1))/'QPR'
[11]  QDN
[12]  →LOOP
[13]  OVER:'ALL DONE'
[14]  'Total Solutions = ',↕SS

```

The size of the board is N . B is a vector used in QPR . Z records the choice of display. SS is the score of solutions. The main loop is contained in lines [9]-[12]. This repeatedly calls QUP and QDN . If a solution is reached line [10] recognises this and calls the print routine QPR . Line [9] detects when all solutions have been obtained.

```

[0]  QUP;Y;L;M
[1]  LPO:C←C+1
[2]  →(C>N)/OUTUP
[3]  Y←0
[4]  LP1:Y←Y+1
[5]  M←|(C-A[Y])
[6]  L←R-Y
[7]  →(M=0)/LPO
[8]  →(M=L)/LPO
[9]  →(Y<N)/LP1
[10] A[R]←C
[11] C←0
[12] R←R+1
[13] →LPO
[14] OUTUP:

```

Line [2] detects if we are 'stuck'. Lines [4]-[9] form a loop which tests whether placing a queen at C, R violates the conditions. This test is carried out for each queen already placed. M is the difference in column numbers. If the absolute value of M equals $R-Y$ then the two queens being examined are on the same diagonal. If the conditions are violated, lines [7] or [8] return us to LPO and we try the next column. If the conditions are not violated a queen is placed (line [10]) and we move to the next row in order to try to place another queen.

```

[0]   QDN
[1]   R←R-1
[2]   →(R=0)/OUTDN
[3]   C←A[R]
[4]   A[R]←0
[5]   OUTDN:

```

In this routine we go back a row. Line [2] detects whether *R* has got to zero. Otherwise the column number of the last queen place is recovered in line [3] and the queen is removed by line [4].

```

[0]   QPR;D;E;V
[1]   SS←SS+1
[2]   →(Z=1)/ON1
[3]   'Soln. No: ',(VSS),'.....',V A
[4]   →OUTPR
[5]   ON1:V←N+1
[6]   LP2:V←V-1
[7]   D←B
[8]   E←A[V]
[9]   D[2×E]←'Q'
[10]  D
[11]  →(V>1)/LP2
[12]  (2×N)ρ'★'
[13]  OUTPR:

```

The printout routine uses the value of *Z* to give either a vector output (line [3]) or a board (lines [6]-[11]). The routine inserts a 'Q' in the row vector *B* set up in *QMAIN*.

The program is set in motion by the call *QMAIN N* where *N* is an integer representing the number of queens to be placed. I suggest you keep *N* reasonably small until you are sure you have the program working.

The relationship between backtracking and recursion is very close. Some would say that backtracking is just one form of recursion. The standard way for a program to find factorial *n*, for example, by recursion is for the program to call itself repeatedly whilst *n* is decremented down to a value where the result is known (! 1) and then retrace its path (backtrack?) upwards till the value of *n*! is calculated.

Let us now consider the problem of placing 5 different articles (we will call them A, B, C, D and E) in 5 ordered boxes. We could impose conditions (such as A must not be adjacent to B) but we will not do so. The first solution will be ABCDE. Using our backtrack technique, we will get ABCED for the next solution. It is rather surprising to realise that we have a method for getting all the

permutations of the 5 items! Writing the program in four routines following the ideas already given we have *PAMAIN*, *PAUP*, *PADN* and *PAPR*.

It is slightly simpler than the Queens program as it is not necessary to have an *RR* array. The rest follows the same pattern as the Queens program. The program is set in motion by the call *PAMAIN N* where *N* is the number of items to be permuted. Again, try it with small *N* to begin with.

```
[0]  PAMAIN N;L;U;A;X;R;SS
[1]  L←65+(65+N)↑□AV
[2]  U←Nρ0
[3]  A←0/'0'
[4]  X←0
[5]  R←1
[6]  SS←0
[7]  LOOP: PAUP
[8]  →(R=0)/OVER
[9]  ±(R>N)/'PAPR'
[10] PADN
[11] →LOOP
[12] OVER: 'ALL DONE.'
[13] 'Number of permutations = ',↑SS

[0]  PAUP
[1]  LP0: X←X+1
[2]  →(X>N)/OUTUP
[3]  →(U[X]=1)/LP0
[4]  U[X]←1
[5]  A←A,L[X]
[6]  R←R+1
[7]  X←0
[8]  →LP0
[9]  OUTUP:

[0]  PADN: Q
[1]  R←R-1
[2]  →(R=0)/OUTDN
[3]  Q←A[R]
[4]  X←L: Q
[5]  U[X]←0
[6]  A←(¯1+ρA)ρA
[7]  OUTDN:

[0]  PAPR
[1]  SS←SS+1
[2]  A
```

It is an interesting challenge to modify the program so that it will permute a number of articles where some are identical. The permutations of the word APPLE would be less in number than the permutations of the word AMPLE.

[Ted Emms has provided a solution to this challenge, to be printed next issue. Any other offers? - Ed.]

Cows and Bulls

by Alan Mayer

This game has appeared in many different forms under many different names for more years than I care to remember. The computer chooses a random selection of letters, and the player takes a guess at what the list may be. The computer then reports the number of bulls (correct letters in correct places) and the number of cows (correct letters in incorrect places). The idea is to get the correct solution in as few attempts as possible.

In my version, you can choose how many distinct letters can be chosen, and how long the list should be. For example if the game is initiated by the command 4 COWBULL 5 a vector of five letters will be selected from A,B,C,D (i.e. there must be at least one double). On the other hand 5 COWBULL 4 will select four letters from A,B,C,D,E.

```
[0]  M COWBULL N;BULLS;COWS;ENTRY;I;INDEX;SOURCE;TARG2;TARGET
[1]  * M IS NO.OF LETTERS ALLOWED. N IS NUMBER OF VALUES IN TARGET
[2]  SOURCE+M+'ABCDEFGHIJKLMNPOQRSTUVWXYZ' * Define available letters
[3]  TARGET+SOURCE[N:M] * Generate target string
[4]  I+1 * Initialise attempt number
[5]  LOOP:'ATTEMPT ',I * Start loop: prompt for entry
[6]  ENTRY+U * Enter attempt
[7]  +((PENTRY)*P TARGET)/ERROR * Check length of entry
[8]  +(M<{SOURCE\ENTRY)/ERROR2 * Check validity of entry letters
[9]  TARG2+TARGET
[10] BULLS+/(TARG2=ENTRY * Count number of bulls
[11] +(BULLS=N)/END * Stop if entry is correct
[12] INDEX+TARG2*ENTRY
[13] TARG2+INDEX/TARG2 * Remove bulls from target
[14] ENTRY+INDEX/ENTRY * Remove bulls from entry
[15] TARG2+/(SOURCE*.=TARG2 * Freq. of letters in TARG2
[16] ENTRY+/(SOURCE*.=ENTRY * Freq. of letters in ENTRY
[17] COWS+/(TARG2\ENTRY * Calculate no. of cows
[18] (VBULLS),' BULLS, ',(VCOWS),' COWS.' * Print no. of bulls and cows
[19] I+I+1 * Increment attempt number
[20] +LOOP * End of loop
[21] ERROR:'WRONG NUMBER OF LETTERS' * Error: wrong length entry
[22] +LOOP
[23] ERROR2:'INVALID LETTER(S)' * Error: wrong letters entered
[24] +LOOP
[25] END:'CORRECT! YOU TOOK ',(I),' ATTEMPTS.' * Successful completion
```

Error messages are produced (a) if the length of any attempt is not equal to the length of the target (specified by the right argument) and (b) if any letters are used which are not in the allowed list (specified by the left argument).

Let's see how the game is played.

```
4 COWBULL 4
ATTEMPT 1
ABCD
0 BULLS, 3 COWS.

ATTEMPT 2
BABC
2 BULLS, 0 COWS.

ATTEMPT 3 DAAC
3 BULLS, 0 COWS.

ATTEMPT 4 DADC
CORRECT! YOU TOOK 4 ATTEMPTS.
```

Our choice of 4 COWBULL 4 produces a vector of four letters chosen from A,B,C,D. Our first attempt, ABCD, produced 0 bulls and 3 cows. In other words, three of the letters are correct, but all are in the wrong place. Clearly one letter must be dropped, and another must be doubled up to take its place.

Our second effort, BABC, is consistent with what we know so far. We have tried dropping D and doubling up on B. The result is very useful - 2 bulls and 0 cows. B was the wrong one to double; there must be at least one D.

The third attempt, DAAC, is consistent with the previous results, and produces 3 bulls and 0 cows. The D must now be in the right place, as must the first A and the C. The only possible solution is now DADC.

If you find this logic a bit trivial, and you have a few hours to spare, try 10 COWBULL 10 or even 26 COWBULL 26.

Finally, for all you programmers, I've done the easy bit. With my program you can play the game and the computer keeps score. Now it's your turn - how about a program in which the computer decides the strategy?

APL Health Warning: the game COWBULL can be addictive!

APL Product Guide

Compiled by Gill Smith

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry — see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage. The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages.

For convenience to readers, the product list has been divided into the following groups ('poa' indicates 'price on application'):

- Complete APL Systems (Hardware & Software)
- APL Interpreters
- APL-based Packages
- APL Consultancy
- Other Products
- Overseas Associations
- Vendor Addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

We also welcome information on APL clubs and groups throughout the world.

All contributions and updates to the APL Product Guide should be sent to Gill Smith, at Brook House, Gilling East, York, YO6 4JJ. Tel: 04393-385

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	AWL486	1,950	486 based 33MHz PC, 140MB Disk, 4MB RAM, VGA Colour. (inc. 1 year on site maintenance.)
Dyadic	IBM RS/6000 MD320	11,736	APL POWERstation (Greyscale) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 19" 1280x1024 Greyscale Graph Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	13,817	APL POWERstation (Colour) 27.5 MIPS, 7.4 Mflops RISC Processor 8Mb RAM, 120Mb Disk 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD320	22,658	Advanced APL POWERstation 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape 16" 1280x1024 Colour Graphics Display AIX, OSF Motif, Dyalog APL (1-user)
	IBM RS/6000 MD520	37,114	APL POWERSystem (8-users) 27.5 MIPS, 7.4 Mflops RISC Processor 16Mb RAM, 320Mb Disk, 150Mb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (2-8 user licence)
	IBM RS/6000 MD530	72,054	APL POWERSystem (16-users) 34.5 MIPS, 10.9 Mflops RISC Processor 32Mb RAM, 1.34Gb Disk, 2.3Gb Tape CD-ROM Drive, 16 Ports AIX, Dyalog APL (8+ user licence)
	IBM RS/6000 MD540	122,842	APL POWERSystem (32-users) 41 MIPS, 13 Mflops RISC Processor 64Mb RAM, 1.7Gb Disk, 2.3Gb Tape CD-ROM Drive, 32 Ports AIX, Dyalog APL (8+ user licence)
Interprocess Systems	APL2 Dev't Workstation	poa	Mainframe APL2 supported on a PS/2 via a co-processor card with 16Mb of memory running VM/ESA (370 mode). A complete system includes a PS/2, a P/370 co-processor card, and software licenses for VM/ESA, APL2, GDGM and the full line of Interprocess APL2 enhancements.
MicroAPL	IBM RS/6000	12,000+	POWER range of RISC systems running AIX. Dumb terminal or graphical interface.
	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std. configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
Optima	IBM Compatible	poa	Complete PC-based station, APL interpreters & all support eq't

APL INTERPRETERS

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace	DYALOG APL DOS 386	poa	Dyadic's PC 386 APL interpreter.
APL Software	APL*Plus/PC Release 10	450	STSC's APL for IBM PCs & compatibles. Upgrades from earlier releases also available.
	Run-time	poa	Closed version of APL*Plus/PC which prevents user exposure to APL.
	APL*Plus II	1,395	All the features of mainframe APL*Plus for your 386PC!
	Run-time Dyalog APL	1000-10,000	2nd generation APL for Unix systems
	APL2/PC	poa	IBM's APL 2 for the PC.
Cocking/Drury	APL*PLUS PC Rel 10	410	STSC's full featured APL for IBM's and compatibles - Version 10 includes the Quad-NA facility to interface to non-APL software, support for Microsoft Windows and mouse devices. The User-command processor has been built in to the interpreter.
	APL*PLUS PC Run-Time	175 for 5	Closed version of the interpreter for developers, prevents user exposure to APL.

	APL*PLUS PC Developer System	950	Gives rights to distribute an unlimited number of copies of Run-Time application.
	APL*PLUS II System	1200	High powered APL Interpreter for the 80386 chip. Price includes one year's maintenance and free upgrades - volume discounts
	APL*PLUS II Developer System	3200	Unlimited distribution of APL*PLUS II Run-Time applications!
	APL*PLUS II for UNIX	poa	STSC's 2nd generation APL for all major Sparc and Risc Unix workstations.
	APL*PLUS VMS	poa	2nd generation APL for DEC VAX computers under VMS.
	APL*PLUS Mainframe	poa	Enhances VS APL with many high performance, high productivity features. For VM/CMS and MVS/TSO offers simple upgrade from VS APL
Dyalac	Dyalog APL for DOS/386	995	Second generation APL for DOS. Runs in 32-bit mode, supports very large workspaces. Unique "window-based" APL Development Environment and Screen Manager. Requires 386/486 based PC or PS/2, at least 2Mb RAM, EGA or VGA, DOS 3.3 or later.
	Dyalog APL/W for Windows	995	As above, plus object-based GUI development tools. Requires Windows 3.0 or later.
	Dyalog APL for Unix	995-12,000	Second generation APL for Unix systems. Available for Altos, Apollo, Bull, Dec, HP, IBM 6150, IBM RS/6000, Masscomp, Pyramid, NCR, Sun and Unisys machines, and for PCs and PC/2s running Xenix or AIX. Oracle Interface available for IBM, Sun and Xenix versions.
IAC/Human Interfaces			
	I-APL/Mac	13	Macintosh version of I-APL
I-APL Ltd	I-APL/PC or clones	8 - 11	ISO conforming Interpreter. Supplied only with manual (see 'Other Products' for accompanying books).
	I-APL/BBC Master	8	As above
	I-APL/Archimedes	11	As above
	I-APL/Macintosh	13	As above
	Iverson Software Products		I-APL is the UK agent for ALL Iverson Software products, including: Sharp APL + enhancements to the ISI-APL standard Versions of J (and the associated texts) for PC, Mac, Arc, Atari. <i>Please call for price list and order form. Please note there is a £3 packing charge on all orders.</i>
IBM (APL Products)	APL Version 2 Release 1	poa	Full APL2 System for IBM 370 and 390. Product No. 5688-228. See your IBM Branch Office.
	APL2 Application Env't Ver2 Ref1	poa	Run-time Environment for APL2 Packages (IBM 370 and 390). Product No. 5688-229. See your IBM Branch Office
	APL2 for the RISC System/6000	poa	Product No. 5765-012. See your IBM Branch Office.
	APL2 for the IBM PC (US version)	\$495	Product No. 5799-PGG. Part No. 6242936. PRPQ No. RJBJ411 (In HONE, RJ0411). (From IBM Direct, order by Part Number; from your IBM Branch Office, order by PRPQ Number)
	APL2 for IBM PC (European vers)	poa	Product No. 5604-260. Part No. 38F1753
	TryAPL2	free	Free APL2 for educational/demonstration use. Write to APL Products, specifying diskette size desired.
IBM UK	IBM PC APL2	348	APL2 for the IBM PC. From all IBM dealers, including MicroAPL
Iverson Software Inc.	APL386	\$30	Sharp APL Release 20 for PC 386, 486 with graphics, and ability to operate under Windows.
	APLIPC	\$30	For PC under DOS
	APLIWIN	\$30	For 386/PC under Windows 3.1
	APL Reference Manual	\$30	Documentation for all the above.
	J System Kit	\$24	J 6.2 diskette with manual "J: Introduction and Dictionary"
	J Source Code	\$90	Full C source code plus 100-page book
MicroAPL	APL68000 Level I	2000	First generation APL with numerous enhancements. Multi-user version (Unix, Mlrege, MCS).

	APL 68000 Level II	2500	Second generation APL. Nested arrays, user defined operators, selective specification etc. Multi-user version (Unix, Mirage, MCS)
	APL 68000/X	1500-6000	Second-generation APL. Nested arrays, user defined operators, selective specification, etc. Multi-user AIX version with full OSF/Motif support.
	APL 68000 Level I Mac, ST, Amiga	87	First generation APL. Single user, full windowing interface, software floating point support.
	Mac, Amiga	260	First generation APL. Single user, full windowing interface, hardware floating point.
	APL 68000 Level II ST	170	Second generation APL. Full windowing interface, software floating point support.
	Amiga	260	Second generation APL. Full windowing interface. Hardware and software floating point support.
	Mac	520	Second generation APL. Full windowing interface. Hardware and software floating point support.
Optima	APL*PLUS Rel 10	450	
	APL*PLUS II V 4.0	1395	
	APL*PLUS/PC	369	
	APL*PLUS II	950	
	APL*PLUS II PC Developers Kit	poa	
	Dyalog APL	999	
Scillon Associates	SHARP APL for MVS	poa	for IBM MVS mainframes
	SHARP APL for Unix	poa	for IBM RS/6000 and Sun SPARC
Unware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti.
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL.
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers
	APL*PLUS II	call	STSC's full feature APL for 386 machines.

APL PACKAGES

COMPANY	PRODUCT	PRICES(£)	DETAILS
Active Workspace Ltd	Syndicate Manager	poa	Lloyd's managing agent's syndicate / company accounting system. Stamp & Personal accounts (Inc. Run offs)
APL-385	APL-385 FSM-385 DRAW-385 DB-385 GEN-385	50(PC), 125(mf)	Including ... Screen development Screen design Relational W.S. Miscellaneous Utilities
The APL Group	Qualedi	\$1500-4000	Electronic Data Interchange (EDI) translation software for the PC, with strict compliance checking.
APL Software Ltd (mainframe)	RDS	poa	Relation Data Base System
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System

Cocking/Drury (for VSAPL)	E'MENTS & SHAREFILE	poa	Component files, quad-functions & nested arrays for VSAPL under VM/CMS & MVS/TSO
	COMPILER	poa	The First APL compiler!
	FILEPRINT	poa	Print APL component files
	FILECONVERT	poa	Converts non-APL files to APL
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Information Centre spreadsheet incorporating data exchange between APL, FOCUS, IFPS, SAS, APL/DI, ADRII, Lotus123, Visicalc, Multiplan & DIF
(for APL2)	SHAREFILE/AP	poa	STSC's shared access component file system for APL2. Comparable to all APL*PLUS file systems: multi-user storage of APL2 arrays with efficient disk usage
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	Workspace documentation utilities
	FILEMANAGER	poa	Extends APL primitives to database management
(for PC's)	APL*PLUS PC Tools	275	Utilities Including: RAM disk, full screen data entry, menu input, report generation, exception handling and games.
	IRMA Module	90	327x IRMA support.
	FIN & STAT. LIBRARY	250	Financial & Statistical routines
	SPREADSHEET MGR	150	APL-based spreadsheet for APL*PLUS/PC. Cell arithmetic; transfers to ASCII & Lotus
CODEWORK	Helm	poa	Decision Support system for top management. Developed in Italy over 7 years. Requires APL mainframe or APL*PLUS/ii
CYBEX AB	APL Graf/PC	290	Presentation graphics for APL*PLUS/PC (CGI)
	APL Graf II/PC	390	Presentation graphics for APL*PLUS II/PC (CGI).
	Utility Functions APL2	1900	For APL mainframe; Incl. a very fast search.
	Utility Functions II/PC	130	Same package for APL*PLUS II/PC.
H.M.W.	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
HRH Systems	APL Utilities	poa	Software to transfer workspaces between APL*PLUS and Sharp, and between APL*PLUS and I-APL Software to import IBM .ATF files to APL*PLUS.
	APL*PLUS Utilities		Public domain software, unlock locked fns, a user-friendly alternative to locking, fns of mathematical physics, menus, and others.
IAC/Human Interfaces	IAC/Graf	15	Graph plotting for I-APL/Mac
	IAC/Vox	15	Spoken APL characters for I-APL/Mac
I-APL Ltd	Educational workspaces	5	PC format disks with the examples from: Thomson, Espinasse (Xlts 1-4), Kromberg, Jizba & FinnAPL. All the examples to save your fingers!
Impetus Ltd	Impetus	poa	Corporate Modelling and Reporting System.
INFOSTROY	APL*PLUS/Xbase Interface (II/386 Version 2)	\$198	Complete package written in C. Comparable with the data, index & memo files of FoxPro, dBASE, & Clipper. Multi-user support. No DBMS license required.
	(PC Version 2)	\$98	As above for APL*PLUS/PC.
	(DLL Version 1)	\$198	The same in a DLL form! Gives your Windows applications all advantages of DLLs.

	Interprocess Systems IEDIT	\$3000-5000	Full screen APL2 editor with immediate APL execution, and full-screen debugger
	AOC	poa	APL2 Optimizing Compiler, translates APL2 functions to FORTRAN programs.
(mainframe)	AFM	\$6500-15300	High performance component and keyed file system (VS APL and APL2)
(PC)	AFM	\$175	Single user component and keyed files for APL2/PC.
	Enhanced Format	\$2575	A QuadFMT data formatter for VS APL and APL2
	PowerCode	\$2000	External functions for APL2
	CALL/AP	\$4700	For calling non-APL programs (VS APL and APL2)
	WSORG	poa	Full-screen Workspace Organizer for APL2.
Mercia	LOGOL 92	poa	Logistics management system for 386/486 & RISC computers. Sales Forecasting, Inventory Management, Master Scheduling, Distribution Requirements Planning, Sales & Operations Planning.
	TWIGS	poa	A modular library of tools to teach and explore state-of-the-art materials management concepts. Developed by R.G. Brown.
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc
	MicroLINK	250	General device communications
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	APL self instruction for APL*PLUS/PC
Soliton Associates	STATGRAPHICS Ref 5	590	
	LOGOS	poa	Application Development Environment
	MAILBOX	poa	Electronic Mail
	VIEWPOINT	poa	Report generator with Interfaces to DB2 and MVS data
UNIWARE (for mainframe)	STSC's ENHANCEMENTS	poa	Quad-functions & nested arrays for IBM VSAPL
	STSC's SHAREFILE	poa	component files for IBM VSAPL and for IBM APL2
	TOOLS & UTILITIES	poa	Including FILEPRINT, FILESORT, FILECONVERT, FILEMANAGER(EMMA) STSC's database package
	EXECUCALC	poa	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL(VM or TSO)
(for APL*PLUS/PC)	APL Debugger 2.1	FF1950 FF9750	A visual APL debugger to help develop applications (site license)
	Menus 3.0	FF2450 FF12250	Complete set of hierarchical menu utilities (site license)
	ETATGEN 2.0	FF1950 FF9750	Page layout report generator (site license)
	UNITAB 2.0	FF4550 FF22750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
	UNIASM 3.0 (site license)	FF4950	Assembler utilities to speed up APL*PLUS/PC applications
(for APL*PLUS II)	UNISTAT 5.1	FF2900	Data analysis add-on module for Statgraphics
	UNIWARE Toolkit II 4.1	FF39000	(site license only). Relational database system and complete set of utilities for APL*PLUS II development
	APL Debugger II 2.1	FF2950 FF14750	A visual APL debugger to help develop applications (site license)
	Menus II 4.0	FF3950 FF19750	Complete set of hierarchical mouse-driven menu utilities (site license)

	ETATGEN II 2.0	FF2950 FF14750	Page layout report generator (site license)
	UNITAB II 2.0	FF6950 FF34750	An APL*PLUS spreadsheet-like data entry and validation system (site license)
	UNISTAT Plus 5.2	FF4300	Data analysis add-on module for Statgraphics
Warwick University	BATS	250	Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to £35 for academic institutions.
Zark	FAB	free	Training program for the above.
	APL Tutor (PC)	\$299	APL computer-based training. Available for APL*PLUS PC & APL*PLUS II. Demo disk \$10.
	APL Tutor (MF)	\$5000	Mainframe version.
	Zark ACE	\$99	APL continuing education. APL tutor news and hotline phone support.
	APL Advanced Techniques....	\$59.95	488pp. book, (ISBN 0-9619067-07) Including 2-disk set of utility functions (APL*PLUS PC format).
	Communications	\$200 pc, \$500 mf	Move workspaces or files between APL environments.

APL CONSULTANCY

COMPANY	PRODUCT	PRICES (£)	DETAILS
Active Workspace	Consultancy	poa	PC Based APL system design, programming and Implementation.
Adfee	Consultancy	poa	Development, maintenance, conversion, migration, documentation, of APL products in all APL environments
APL People	Consultancy	poa	Consultants available at all levels. Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation, etc.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Ray Cannon	Consultancy	poa	APL, C, Assembler, Windows, Graphics: PC and mainframe
Cocking/Drury	Consultancy	175-275 275-350 300-450 400-600 450-750	Junior consultant Consultant Senior consultant Principal Consultant Managing consultant
David Crossley	Consultancy	poa	Broad experience in many APL environments
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Dyadic	Consultancy	poa	APL and Unix system design, consultancy, programming and training.
E & S	Consultancy	poa	System prototyping: all types of information system, engineering software, graphics and decision support systems APL*PLUS/PC, APL2, Dyalog APL
General Software	Consultancy	from 120	
Greymantle Assoc Ltd	Consulting	poa	Company reporting, business graphics, Windows applications with Dyalog APL/W.
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Michael Hughes	Consultancy	poa	Consultant with 10+ years experience with various APL interpreters and C.
IAC/Human Interfaces	Consultancy	350	APL on Macintosh & PC. HCI design. VDU ergonomics: EC/Health & Safety compliance.

	Documentation	100-200	On-line assistance, product demos & mock-ups, manual writing; foreign language software localization.
	Training	poa	Using I-APL for courseware & distance learning materials; Mac programming in C, APL & HyperCard.
INFOSTROY	Consultancy	poa	APL*PLUS & Windows consultancy. Porting of software written in C into APL*PLUS.
Intelligent Programs	Consultancy	poa	Systems development, enhancements, support.
	Documentation	poa	Preparation of new manuals, rewriting of existing materials.
	Training	poa	Training for APL experts through to non-technical system users.
Kestrel	Consultancy	poa	All APLs, all environments. Design, analysis, coding, maintenance, documentation, training, interfacing.
Lingo Allegro USA	Consultancy	poa	General APL consultancy specializing in Prototyping, Migration, Mainframe to PC Downsizing, Performance Analysis, Troubleshooting, and Graphics.
MicroAPL	Consultancy	poa	Technical & applications consultancy.
Ellis Morgan	Consultancy	250-500	Business Forecasting & APL Systems.
Optima	Consultancy	poa	A range of consultants with 3-15 yrs APL PC and mf experience.
Parallax Systems Inc	Consultancy	\$750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	350	Specialising in Banking, Financial & Planning Systems.
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
Rex Swain	Consultancy	poa	Independent consultant, 15 years experience. Custom software development & training, PC and/or mainframe.
Uniware	Consultancy (Senior)	FF/day 5000	Consultancy from people with at least 8 years APL experience.
	Consultancy (Senior)	FF/day 7500	Advice and training in Windows programming with APL*PLUS II
	Training	FF10000	5-day class on Windows programming with PLUS II version 4.0
Wickliffe Computer	Consultancy	poa	System design, consultancy, programming and documentation. Especially project management and decision support systems

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Adfee	Employment	poa	Contractors and permanent employees
APL People	Employment Agency	poa	Employees placed at all levels.
HMW	Employment	poa	Contractors and permanent employees placed.
HRH Systems	APL lessons		On-screen interactive APL lessons for APL*PLUS, TryAPL2, Sharp and I-APL — in English or French.
	The BBS/APL	\$24 p.a.	703-529-7617, 1200-14400b, N-8-1, 24 hours. APL educational material is downloadable free. An additional 30 megs of APL software for APL*PLUS, PLUS II, IBM, Sharp & I-APL is available to subscribers (cost is \$24/yr). Selection available on disk for \$15 post-paid. Free on-disk catalogue.
I-APL Ltd	An APL Tutorial	3	45pp by Ahvord & Thomson
	An Encyclopaedia of APL (2d Ed)	6	228pp by Helzer
	APL in Social Studies	3	36pp by Traberman
	I-APL Instruction Manual (2d Ed)	3	55pp by Camacho & Ziemann
	APL Programs for the Mathematics Classroom (Springer-Verlag)	16	185pp by Thomson
	J Dictionary	16	by Ken Iverson
	Programming In J	10	75pp by Ken Iverson
	Arithmetic	12	118pp by Ken Iverson
	An Introduction to J	8	47pp by Ken Iverson
	Tangible math	8	36pp by Ken Iverson
	Sharp APL Reference Manual	18	349pp by Berry
	APL Press Books	poa	A comprehensive selection of early APL literature
	<i>Please note there is a packing charge of £3 per order</i>		

IBM (APL Products)	APL2 Keycaps	poa	Keycaps for the PC and PS/2 (USA and UK standard). Product Number SX80-0270.
	APL2 Keyboard Stickers	poa	Product Number SC33-0604.
Iverson Software Inc.	Programming In J	\$15	76pp.
	Tangible Math	\$12	34pp.
	Arithmetic	\$18	123pp.
Kestrel	Employment	poa	Permanent and contract, home and abroad. From individual placement to supply of complete project teams.
	Software Library	poa	Low-cost software distribution service; call for details.
Renaissance Data Systems	Booksellers		The widest range of APL books available anywhere. See Vector advertisements.
Soliton Associates	MVSLINK	poa	Interface from Sharp APL (Unix & MVS) to non-APL data and software in the MVS environment.
	SSQL	poa	High-performance DB2 Interface for Sharp APL (Unix and MVS).

OVERSEAS ASSOCIATIONS

GROUP	LOCATION	JOURNAL	OTHER SERVICES	Ann.Sub.
ACM/SIGAPL	USA	Quote Quad		
APL Bay Area	USA N. California	APLBUG	Monthly Meetings (2nd Monday)	\$15
APL Club Austria	Austria	-	Quarterly Meetings	200AS(indiv), 1000AS(corp)
APL Club Germany	Germany	APL Journal	Semi-annual meetings	DM60
Ass. Francophone pour la promotion d'APL	France	Les Nouvelles d'APL		
BACUS	Belgium	APL-CAM	Conferences & Seminars	£18 (\$30)
CPC UG APL SIG (Capital PCUG)	Washington, D.C.	Capital PC Monitor	Monthly meetings, occasional classes	free
Dutch APL Assoc.	Holland	-	Mini-congress, APL ShareWare Initiative	
FinnAPL	Helsinki, Finland	FinnAPL Newsletter	Seminars on APL	100FIM(private), 30(student), 1000 (Co)
SwedAPL	Sweden	SwedAPL Nytt	Semi-annual meetings, seminars	SEK 75
SWAPL	Texas, USA	SWAPL		\$18
Swiss APL (SAUG)	Bern	Part of City SI-Info		SF60 (SI) + SF20 (SAUG)
Sydney APLUG	Sydney, Australia	Epsilon	Monthly Meetings	

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
ACM/SIGAPL		ACM, 1515 Broadway, New York, NY 10036 USA Tel: (212) 826-0611
Active Workspace Ltd	Ross D Ranson	Moulsham Mill Centre, Parkway, Chelmsford, Essex, CM2 7PX. Tel: (0245)-496647; Fax: 0245-496648.
Adfee	Bernard Smoor	Dorpsstraat 50, 4128 BZ Lexmond, Netherlands. Tel: 31-3474-2337, Fax: 31-3474-2342
APL-385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APLBUG	Lewis H. Robinson	1100 Gough St, Apt 14A, San Francisco, CA 94109, USA Tel: (415) 928-2058
APL Club Austria	Erich Gall	IBM Österreich, Obere Donaustrasse 95, A-1020 Wien, Austria
APL Club Germany	Dieter Lattermann	IBM Germany, Wilckensstrasse 1a, D-6900 Heidelberg, Germany. Tel: (49) 6221-404-243

The APL Group Inc	Stuart Sawabini	644 Danbury Road, WILTON, CT 06897 USA. Tel: (203) 762-3933 Fax: (203) 762-2108
APL People / Software	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
Association Francophone pour la promotion d'APL	Dr. Gérard Langlet	SCM, C.E. Saclay, F-91191-Gif sur Yvette, France. Fax: (33) 1 69 08 79 63
BACUS	Joseph de Kerf	Roolinberg 72, B-2570 Duffel, Belgium.
Anthony Camacho		11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-730036.
Ray Cannon		21 Woodbridge Rd, Blackwater, Camberley, Surrey GU17 0BS Tel: 0252-874697
Paul Chapman		41 Lambs Conduit Street, London WC1N 3NG. Tel: 071-831-3762.
Cocking & Drury Ltd.	Ronlly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 071436 9481 Fax: 071-436 0524
CODEWORK	Mauro Guazzo	Corso Cairoli 32, 10123 Torino, Italy. Tel: 011 885 168 Fax: 011 8122-652
CPC UG	Lynne Startz	Capital PC User Group, 51 Monroe Street, Suite PE-2, Rockville, Maryland 20850, USA. Tel: 301-762-9372.
David Crossley		187 Le Tour du Pont, Quartier Le Moure, 84210 ST DIDIER, France Tel: 90-68-08-87
CYBEX AB	Lars Wentzel	Gruvgatan 35B, S-421 30 V. Frölunda, Sweden. Tel: (46) 31-45 37 40. Fax: (46) 31-45 24 23.
Peter Cyriax Systems	Peter Cyriax	22 Hereford Road, London W2 4AA. Tel: 071-229-5344
Datatrade Ltd.	Ian Tomlin	1 & 2 Sterling Business Park, Salhouse Road, Brackmills, Northampton, NN4 0EX. Tel: 0604-760241
Dutch APL Association	Bernard Smoor (Sec)	Postbus 1341, 3430BH Nieuwegein. Tel: 03474-2337
Dyadic Systems Ltd.	Peter Donnelly	Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL. Tel: 0258-811125 Fax: 0258-811130
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS. Tel: 0689-824741
FinnAPL		SUOMEN APL-YHDISTYSS RY, FinnAPL RF, PL 1005, 00101 Helsinki
General Software Ltd	M.E. Martin	22 Russell Road, Northolt, Mldx, UB5 4QS. Tel: 081-884-9537
Greymantle Associates	George MacLeod	Bartrum House, Ravens Lane, Berkhamsted, Herts, HP24 2DY Tel: 0442-878065
H.M.W. Trading Systems	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, London EC4Y 0HA. Tel: 071-353-4212; Fax: 071-353-3325
HRH Systems	Dick Holt	3802 N Richmond St, Suite 271, Arlington, VA 22207 Tel: (703) 528 7624 Internet: dick.holt@acm.org
Michael Hughes		28 Rushton Road, Wilbarston, Market Harborough, Lincs., LE16 8QL. Tel: 0536-770998
IAC/Human Interfaces	Ian A. Clark	9 Hill End, Frosterley, Bishop Auckland, Co. Durham DL13 2SX Tel: 0368-527190. Email: clark.i@applelink.apple.com
I-APL Ltd	Anthony Camacho (for queries, order forms)	11 Auburn Road, Redland, Bristol BS6 6LS. Tel: 0272-760036
	J C Business Services (for pre-paid orders only)	56 The Crescent, Milton, Weston-super-Mare, Avon, BS22 8DU
IBM (APL Products)	Nancy Wheeler	APL Products (M46V/D12), IBM Santa Teresa, PO Box 49023, 555 Bailey Avenue, San Jose CA 95161-9023, USA. Tel: 1-408-483-APL2
Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
INFOSTROY	Alexei Miroshnikov	3 S. Tulenln Lane, St. Petersburg 191186 Russia. Tel: +7 812-3111611 Fax: +7 812-3153321 Email: alm@infostroy.spb.su
Interprocess Systems Inc.	Stella Chamberlain	11660 Alpharetta Highway, Suite 455, Roswell, Georgia 30076, USA Tel: (404) 410-1700. Fax: (404) 410-1773
Intelligent Programs Ltd	Mike Bucknall	9 Gun Wharf, 130 Wapping High St, London E1 9NH Tel: 071-265-1120
Iverson Software Inc.	Eric Iverson	33 Major Street, Toronto, Ontario, Canada M5S 2K9 Tel: (416) 925-6096 Fax: (416) 488-7559
Kestrel Consulting	Mark Harris	Business & Technology Centre, Bessmer Drive, Stevenage, Herts SG1 2DX Tel: 0438-310155 Fax: 0438-310131 E:kestrel@apl.demon.co.uk
Lingo Allegro USA Inc.	Steven J. Halasz	203 North LaSalle St., Suite 2100, Chicago IL 60601 USA Tel: (312) 558-1342 Fax: (312) 346-9603
Mercia Software Ltd.	Gareth Brentnall	Holt Court North, Henage Street West, Aston Science Park, Birmingham B7 4AX. Tel: 021-359-5096, Fax: 021-359-0375

MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 071-922 8866 Fax: 071-928 1006
Ellis Morgan		Myrtle Farm, Winchester Road, Stroud, Petersfield, Hants. Tel: 0730-263843
Optima Systems Ltd	Paul Grosvenor	Airport House, Purley Way, Croydon, Surrey CR0 0XY Tel: 081 781-1812 Fax: 081 781-1999
QB On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd., Camberley, Surrey, GU15 1LB. Tel: 0276-20789. Fax: 0278-663427. Mobile: 0831-307548
Renaissance Data Systems	Ed Shaw	P.O. Box 20023, Park West Finance Station, New York, NY 10025-1510, U.S.A. Tel: (212)-864-3078
The Rochester Group Inc.	Robert Pullman	50 S.Union St., Rochester NY 14607-1828, U.S.A. Tel: 716-454-4360. Fax: 716-454-5430
Shandell Systems Ltd.	Maurice Shanahan	Chiltern House, High Street, Chalfont St Giles, Bucks., HP8 4QH. Tel: 02407-2027. Fax: 02407-3118
Sollton Associates	Laurie Howard	Sollton Associates Ltd, Groot Blankenberg 53, 1082 AC Amsterdam, Netherlands Tel: +31 20 570 8733 Fax: +31 20 570 8758
Rex Swain		8 South Street, Washington, CT 06793, U.S.A. Tel: 203-868-0131
SWAPL	Stuart Yarus	PO Box 210387, Bedford, Texas 76095, USA Tel: (817) 577 0165
SwedAPL	Glan Medri	Box 16181, S-103 24 Stockholm, Sweden Tel:+46 (8) 96 09 47
Swiss APL User Group		Swiss APL User Group, CH-3001, Bern 1, Switzerland
Sydney APLUG	Rob Hodgkinson	GPO Box 1425, Sydney, NSW 2001, Australia
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France. Tel: (1) 45-27-20-61. Fax: (1) 45-27-20-71. Telex: 648348F UNIWARE
Wickliffe Computer Ltd	Nick Telfer	76 Victoria Rd., Whitehaven, Cumbria, CA28 6JD. Tel: 0946-692588
Warwick University	Prof. Jeff Harrison	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523369
Zark Incorporated	Gary A. Bergquist	53 Sherpissit St, Vernon CT 06066, USA. Tel: (203) 872-7806

BAA-GUI: Advance Booking Form

Due to the interactive nature of the workshop it will be necessary to restrict the number of delegates in each stream. We are therefore giving an opportunity to reserve places.

Name: _____

Address: _____

Telephone Number: _____

Subject to demand we plan to run parallel streams in each of the main interpreters with a graphical user interface on the PC or Apple Macintosh. Please indicate below how many places you wish to reserve on each stream.

APL*PLUS II _____

Dyalog APL/W _____

APL.68000 _____

APLIWIN/JWIN _____

Please return the completed form to:
 Duncan Pearson
 143 Hull Road
 YORK YO1 3JX



SOLITON ASSOCIATES

***Providing customers worldwide
with business solutions through
SHARP APL***



**SHARP APL is now available for MVS
and UNIX with communication software
connecting environments.**

**Learn more about our new company and our
products at APL93 August 15-19 in Toronto, Canada**

In Europe

Soliton Associates Limited of Canada
Groot Blankenberg 53
1082 AC Amsterdam, The Netherlands
Tel: +31 20 570 8733 Fax: +31 20 570 8758

SOLITON ASSOCIATES LIMITED

2 First Canadian Place, Suite 1900
Toronto, Ontario, Canada M5X 1E3
Tel: (416) 364-5361 Fax: (416) 364-2910

In the U.S.

Soliton Associates Incorporated
1100 University Avenue, Suite 111
Rochester, New York, USA 14607
Tel: (716) 256-6466 Fax: (716) 256-6469

Sharp APL Reference Manual

Published by Iverson Software Inc

reviewed by Jonathan Barman

I thoroughly enjoyed reading this manual. In fact I read it all, from cover to cover, on the train to work over a period of about a week. The description of each APL feature is a delight. There are plenty of diagrams to guide you through the more arcane aspects of APL, and each feature is carefully and clearly explained.

It is many years since I have used Sharp APL, so I was particularly interested in all the new features that have been added to the language. My main experience with nested arrays has been in APL2 and APL*PLUS, and I have only read the papers presented at conferences about the rather different approach to nested arrays taken by Sharp APL. Having all the details put together in the reference manual enabled me to see the overall effect of the changes.

There are 13 sections, starting with an overview of APL and ending with details of messages and interrupt event numbers. The majority of the text is taken up with the description of APL primitive functions and operators. The pages are numbered within each section, and there is no overall page numbering; which I dislike. However, the really heinous crime is, in my view, the lack of an index. A true reference manual has to have an index to enable you to refer to the topic of your choice. In order to find a topic you have to scan the Contents, which is a full listing of all subheadings going down 4 levels. If you already know APL this is quite easy, but I am not sure how the tyro APLer would manage.

The manual is not a beginners' text. The Overview starts with a small example of an APL session which is used to define the terms *nouns*, *pronouns*, *verbs*, *adverbs*, etc. I enjoyed the terse and exact descriptions of the terms, but would not want to start an APL course that way.

The second section explains the APL Grammar in considerable detail — in so much detail that I felt it became a little long-winded. However, the paragraphs on precedence, which explain how adverbs and conjunctions (operators) are evaluated before verbs (functions), are excellent. If I had read this description years ago it would have saved quite a bit of aggravation and unnecessary experimentation. Bracket indexing is dismissed as an anomaly in the grammar, and J is cited as the new dialect of APL where new, more convenient, mechanisms are provided to replace the bad old ways!

The next two sections cover Nouns and Pronouns and the naming of Nouns and Verbs. The scene is set for the Rank Conjunction with a detailed explanation of how arrays may be partitioned into cells. The sections also include details of the characters in `⊞av`, how nouns are displayed, the *interface nouns* which are `⊞` and `⊞` input, localisation of names, and indexed assignment. A bit of a hotchpotch, but all these topics have to go somewhere and the grouping is not unreasonable. This is exactly the situation where an index would be invaluable. For example, the detailed explanation of `⊞av` in Section 11 makes no mention that the character set is listed in Section 3.

Section 5 is the longest and details the verbs with outstanding clarity. Dyadic transpose (`⊞`) is always difficult to explain, and I was very pleased to see that two highly visual methods were used, one suitable for three-dimensional arrays using pictures of bricks, and a more general method depicting the switching of axes which is easier to visualise when dealing with arrays of 4 or more dimensions. It was also good to see the relationship between `⊞` and the left argument to `⊞` explained. There were many examples of other verbs which gave me a new insight on the workings of APL. I had not fully appreciated the subtleties of the collating sequence given in the left argument to `⊞`. Also, as I have never used complex numbers in an application, I found the exposition of the way in which each primitive has been applied to complex numbers quite fascinating.

Adverbs and Conjunctions are covered in Section 6. The examples in Section 5 make liberal use of the Rank Conjunction so that on reading through the manual the rather technical explanation seems easy and obvious. Not having used `⊞` and `⊞` before, I felt that the manual has given me an excellent grounding in how to use them properly. When I need to use ISI APL for real I will review this section before I do anything else.

The next sections cover editing verbs, branching, and event handling using `⊞trap`; all covered equally well.

After all this excellent material the manual seems to peter out. The section on APL Component files is a mere 4 pages, and the description of the System Nouns, Verbs and Commands is equally brief. Packages get mentioned, but there are few examples.

The manual is published by Iverson Software Inc. and has *Iverson APL Reference Manual* on the front cover and *Sharp APL Reference Manual* on the top of each page. The ISI sales literature states that this is a reprint of the second edition of the Sharp APL Reference Manual, and that by reducing the font size and removing material not relevant to the PC they were able to print it in a single

volume with their standard wire binding. The result is a very handy reference booklet. It is printed on A5 paper and the wire binding enables it to lie flat when opened.

There are 362 pages, which is just about the maximum that a wire binding can cope with easily. The font size is very small. The print in my review copy was slightly smudged so that • was just a blob and in some cases it was only possible to distinguish + from ÷ by the context. This is a relatively minor quibble as normal text was quite clear enough to read easily.

I would recommend this manual to anyone who is interested in APL, whether or not they intend to use Sharp APL or any of the products from ISI. Even someone dedicated to the other brands of nested arrays would find the descriptions of the primitives that share a common definition invaluable.

Update on ISI APLIWIN

by Jonathan Barman

I received review copies of ISI's latest offering of APLIWIN and JWIN just before the copy deadline for Vector, so this is a very brief update on the new facilities.

Session Manager

The Session Manager (version 2.0) has been greatly improved. There is a standard Windows menu with the normal entries of File Edit and Help together with a Font option. The menu cannot be accessed by the normal accelerator keys as ISI have commandeered all the Alt key combinations for the entry of APL characters. Alt+F, for example, gives a ♦, so this cannot be used to access the File menu entry. However, the Alt key on its own highlights the first menu item as is usual in Windows.

The File entry just contains an Exit option, so you still have to use the normal)lib and)load for workspaces. The Edit option contains the standard cut and paste entries, and shows that the standard accelerator keys of Ctrl+C, etc, can be used, including Ctrl+Z for undo.

The Font option allows you to change the size of the APL font, or to switch to any other font of your choice. The APL font can be changed to italic, and/or bold. Switching to a non-APL font might be useful as a temporary measure when running old DOS-based applications. If APL characters are on the screen the results are quite horrid!

Help now includes a really compact keyboard layout which you can pop up with Ctrl+F1. The window is quite small and is very clear. It stays on top of the APL session window, but disappears behind other windows when you switch applications.

The boxed entry area at the bottom of the screen has been removed, but the input area is still at the bottom of the session. The input line is the only line on which you can execute an APL expression. If you place your cursor on any other line on the screen and press the Enter key the text is automatically transferred to the input area and you then have to press the Enter key again to execute it. The text on the screen can be altered in any way you like. Cut and paste are character oriented, so that you can clean up the output if you need to.

There is now a scroll bar along the bottom of the window so that if `⎕PW` is set to a large value you can easily scroll to the right hand portion of the output.

The JWIN session manager is identical except that the APL keyboard is missing. You can switch to any font, but anything other than the standard font wreaks havoc on the J display of verbs and boxed nouns.

I was pleased to see these changes, particularly as they are CUA compliant. The overall effect makes for an easy environment in which to work. I particularly liked the pop-up keyboard layout.

Windows Interface

The major additions to the Windows interface are the DDE and OLE interfaces. The list of Windows features is considerably longer, but I have not had time to investigate what they all do. One of the problems with the APLIWIN and JWIN Windows interface is that the only documentation is in the Help file which is decidedly terse. At least ISI have now provided the whole of the Help as a file in Write format so one does not have to go through the help screens printing them out.

The OLE demonstration workspace takes one through some simple but stunning examples of OLE. Linking with Microsoft Graph, which is included with Word

for Windows, allows one to produce graphs from APL and J with the minimum of effort. There is a lot of power here which is waiting to be exploited.

I do not have Excel, so could not run the DDE demonstration directly. I spent a few minutes trying to do the equivalent job with Quattro Pro for Windows, but found that I needed to study the manuals and get better acquainted with DDE before I could proceed. The ISI documentation recommends that you start by linking data between two APL or J applications to get the feel of the thing. As Adrian showed in his article in on Shared Variables and DDE, in Vector 9.2 page 128, the DDE interface is a powerful way of exchanging data between applications. I look forward to playing with the ISI version of DDE.

There is a new picture class of object which allows one to display bitmap files (.BMP), Windows metafiles (.WMF) or icon files (.ICO). I was able to display .BMP files easily, but Ray Cannon's sample bridge picture came out in black and white in a little box. More investigation is needed to see if I can get it to display in the full 256 colours at its proper size.

A whole bunch of commands has been added. I noticed that one can now invoke the standard Windows font selection dialog box. A rectangle with rounded corners can be drawn in a graphics window. Colours can be set in a dialog box. There is more control over events that will end wait commands.

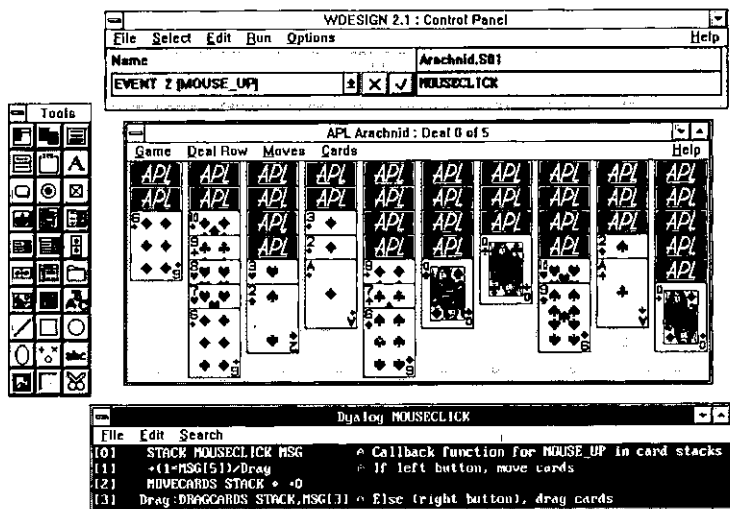
All these new features are beginning to look as though one will be able to create some reasonable Windows applications with APLWIN and JWIN.

Conclusion

For a total of \$70 (US or Canadian) you can get APLWIN and the APL reference manual, which is an unbeatable price. This is definitely the best buy APL which is ideal for production systems as well as for home use. On a 486 the APL performs at a perfectly adequate speed, and you have the tools to build quite respectable Windows applications.



The DEFINITIVE APL for WINDOWS™



- Produces snappy, professional-looking Windows applications.
- FREE Run-Time.
- Powerful, elegant and easy-to-use GUI support.
- Complete device-independent graphics, including bitmaps, icons.
- Built-in support for drag-drop.
- WDESIGN workspace for interactive GUI development.
- DDE support through Shared Variables.
- Direct access to Windows clipboard.
- On-the-fly calls to DLLs (eg Oracle, Microsoft SQL Server) using □NA.
- No workspace limitations.
- Supports all Windows printers with TrueType APL font.
- Workspaces portable to OSF/Motif, Windows NT and OS/2.

DYADIC

Dyadic Systems Limited, Riverside View, Basing Road, Old Basing, Basingstoke, Hants RG24 0AL, United Kingdom. Tel: (0256) 811125 Fax: (0256) 811130 Email: sales@dyadic.co.uk.

Dyalog APL Version 6.3

reviewed by Jonathan Barman

Summary

I am most impressed with Dyalog APL. It is full of ingenious and useful features and extensions to APL, and is one of the best implementations of APL on the market. However, whilst preparing this review it was quite difficult to avoid harping on those relatively minor aspects that I found annoying. The main annoyance was the manual, which is reasonably well written but is very poorly presented and is therefore difficult to use. Also, I would dearly like a CUA interface so that I did not have to learn Dyadic's quirky key assignments for cut and paste, etc.

New Features

Dyadic have gone down the road of providing specific Windows facilities that are embedded into APL, so that one does not have to learn C or the Windows System Development Kit (SDK). Version 6.3 of Dyalog APL completes the process of providing the Windows facilities that can be called through System Functions. The number and range of Windows functions provided is impressive. Windows applications can now be written with full control over the use of bitmaps, fonts, line drawing, the clipboard, printing, drag and drop — more than enough to enable you to create fancy Windows applications. A few things are not included, the major omission being the ability to create Multiple Document Interface (MDI) applications.

A new System Function `⌈NA` allows you to call Windows Dynamic Link Libraries (DLLs) from APL. This makes it possible to call those Windows functions that are not available in Dyalog APL to be accessed from the DLLs supplied with Windows or from DLLs that you have written yourself. However, this means that you need to become familiar with the Windows SDK and programming in C. `⌈NA` is also a useful tool to access other Windows applications that are programmed using DLLs.

There are numerous workspaces, starting with simple tutorials on how to use the new facilities, and going up to a full blown application to help you design a Windows application.

There is so much material that it is impossible to do justice to it in a single article in Vector. This review will give a brief overview of the facilities, and further articles will be needed to describe specific aspects in more detail.

Getting Started

Installation was, initially, very simple. A glance at the User Manual showed that one did the normal thing for a Windows product, that is, poke the first disk into my A: drive, go into Program Manager and run `a:setup`. After a few minutes I got an odd little message asking me to edit the `QUADCMD.PIF` file to show the actual directory I had specified. Looking in the index of the User Manual drew a blank, as did running the thing, which just launched me into a DOS session. Later on I found the section in the manual that explains that the PIF file is required for the `□CMD` processor, and the last thing I should have done was to run it from File Manager. The index has an entry for DOS Command, but not `□CMD` or `QUADCMD`.

Firing up APL looked as though it was a success, but the character mapping was all over the place. Trying to type `)OFF` resulted in an undignified search for the `)` which was finally located next to the backspace key. Back to the manuals for a look at the character mappings, which are simple and very flexible. I was able to use my favourite Unified keyboard, except that the APL quote is different from the ASCII quote, which caused a great deal of confusion as I kept getting *SYNTAX ERROR* when entering an apparently valid character variable. The `Ctrl+K` key produces the APL quote and the normal key produces the ASCII quote. George MacLeod suggested that I reversed the mapping of the two keys, and since then everything has been very easy.

I had some difficulty getting started with the Windows features of Dyalog APL. There are several reasons for this. First, I have not had the opportunity to use Dyalog APL before, so some time was spent getting to grips with the environment, such as how to cut and paste in the editor, how to edit variables, and all the little things that one takes for granted after using a product for a while. Secondly, I started by trying to implement a relatively complex Windows application that required a substantial amount of programming and I found that this was too much to handle all at once. I should have spent more of my time studying the supplied workspaces. Lastly, Dyadic have not supplied a quick reference card for the Windows facilities, and the layout of the manuals makes them difficult to use as quick reference material. Adrian Smith sent me his own 'home-brew' quick reference guide, which proved to be invaluable.

Bitmap Manipulation

Ray Cannon's article *Windows BMP bitmap files and APL* is an excellent introduction to bitmaps. Ray has done amazing things with Pocket APL to manipulate the bitmaps, but if he had had Dyalog APL he would have found life much easier. For example, reading a bitmap can be carried out with the simplest of functions:

```

      v←getbmp a;bm
[1]  a Get the bitmap from file <a>.BMP
[2]  'bm'⊂WC'BITMAP'a
[3]  r←'bm'⊂WG'BITS' 'CMAP'
      v

```

Reading the bits and palette into the workspace is then just a matter of typing `b←getbmp 'bridge'`.

A short explanation of the function will be useful for those who have not seen the Dyalog APL facilities before. `⊂WC` on line [2] creates a Window, WC standing for Window Create. The left argument gives the name of the window and the right argument is a nested vector of properties. The first element of the right argument specifies the type of object. In this case `'BITMAP'` specifies that a bitmap object is to be created, which is invisible.

Each object type has a different set of properties which can be specified as the remaining elements of the nested array. In this case the name of the bitmap file is given in the variable `a`. The properties can be specified in the correct sequence or by name. For example I could have given the file name first and then specified that this is a bitmap object with `( 'FILE'a )( 'TYPE' 'BITMAP' )`. Specifying the properties in the default sequence means that I can omit the property names. Notice that the name of the window object, `bm`, is specified as a local variable. This is necessary as `⊂WC` creates `bm` as a new class of object in the workspace. Windows objects have the equivalent status of a variable, but do not appear in the `) VARS` list, and their existence can only be displayed with `⊂NL 9`.

Line [3] reads the contents of the bitmap file with `⊂WG` (Window Get) which works in much the same way as `⊂WC` except that you always have to specify the names of the values of the properties required. The function specifies the properties `BITS` and `CMAP`, which are returned as a nested vector in the sequence specified.

When processing Ray's article for Vector I used the bitmap facilities of Dyalog APL to get the images into a form that would fit nicely onto the page. For

example, putting several transformations side by side on the page was carried out by the following little function:

```

v new combine list;b;c;i;n;s;b0;b1;c0;c1;bm;QIO
[1]  a Combine BMP files in <list> into <new>
[2]  QIO←0
[3]  n←plist p i←0
[4]  b0 c0←getbmp Directory,i>list      a Read first bitmap
[5]  s←((>p b0),10)p-1+>p c0           a 10 pixel space using last pite row
[6]  c←20+180[ln÷10+1+p b0           a Indices of portion to display
[7]  b←b0[;c]                         a First image in result
[8]  i←i+1 o →L2/÷ns i               a Loop for remaining images
[9]  L1:b1 c1←getbmp Directory,i>list  a Read ith bitmap
[10]  s(c0=c1)+'+ '                   a Crash if different palettes
[11]  b←b,s,b1[;c]                   a Catenate selected portion of image
[12]  i←i+1 o →L1/÷n>i               a Continue loop while images left
[13]  L2:'bm'∘HC'BITMAP'(Directory,new) b c0 a Create form with new bitmap
[14]  1 QNQ'bm' 91                   a Generate event to write to file
v

```

All this function does is to loop around getting in each bitmap and concatenating a selected portion of them together. A properly constructed function would need to allow the preferred portion of the picture and the width between pictures to be specified as arguments, rather than the program constants of 10 20 and 80 which are hard-coded into the function. The global variable *Directory* contains the path to Ray's bitmap files.

I put together a number of little utilities to view bitmaps, to copy bitmaps from and to the clipboard, and to write variables and functions to the clipboard so that I could paste them into this article. It was all very quick and easy to do, and made the job of writing the review easy.

I also had fun changing the colours of the pack of cards in the Solitaire patience game distributed with Windows. A colleague at work is colour-blind and finds red difficult to distinguish from black, which makes playing Solitaire rather difficult. Using a resource editor I copied the red cards to the clipboard and then used APL to convert the reds to other colours. Playing Solitaire where hearts and diamonds are deep blue is incredibly difficult and gives one a good idea of what it must be like to be colour blind!

Workspaces

There are several introductory workspaces. *WINTRO* is an introduction to the Windows facilities with 45 simple lessons taking you through the main features of GUI programming in APL. *WTUTOR* is a more elaborate introduction where there are several lessons for each Windows facility. *CALC* is a demonstration of an implementation of a calculator. *BMED* is a simple bitmap editor for creating

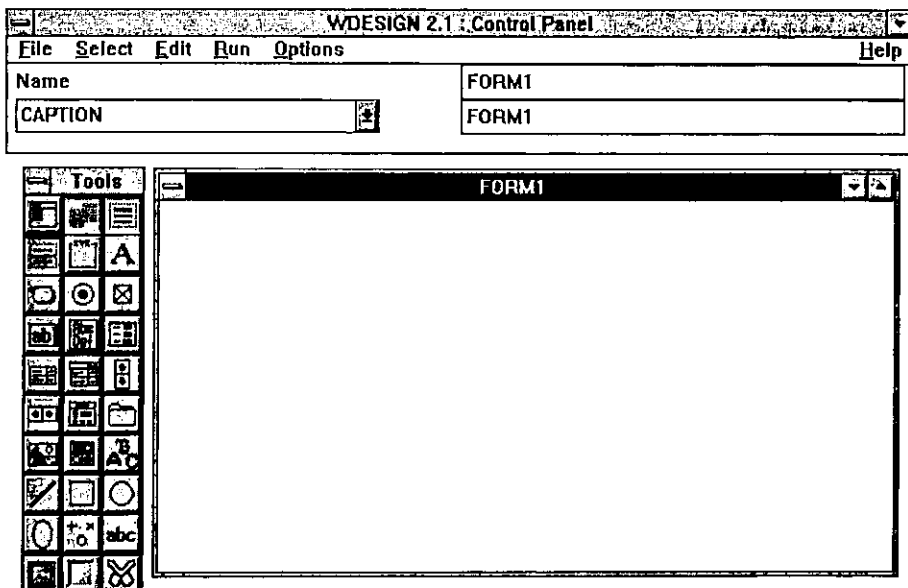
small objects such as buttons. *BUDGET* is an example of a spreadsheet using the *SM* facilities combined with Windows. *GRAPHS* contains a very full demonstration of many types of graphs and should give you a flying start in the production of graphics applications. *QUADNA* gives some examples of using *NA* to access standard Windows facilities.

There are two complete application workspaces. *WDESIGN* is a sophisticated tool for designing Windows applications. *ARACHNID* is a game of patience using bitmaps with drag and drop.

Workspace *WDESIGN*

Dyadic have put a considerable amount of effort into their *WDESIGN* workspace, and the result is a excellent example of what can be achieved with the new facilities.

On loading the workspace three new windows pop up on your screen, a Control Panel, a Tool Bar, and a Form. As there is no MDI facility included with Dyalog APL these windows float freely on the screen and appear to be unconnected. In fact they are connected. If you press Ctrl+Esc or Alt+Tab to switch between applications only one entry is displayed for the group of windows. In the following example I have moved the three windows together to allow them to fit on the page and to remove the extraneous rubbish which I happened to have on my screen at the time. I used the Dyalog APL facilities to create the image.



Everything in the picture has been created in Dyalog APL. The Tool Bar is a set of bitmaps which are held in a nested array in the workspace. The Control Panel is created by a fairly lengthy function which specifies the menu and the child controls. The Form is the first Windows object which you are about to define in your application. The only thing that you could not have created directly in Dyalog APL is the help text, where you would have needed a help compiler to create the standard Windows help file that has been provided. The help provided is excellent and quite extensive, running to about 8 pages when printed out.

The objective of the workspace is to enable you to design the forms for your Windows application and, if required, to define the main functions to create the objects and the call-back functions to process the events. The names of the 412 functions and variables in the workspace all start with the letters *Wd*, so you have to be careful not to use these starting letters for any objects in your own code.

Before you start using *WDESIGN* you should design the way in which your application is going to work. It is hopeless to expect to use this design tool, or any other design tool, without a clear idea of what it is you are about to produce. To get used to the facilities available it is probably best to copy the example from the user guide, or to implement the example Duncan Pearson gave in his review of Dyalog APL/W in Vector 9.2, on page 55.

The functions produced have text hard coded where I would normally have variables, and need editing before they can be used. For example, adding a default list box to the default form shown above results in the following functions being defined:

```

      V MAIN
[1]  A Main Function
[2]  DDQ'.
      V

      V MAKE
[1]  MAKE_ROOT
[2]  MAKE_FORM1
      V

      V MAKE_FORM1
[1]  'FORM1'WC'FORM'('CAPTION' 'FORM1')('POSN'(27
16.25))('SIZE'(65 80))
[2]  'FORM1.LIST1'WC'LIST'('ITEMS'('One' 'Two' 'Three' 'Four'
'Five' 'Six' 'Seven' 'Eight' 'Nine' 'Ten'))('POSN'(18
16))('SELITEMS'(0 0 0 0 0 0 0 0 0 0))('SIZE'(42.05128098 17.96875))
      V

```

```

      ▽ MAKE_ROOT
[1]  A ROOT object has default properties
      ▽

```

All the functions are exactly as produced by the minimal use of the *WDESIGN* process. The function *MAIN* is presented for editing so that one can include the *MAKE* function and other code which would be essential for your application, but I have not altered the supplied default. Normally, an application would have a number of *MAKE_x* functions which would be called by *MAKE*.

Line [2] of *MAKE_FORM1* contains the list of items that are needed when creating a list box as a vector of text strings. I would expect to use a variable for the contents of a list box, as elements are usually things that can be added or deleted elsewhere in an application. If matrix *Mat* is to be used to specify the contents, then I would probably edit line [2] as follows:

```

[2]  'FORM1.LIST1'□WC'LIST'('ITEMS'(+Mat))('POSN'(18 16))
      ('SELITEMS'(0p~>pMat))('SIZE'(42 18))

```

At some later stage I might need to change the position and size of the list box, probably in conjunction with the position and size of all the other objects in order to make room for something else. To do this I would have to run the *MAKE* function to create the Windows objects,)*COPY WDESIGN*, run *Wd* and then drag the objects to the desired positions. However, if I request that the objects are to be saved as functions, then I lose my original definitions. In the following example I had defined *Mat* ← '*M*'□*NL* 3 and then dragged and resized the list box. On saving the object in functions, line [2] is redefined with the current contents of the object in the workspace:

```

[2]  'FORM1.LIST1'□WC'LIST'('MAIN      ' 'MAKE
      'MAKE_FORM1' 'MAKE_ROOT '))('POSN'(10 14))('SELITEMS'(0 0 0
      0))('SIZE'(58 38))

```

In order to avoid this I have to save the object in the workspace, and then to extract the new positions using '*FORM1.LIST1*'□*WG*'*POSN*' '*SIZE*'. The results would then have to be used to edit the numbers in the functions. This would drive me into keeping the positions and sizes in variables and writing a little function to update the variables after using *Wd*.

Another, but minor, annoyance was that the *WDESIGN* cover function, *Wd*, checks to make sure that you are working in 16 colours, and exits if any other number is found. I like using 256 colours, so had to switch to 16 colours and restart Windows before loading the workspace. I could not find a reason in the manuals for this restriction.

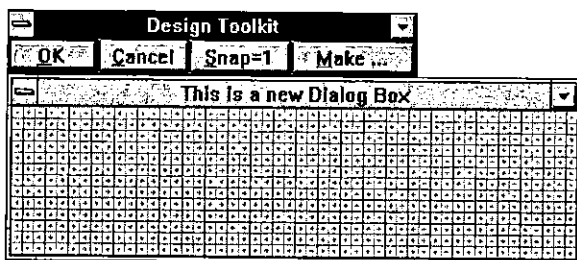
Once mastered, *WDESIGN* will enable you to create complex applications without having to look at the manuals. The help is excellent, and it is easy to select the properties required from lists of the properties available for each object.

Adrian Smith's Window Designer

Adrian is currently experimenting with a completely different design approach. He has programmed a low-level set of cover functions with a different function for every type of Windows object that Dyalog APL supports. These functions are used by the design process and also by the application. All functions have the same syntax, with the left argument being the object name and the right argument a 5 element nested vector which specifies a standard set of properties.

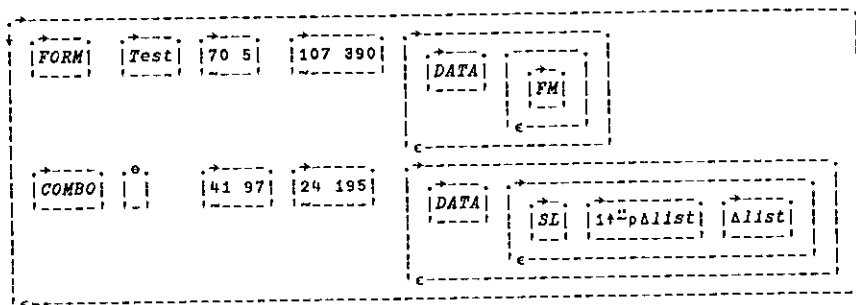
There are some problems in implementing this design. Dyadic have specifically designed the interface so that the frequency of use of the properties of an object defines the default sequence. This means that in majority of cases you do not have to specify the property names when creating a function with the properties defined as program constants. Also, each object has its own list of properties, for example the *FILE* property is used for a bitmap object but will cause a *DOMAIN ERROR* if applied to a combo box. All objects, though, may have a *DATA* property in which one can store any APL variable. Adrian has used this to store parameters which are special to the various objects.

Typing `Gui_dbx 'test'` presents the following screen:



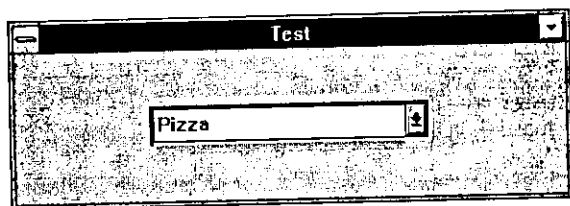
To add a control you click on the **Make** button and a menu of controls appears. When the control is on the form you can click on it and add the properties and APL expressions for the contents of list boxes, etc. The process is much the same as with *WDESIGN*, but without the fancy toolbar. (Having seen how the toolbar is created in *WDESIGN* it would be easy to add an equivalent facility.)

I added a *Combo Box* in my test and specified $\Delta list$ as the contents and $1+\rho\Delta list$ as the selection, and then clicked the OK button, which exits from the function. The variable *test* is defined as a 2-element nested array, with the first element specifying the form and the second element the combo box. *DISPLAY+test* shows the structure of the beast:



Prior to displaying the variable I changed the form title using $(1\ 2\rightarrow test)+ 'Test'$, so that the display would fit on the page.

Running *Gui_db test* produced the following window:



Adrian has added a sunken look to the combo box, and uses a grey background rather than the default colour that is specified in the Windows control panel.

The different design approach taken by Adrian is most interesting. I feel that holding the specifications in an array it is much more in line with the spirit of APL as an array language. The Dyadic approach results in lengthy functions with calls to $\square WC$ on every line. Adrian's code does, of course, have to have the same number of calls to $\square WC$, it is just that they are not so visible. I will probably use a combination of both methods when implementing an application.

Dyadic have used arrays to some extent in their *WDESIGN* workspace. The bitmaps for the toolbar and some other objects are held in an arrays. The function that creates the toolbar tacks on the properties required to the basic bitmap arrays and then calls `WC` to create the toolbar in one go. A similar approach has been used in several places in the *ARACHNID* patience game.

Workspace *GRAPHS*

The *GRAPHS* demonstration workspace contains the sort of functions that I would write to produce a graphics application. There are a couple of reasonably well generalised functions. *GRAPH* produces a graph from a nested array right argument which specifies most of the parameters required. *AXES* writes the axes using a number of global variables. If I was going to write a specialised graphics utility, rather than rely on the facilities available in other Windows products, I would start by using this workspace as a basis for development. No doubt by the time I had finished the functions would be entirely different, but at least I would have had a basis on which to start work.

I think Dyadic spoilt their demo by using some system fonts which are visually unattractive on the screen, with jagged edges when displayed in large font sizes. Don't be put off by this, the demo shows you how to use fonts, and it is a simple matter to replace the system fonts with True Type fonts which look much smarter.

Needless to say Adrian Smith has an entirely different approach in converting his existing graphics code. He sent me a workspace with some impressive demonstrations. His foundation APL code is based on converting PostScript instructions into Dyalog APL graphics. Adrian has been doing a lot of work with PostScript, and basing the core graphics engine on PostScript means that he can lift applications from one version of APL graphics to another by just writing a new set of basic functions. I would have to learn PostScript to be able to use Adrian's workspace, so for the moment I prefer to stick with Dyadic's method.

Manuals

I must get my beef about the manuals over first. All text is printed in a single typewriter-style font, so none of the normal methods of using different fonts for distinguishing different definitions are available. The font also makes the manuals physically much larger that they would have been with a proper proportionally spaced font. The User Guide does not have a Contents; each chapter starts with a Contents but there is no overall Contents. The User Guide

and Reference Manual have indexes, but there are few entries and they have not been constructed properly, so they are virtually useless.

It is a shame that the manuals are so poorly produced because they are actually quite well written. The description of each feature is clear, and the grouping of subjects is sensible.

Dyadic should buy themselves a decent word processor and get some professional help to make their manuals more attractive.

Problems

When writing this article I found it convenient to have Dyalog APL in one window and Microsoft Word for Windows in another. I found that I had to start Dyalog APL on entry to Windows otherwise it complained that there was not enough memory on my machine. It was no good starting Word for Windows and then closing it, I just got the same message. I can only assume that Dyalog APL requires a contiguous area of memory, and loading any other application causes memory to become fragmented.

Dyalog APL crashed four or five times whilst writing the review. It was most aggravating that I was unable to recreate the problems, so I was not able to tell Dyadic how to recreate the errors. Dyadic were most anxious to correct any errors before the final release of the product, and they tell me that several bugs have been found and corrected in the month that I have had the review copy prior to final release.

Conclusion

Dyalog APL have created an excellent environment for the development of Windows applications. Every feature that I could wish for has been included, and once I had got over the initial learning curve I found programming easy. The ability to access DLLs will, I am sure, prove to be an invaluable tool in linking APL with other applications, which I am looking forward to exploring further.

Business Graphics — A Second Look

by George MacLeod

This is a summary of the presentation given by George MacLeod at the BAA meeting in March 1993.

The presentation discussed MacLeod's views on business graphics, how his ideas had been implemented in the graphics part of the *Impetus* product and how these ideas had been further developed by prototyping a new business graphics package (code named *Trellis*) which will run under Windows and is being implemented in Dyalog APL/W. It is hoped to provide similar facilities under APL*PLUS II in the future.

The underlying principles of *Impetus* Graphics are threefold.

1. It must be easy to produce simple graphs.
2. It must be possible to produce elaborate graphs when needed.
3. The graphs when printed should be of a high quality.

The third item is based on the belief that most graphics programs concentrate on the presentation on the screen rather than on the printed graph. Although many executives use Personal Computers there is still a need to produce regular high quality printed graphical output for distribution to senior management.

There were considerable difficulties in implementing these aims in *Impetus* as the GSS graphics drivers supplied with most DOS-based APLs were not bug-free. This was particularly true when use was made of variable-pitch fonts. This resulted in graphs which could take several hours to print, and which were still incomplete. A paper on this implementation was presented to the BAA in 1991. Since that time there have been new releases of the graphics drivers and the problems are now resolved.

In the Windows world there is a wide variety of graphics applications programs, but most of them are really drawing programs that also have some ability to produce business graphics. The modern spreadsheet offers graphs tightly integrated into the spreadsheet data, and it is possible to produce graphs of considerable elaboration — although many of the features of a good business graphics system are not available.

Trellis graphics has the same underlying aims as did the earlier Impetus graphics but is intended to offer a more complete range of graph types and to take advantage of the Windows Graphical User Interface. As with Impetus graphics, the data and the graph properties are kept separate. Fundamentally Trellis works on three variables. The first is the **Graphics Description** which is a complicated nested variable containing many properties. The data is contained in a simpler nested variable known as the **Data Source**.

A Data Source may be used with more than one Graphics description, and more importantly, a Graphics Description can be used with more than one Data Source.

The third variable is the **Script**. This is an editable variable which is equivalent to a batch file in DOS. It uses a very simple and limited language, and facilitates the production of regular batches of graphs.

MacLeod's own introduction to business graphics came via the use of the Boeing Executive Information Services (EIS) product in the late 1970s. Boeing EIS runs on an IBM mainframe and has extensive business graphics facilities. The user has great control over the graphs that are produced, but the syntax of the command structure is complex and has to be learned and would be totally unacceptable on a modern PC.

At the heart of Boeing EIS is the recognition that most business and manufacturing data is time-related. For example, salaries may be paid monthly, VAT may be paid quarterly and rent every six months. A capital purchase is made at a point in time. We call this data "Time Spread" data and *Trellis* will support daily, weekly, monthly, quarterly, semi-annual and annual time spread data. Such data is defined by three properties. The periodicity (monthly, weekly, etc.), the start date (day, month and year) and a simple vector of values.

Time-spread data has a number of advantages. Firstly you do not have to provide the labels for the time dimension on a graph — the labelling is inherent in the data. Secondly the total duration of the data is clearly defined, for example, there is no ambiguity about any data before the start date. Thirdly it is possible to plot a graph of data with different periodicities, for example, monthly data can be plotted with quarterly data by automatically summing the months into quarters. Cumulated data is readily available to produce cumulative graphs, and finally it is possible to have a time window set to limit the data that will appear in a particular graph.

The underlying architecture of a Graphics Description is as follows:

```
Page Level Properties
  Graph Level Properties (Graph 1)
    Plot Properties (Plot 1)
    Plot Properties (Plot 2)
    Plot Properties (Plot 3)
    etc.
  Graph Level Properties (Graph 2)
    Plot Properties (Plot 1)
    Plot Properties (Plot 2)
    Plot Properties (Plot 3)
    etc.
  etc.
```

Page level properties are concerned with such things as should there be a box around the page, should there be a time stamp, what is the background colour, etc.

Graph level properties relate to matters such as graph type, description of the axes, grids, etc. These properties can be specified for each graph on a page.

Most graphs have more than one plot. For example a Line Graph will often show more than one line of Plot. Plot properties relate to such matters as line types, bar patterns, where the data comes from, etc. As it requires a large number of properties to draw a graph it is important that most properties take default values and are only changed when the user wishes to change them. Trellis will be delivered with a complete set of defaults for all of the graph types. When a Graphics Project is first created a complete set of the system defaults will be placed in it.

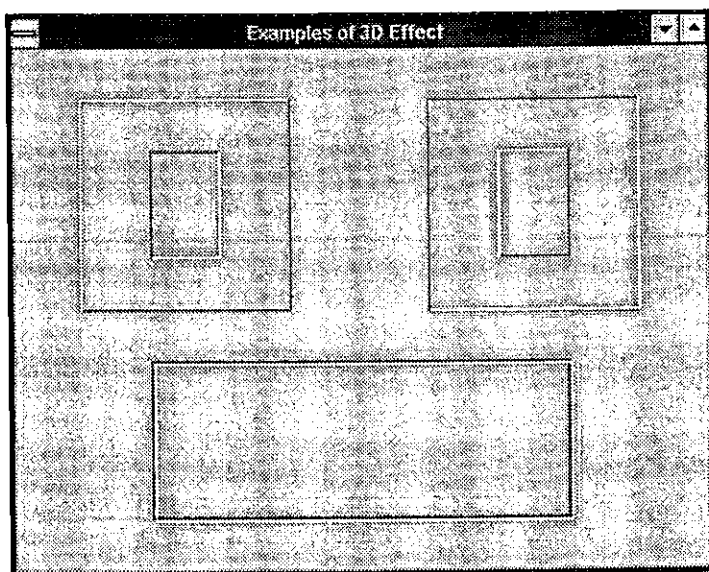
The concept of heredity will be used in much the same way as with Object Oriented Systems. A *Plot* is the child of a *Graph*, which in turn is the child of a *Page* which is the child of a *Project*. For example, if we are producing a line graph and we are about to draw the first line *Trellis* will look for the properties of the line at the Plot Level. If it finds nothing it will look at the Graph level, if it again finds nothing then it looks at the Page level and if nothing is specified there then it looks at the Project level, where everything is specified.

This means that properties for the first line to be drawn in a line graph may be found at a number of levels. Not all properties can be treated in this way. For example the location of a graph on the page has to be specified at the graph level. If it was specified at the page level then all graphs on a page would be superimposed. (It may be useful to define the size and aspect of a graph at the page level so that several graphs on a page will have the same size and aspect.)

Trellis will use the facilities of the Windows GUI to provide users with interactive facilities to enter data and to organise the "look" of a page of graphs. Provision will be made for the actual display and printing of graphs to be driven using Dynamic Data Exchange (DDE) from other Windows applications.

Most *Trellis* users will be unaware that *Trellis* is written in APL and most users will not have heard of APL! However, *Trellis* will be especially convenient for users writing application programs in Dyalog APL/W. They will be able to produce a Data Source in their program and then share this with *Trellis*. A definition of the structure of a Data Source will be available together with code that can be used to check that a Data Source produced by an application program is structurally correct.

Addendum: Getting that 3D Look



George showed the results of a number of experiments he had been making with the look and feel of the Windows GUI interface. One effect that was surprisingly effective, yet very easy to achieve was the generation of three dimensional effects in a window with a grey background. The example above shows a pairs of raised and recessed effects created by drawing white and dark grey lines on the light grey background.

K

by Arthur Whitney

Introduction

K is the executable notation at the heart of a high performance programming platform. It is designed for analyzing massive amounts of real-time and historical data — ideal for financial modelling.

K is fast, comprehensive, accessible and productive. It gets its speed from an incremental compiler and well-designed memory management, data types and primitives. It gets its universality by combining easily with other products and including high-performance built-in spreadsheet, graphics and database management. The productivity and accessibility come from the design of the core language.

This paper gives some examples written in K and a brief summary of the language. This work came about because of my experience in using C and Fortran and my work designing, implementing and using several different programming environments including LISP's, object-oriented systems and APL's. In particular, I learned a lot from designing, implementing and using the very successful A+ at Morgan Stanley. I am convinced that the best possible programming environment, at least for finance, is one that is based on high-performance list processing with APL-like primitives. This gives the best combination of productivity and performance.

Examples:

K has a small and powerful vocabulary. Every primitive operation is a single ASCII symbol. For example,

p: 1.5 1.6	p gets 1.5 and 1.6
q: 3 4	q gets 3 and 4
p*q	p times q
4.5 6.4	
+/p*q	plus OVER p times q
10.9	

The basic atoms of the language are characters, numbers, symbols and lambda-expressions. Lists are made up of items. Each item is a list or an atom. A table is a list of equal length lists. In mathematics instead of atom, list and table we say scalar, vector and matrix. For example,

s: 3 a scalar, i.e. an atom.
 v: 3 4 a vector, i.e. a list of 2 atoms.
 m: (2 3 4 a matrix, i.e. a list of 2 lists of 3 atoms.
 5 6 7)

Then,

+/v plus OVER items of v, i.e. 3 + 4.
 7
 +/m plus OVER items of m, i.e. 2 3 4 + 5 6 7.
 7 9 11
 +/'m plus OVER EACH item of m, i.e. (+/2 3 4;+/5 6 7).
 9 18
 +\v plus SCAN items of v, i.e. (3;3+4).
 3 7

+ and * are binary atomic functions. / \ and ' are operators. Operators, which are spelled in capitals in the commentary, take functions as arguments and derive related functions as results. E.g. + is plus and +/ is sum. In general,

f/ x applies f successively OVER the items of x.
 f\ x applies f successively SCANNing the items
 of x from beginning to end.
 f' x applies f to EACH of the items of x in
 parallel.

Also,

v+'v v plus EACH v, i.e. (3+3;4+4).
 6 8
 v<\v v less-than EACHLEFT v, i.e. (3<3 4;4<3 4).
 0 1
 0 0
 v</v v less-than EACHRIGHT v, (3 4<3;3 4<4).
 0 0
 1 0

The binary cases of the functions derived from / \ and ' are all parallel operations. In general,

x f' y applies f to EACH pair of the items of x and y.
 x f\ y applies f to EACH of the items on the
 LEFT(x) while keeping y fixed.
 x f/ y applies f to EACH of the items on the
 RIGHT(y) while keeping x fixed.

A function f is atomic if and only if f is equivalent to f' . For example,

$(v+v) \sim v+'v$ v plus v is-equivalent-to v plus EACH v .

Note,

$v*m$ v times m , i.e. $(3*2\ 3\ 4;4*5\ 6\ 7)$.

6 9 12

20 24 28

$+/v*m$ plus OVER v times m , i.e. $6\ 9\ 12+20\ 24\ 28$.

26 33 40

Composition arises from the juxtaposition of functions. For example,

$\sim=$ not equal.

$\sim<$ not less than, i.e. more than or equal.

$\sim>$ not more than, i.e. less than or equal.

$+/*$ plus OVER times — generalized dot product.

$(+/*)\backslash$ (plus OVER times) EACHLEFT — matrix product.

In general, for functions f and g :

$((f\ g)x) \sim f\ g\ x$ $(f\ \text{compose}\ g)$ of x is equivalent to f of g of x .

$(x(f\ g)y) \sim f\ x\ g\ y$ $x(f\ \text{compose}\ g)y$ is equivalent to f of $x\ g\ y$.

$((x\ f)y) \sim x\ f\ y$ $(x\ \text{with}\ f)$ of y is equivalent to $x\ f\ y$.

For example, from linear algebra we know that:

$s*$ is a linear transform from R to R .

$m(+/*)\backslash$ is a linear transform from R^2 to R^3 .

Of course these lists do not have to be rectangular nor of uniform type. Therefore lists are a simple yet powerful extension to the rectangular homogeneous arrays of the original APL. There is no need for enclose or disclose. Moreover, the parallel operators $/\backslash$ and $'$ simplify and generalize, and thus obviate, inner and outer products as well as the bracket-axis operator.

Another important generalization is making expressions first-class data objects and allowing them any number of arguments. The application of an expression to its arguments is done with brackets. For example, the fundamental calculation in finance is present value, i.e. price.

Given a list of cashflows c that come at corresponding times t with a prevailing discount rate d , then ...

$pv: \{[c; t; d] + /c * d \wedge t\}$ pv gets: sum of cashflow times d to the power t .

The discount rate is the reciprocal of 1 plus the interest rate, for example, a discount rate of 0.9 corresponds to approximately 11%, and means that 1 dollar one year from now is worth 90 cents today.

For example, a 3-year bond issued with an annual 10% coupon would have:

$c: 0.1 \ 0.1 \ 1.1$	cashflow is coupon, coupon and principal+coupon
$t: 1 \ 2 \ 3$	at years 1 2 and 3.

Then,

$pv[c; t; 0.9]$	present value of the bond at 0.9 annual discount
0.9729	is approximately 97 cents.

This formula is well-known and widely used. Unfortunately it is also inexact. It assumes a constant interest rate over the life of the bond. The right way to make this calculation is to use the current prices of all relevant financial instruments to derive a discount function $d[t]$ (read d of t). $d[t]$ is the present value of a dollar at all times t . So now the formula is:

$pv: \{[c; t; d] + /c * d[t]\}$ pv gets: sum of cashflow times discounts at times t .

$pv[c; t; ([t] \ 0.9 \wedge t)]$ call pv with c , t and lambda-expression for d .

0.9729

Instead of storing a function as an expression we could store the actual mapping itself. This can be much more efficient, e.g. in this case,

$d: 1 \ 0.9 \ 0.81 \ 0.729$ d gets discount for years 0,1,2 and 3.

$pv[c; t; d]$... and once again ...

0.9729

Note the pv definition didn't change. Function call and indexing use the same notation, i.e. if the left argument is a list then index — if it is an atomic lambda-expression then execute. This is a powerful idea. A list v is often a very effective representation of a function from 0 1 2 ... to v[0] v[1] v[2] For example,

```

v:1 2 3 4 5      define a list v, i.e. the range of v.
!#v              enumerate count v, i.e. the domain of v.
0 1 2 3 4
  v[3]           v at 3
4
f: {[x] 1+x}     define a functional superset of v.
f[3]             f at 3
4

```

It is possible to fix some arguments to a function just as it is possible to specify just some axes in indexing. In the following example m and f both implement + — m1 and f1 both implement 1+.

```

m:n+\n:14        m gets n plus EACHLEFT n gets enumerate 4,
                  i.e. m is defined to be the 4 by 4 addition table.
m
0 1 2 3
1 2 3 4
2 3 4 5
3 4 5 6
f: {[x;y] x+y}   f is defined to be addition.
m[1;2]           m at 1 2
3
f[1;2]           f at 1 2
3
m1:m[1]          m1 gets m at row 1.
m1
1 2 3 4

f1:f[1]          f1 gets f with x fixed at 1.
m1[2]            m1 at 2
3
f1[2]            f1 at 2
3

```

The [and] are for convenience. The underlying primitive is @.

Summary

K stands for "keys to the kingdom". The keys to expressiveness include:

0. simple and general atom and list data types.
1. simple and general atomic and list primitives.
2. a single ASCII symbol for each primitive.
3. the identification of list indexing with function call evaluation.
4. symbolic indexing (not described here).
5. lambda-expressions with any number of arguments.
6. function composition.

The keys to performance include:

0. constant time access, i.e. indexing.
1. constant time searching, i.e. hashing.
2. constant time append, i.e. clever memory management.
3. linear time sorting.
4. parallel functions and operators.
5. in general, having primitives that can express the important algorithms.
6. incremental compiler.

In summary, what is K? It is a powerful programming language. Why use K? For productivity and performance. What is new, different and exciting?

0. The lists of K are a simple superset of the rectangular arrays of APL.
1. Every primitive is a single ASCII symbol — no non-standard characters.
2. User-defined functions can have any number of arguments.
3. Indexing has cross-sectional, scatter and symbolic access.
4. Lambda-expressions are an atomic data type.
5. There is no branching — control is through iteration and parallel operators.
6. Bracket axis, inner product and outer product are all superseded by EACH.

Conclusion

K can run on any architecture but runs especially well on high-bandwidth RISC and parallel RISC. This is because of its emphasis on arrays and parallel operations on the arrays. The natural tendency in K to write parallel solutions to problems helps make K an ideal language for distributed and parallel processing

Windows BMP Bitmap Files and APL

by Ray Cannon

Introduction

I have been playing around with bitmap files and found that APL was an ideal vehicle for gaining an insight into the way bitmaps can be manipulated. One of the problems with the books and documentation about bitmaps is that they assume that you are going to use C, so the explanations and general thought process is limited by that language. Bitmaps are just matrices of numbers, so the power of APL array handling is a great help in understanding how to carry out any required transformations.

This article starts with an outline of the way in which Microsoft Windows device-independent bitmap files (BMP) are constructed, along with some colour-model theory. This is followed by examples of APL functions for image enhancement and manipulation including grey scaling, noise reduction, and edge detection.

Bitmaps

The term bitmap (according to the Windows User Guide) is "An image stored as an array of bits." Microsoft defined a standard for PC files containing device-independent bitmaps (DIB) in version 3.0 of Windows and has not changed for version 3.1. OS/2 bitmaps are very similar, and in fact many Windows applications can read either format. Files containing Device-Independent Bitmaps, including OS/2 files, normally have an extension of BMP or DIB. For example, the Windows file CHESS.BMP contains the famous flying chess pieces bitmap.

Previous versions of Windows defined only device-dependent bitmaps, and although it is possible, it is now not usual to store them in files.

Microsoft's BMP files are not the only bitmap graphic file standards. Others include ZSoft's Paintbrush (PCX), CompuServe's Graphics Interchange Format (GIF), Tagged Image File Format (TIFF), Truevision's TARGA Format (TGA), GEM image format (IMG) and the MacPaint format (PIC).

Device Support and Graphics

Windows is a graphics environment. The primary output device (the computer monitor) must be able to support graphics with at least 16 colours, or grey scales.

There are two main types of output graphic devices: those that draw pictures dot by dot are "raster devices" (laser printers, dot matrix printers, CRT displays and printing presses), and those that draw an image as a series of lines are "vector devices" (pen plotters and "etch-a-sketch" toys). In Windows, only devices with raster capabilities support bitmaps. Devices without raster capabilities do not support bitmaps. Some non-graphic devices, such as Daisy-wheel and Golf-ball printers, can in theory support raster graphics, via repeated use of the "." character, but I don't recommend trying it!

Windows does however support graphics output on vector devices via Metafiles. A Metafile contains the commands to draw an image, rather than a representation of the image itself. This is rather like Turtle graphic commands in LOGO. Metafiles are also supported on raster devices.

As a generalisation, bitmaps tend to be fast and big (require a lot of memory for storage) while Metafiles are small and slow. Bitmaps do not "scale" as well as Metafiles. A 45 degree diagonal line in a bitmap will become a staircase when enlarged, while a Metafiles line will remain a line. It is quite simple to convert a Metafile into a bitmap and this can be a quite useful technique in some instances. However, the reverse process, conversion from bitmap to Metafile, is "not trivial". "Not trivial" is a euphemism for "I don't know how do it!")

An alternative to bitmap graphics and Metafiles are "Block Graphics". Block graphics are produced by special graphic characters in text modes such as those used on TeleText and also mode 7 on the BBC micro computer. Block graphics are not in general supported by Windows.

A Little about Colour Models

A little background about eyes and colour models might come in handy here. In the retina, there are two types of light sensitive cells: rods and cones.

Rods are very sensitive to light, but do not distinguish colour. Cones on the other hand are far less sensitive, but can distinguish colours. Hence, under low light conditions when only the rods are active, objects appear as black, white or grey.

The cones occur in 3 types (red, green, and blue) each of which has a maximum response to a different wavelength (colour) of visible light. The colour you

perceive depends on the relative response of each type of cone to the wavelength of light received by it. For example, yellow light stimulates both the red and green cones. By the same process, a red and a green light together will appear as yellow. This is an additive process.

However the normal eye's colour response is not linear; its strongest response is to green (59%) then red (30%) and lowest to blue (11%). Short wavelengths are blue (or violet) and long wavelengths are oranges and reds. Black is the absence of any wavelength in the visible spectrum. An even mix of the spectrum colours is perceived as white light. White light can also be produced by mixing 3 primary colours such as Red, Green and Blue (RGB) or Cyan, Magenta and Yellow, (CMY) or two complementary colours such as Red and Cyan, or Green and Magenta, or Blue and Yellow.

To an artist using pigments, the three primary colours are red, blue and yellow, which when mixed together in equal proportions produce black. This is because pigments work by light absorption, and only reflect the colours they do not absorb. This is a "subtractive" process, black being the result because all the colours are absorbed.

Modern colour printing tends to use a four-pigment process, often Cyan, Magenta, Yellow and Black (CMYK; "K" stands for key, a printing term for black). The black is required due to practical limitations in ink pigmentations rather than on theoretical grounds. White, on the other hand, is reproduced by not putting any pigment on the surface medium, white paper.

Just as a pixel's colour can be adjusted by varying the mixture of primary colours used to make it up, the apparent colour of an area covering a group of pixels can be simulated by making the group up of a mixture of different colours. This process is known as *dithering*, and is widely used in Windows. To work, dithering depends on the eye's inability to resolve the individual dots.

Colours possess *Luminosity*, *Hue* and *Saturation*. Luminosity is the brightness (or intensity) of the light. Hue is the wavelength of the light. Saturation is degree to which the colour departs from a pure spectral colour and approaches white. A pure continuous spectrum shows a continuous variation of saturated hues. When a hue is diluted with white light, the colour is classified as a tint, thus Pink is a tint.

Grey can be regarded as a tint when the hue being diluted has zero intensity. Grey is in effect low intensity white. You can't have a "grey light", but white pigments can look grey in low light conditions.

Windows stores colours via a RGB model, with the intensity of each primary colour in the range 0 to 255. Thus a colour requires 24 bits to define it, and the model supports $256 \times 256 \times 256$ (about 16.7 million) colours. Actually, the standard VGA device can only resolve 64 intensity levels of any one RGB colour.

Device-Independent Bitmaps

Microsoft's Windows bitmaps exist in 2 distinct forms, Device-Compatible bitmaps, and Device-Independent bitmaps (DIB). Since device-compatible bitmaps of the same image will vary from one device to another, they are not very suitable for program manipulation via general utility code. A different utility would be needed for each device. The rest of this article, therefore, will deal with DIBs only.

The simplest form of the bitmap is the monochrome bitmap, where each pixel is represented by a single bit. Coloured bitmaps require multiple bits per pixel. The most common Windows bitmaps are its icons. The famous arrow and hourglass cursors are also modified forms of bitmap. Icons and cursors differ from standard bitmaps because they can contain transparent areas.

The Windows 3.0/3.1 Device-Independent Bitmap (DIB) format

Windows supports 4 different levels of colour in its DIB format:

- 2 colours. The so-called "monochrome" being 1 colour bit per pixel;
- 16 colours. 4 bits per pixel;
- 256 colours. 8 bits per pixel; 16.7 million colours. 24 bits per pixel, that is 8 bits for each of Red, Green and Blue.

The colour level defines how many colours can be selected from the 16.7 million possible colours. For example, a 256-colour bitmap of a rain forest might use 200 different green tones. The set of colours used in a bitmap are called its colour palette.

The format of a BMP file consists of 4 distinct parts, the file header, the bitmap header, the colour palette and the bitmap bits:

1. The bitmap file header

Ofs	Size	Microsoft name	Description
0	2	bfType	The character BM for bitmap
2	4	bfSize	The total size of the file
6	4	bfReserved1/2	Reserved, must be set to zero
10	4	bfOffBits	The offset to the bitmap bits from the beginning of the file.

2. The bitmap information header

Ofs	Size	Microsoft name	Description
0	4	biSize	The size of this structure in bytes
4	4	biWidth	The width of the bitmap in pixels
8	4	biHeight	The height of the bitmap in pixels
12	2	biPlanes	Number of colour planes, set to 1
14	2	biBitCount	Colour bits per pixel. 1 4 8 or 24
16	4	biCompression	*Code for the compression scheme
20	4	biSizeImage	*Size of the bitmap bits in bytes
24	4	biXPelsPerMeter	*Horizontal resolution in pixels per meter
28	4	biYPelsPerMeter	*Vertical resolution in pixels per meter
32	4	biClrUsed	*Number of colours defined in the palette
36	4	biClrImportant	*Number of important colours in the image.

The biSize is 40 for Windows DIBs and 12 for OS/2 DIBs and provides the simplest way of distinguishing them. The colour level is defined by the biBitCount value.

Items marked with a * are not mandatory and may often be set to zero, meaning that the default value applies.

3. The RGB palette (not used in 24-bit DIBs) starts at file offset 14+biSize and consists of 2 or more RGBQuad entries each of which consists of a 4-byte entry:

Ofs	Size	Microsoft name	Description
0	1	rgbBlue	1st Blue intensity in the range 0-255
1	1	rgbGreen	1st Green intensity in the range 0-255
2	1	rgbRed	1st Red intensity in the range 0-255
3	1	rgbReserved	Set to 0

A Windows DIB may have fewer colours defined in the palette than the maximum number suggested by the biBitCount; if so biClrUsed must define the number of entries.

OS/2 DIBs use an RGB triplet, a 3-byte entry for each palette colour.

4. The bitmap bits start at file offset `bfOffBits`. The bits (1, 4, 8 or 24 per pixel) form an array, starting at the bottom row of pixels, each row beginning with the leftmost pixel. Within each byte, the most significant bits are used before the least significant bits. Each row of the bitmap uses a multiple of 4 bytes, and may be padded to the right to ensure this. OS/2 and Windows share the same bitmap bits format.

With all but the 24-bit colour bitmaps, the bits making up the image do not actually define the colour of the pixel, but are an index into the palette of 24-bit RGB colours.

Bitmaps take up a lot of space, but can often be compressed by encoding repeated information. Some encoding algorithms are very complex, for example the JPEG algorithm, therefore they are slow to unpack, but take up much less space and require less disk I/O. DIBs support a much simpler form of run-length encoding. Since compression and expansion of bitmaps is best carried out in a compiled language, it is not discussed further.

How you get your bitmap information into and out of APL will depend on the APL you are using. The following example functions were written using STSC Pocket APL.

To allow easy cross-reference between elements of APL variables and elements of Windows structures, the function *DEFINE* sets up many globals with Microsoft type names.

```

▽ DEFINE;ⓘIO
[1]  ⍝ DEFINE GLOBAL CONSTANTS
[2]  ⓘIO←0
[3]  ⍝ ALL INDEX VALUES ASSUME ⓘIO←→0
[4]  ⍝ BIT MAP FILE HEADER VECTOR BF
[5]  BFSIZE←0 ⍝ BFRESERVED←1 ⍝ BFOFFBITS←2
[6]  ⍝ BIT MAP INFO HEADER VECTOR BI
[7]  BISIZE←0 ⍝ BIWIDTH←1 ⍝ BIHEIGHT←2 ⍝ BIPLANES←3 ⍝ BIBITCOUNT←4
[8]  BICOMPRESSION←5 ⍝ BISIZEIMAGE←6 ⍝ BIXPELSPERMETER←7 ⍝ BIYPELSPERMETER←8
[9]  BICLRUSED←9 ⍝ BICLRIMPORTANT←10
[10] ⍝ PALETTE RGB QUAD BP
[11] BPBLUE←0 ⍝ BPGREEN←2 ⍝ BPRED←2
[12] ⍝ OUTPUT GRAY PALETTE LOOKUP
[13] PALETTEOUT← 256 4 0
[14] PALETTEOUT[;0]←;256 ⍝ CLRUSED=256
[15] PALETTEOUT[;1]←;4/;64 ⍝ CLRUSED=64
[16] PALETTEOUT[;2]←;16/;16 ⍝ CLRUSED=16
[17] PALETTEOUT[;3]←;128/;2 ⍝ CLRUSED=2
[18] ⍝ OTHER SIZES
[19] FILEHEADSIZE←14 ⍝ INFOHEADSIZE←40
[20] ⍝ MONO GRAY SCALE
[21] MONO←GRAY2
▽

```

Separate functions are used for reading and writing the various components of the bitmap using global variables. If nested arrays were available then more elegant functions would be possible.

```

V READBITMAPHEADER TIN
[1]  R READ A BIT MAP FILE HEADER, INFO HEADER, AND PALETTE
[2]  BF←READBITMAPFILEHEADER TIN
[3]  BI←READBITMAPINFOHEADER TIN
[4]  BP←READBITMAPPALETTE TIN
V

V WRITEBITMAPHEADER TOUT
[1]  R WRITE A BIT MAP FILE HEADER, INFO HEADER AND PALETTE
[2]  OBF WRITEBITMAPFILEHEADER TOUT
[3]  OBI WRITEBITMAPINFOHEADER TOUT
[4]  OBP WRITEBITMAPPALETTE TOUT
V

V R←READBITMAPFILEHEADER TIE;BUFFER;IO
[1]  R READ THE FILE HEADER PART OF A BIT MAP, RETURNS
[2]  IO←0
[3]  BUFFER←INREAD TIE,82,FILEHEADSIZE,0
[4]  IERROR(¬A/'BM'=2+BUFFER)/NOT A BIT MAP FILE'
[5]  R←256+IAV[4 3 4 p2+BUFFER]
V

V BF WRITEBITMAPFILEHEADER TIE;BUFFER;IO
[1]  R WRITE OUT A BITMAP FILE HEADER BF
[2]  IO←0
[3]  BUFFER←'BM',,4IAV[4 256 256 256 256 TBF]
[4]  BUFFER INAPPEND TIE
V

V R←READBITMAPINFOHEADER TIE;BUFFER;P;IO
[1]  R READ THE BM INFO HEADER, RETURN VECTOR OF VALUES.
[2]  IO←0
[3]  BUFFER←INREAD TIE,82,INFOHEADSIZE,FILEHEADSIZE
[4]  R EXPAND 'WORD' VALUES TO 'DWORD' (BIPLANES AND BIBITCOUNT)
[5]  P←(12p1), 1 1 0 0 1 1 0 0 ,24p1
[6]  BUFFER←P\BUFFER
[7]  BUFFER[14 15 18 19]←IAV[0]
[8]  R CONVERT EACH 4 CHARS INTO AN INTEGER VALUE
[9]  R←256+IAV[4 11 4 pBUFFER]
[10] IERROR(R[BISIZE]*INFOHEADSIZE)/'SORRY, OS/2 BIT BITMAP NOT SUPPORTED'
[11] IERROR(R[BIBITCOUNT]=24)/'SORRY, 24 BIT BITMAP NOT SUPPORTED'
[12] IERROR(R[BICOMPRESSION]≠0)/'SORRY, COMPRESSED BITMAP NOT SUPPORTED'
[13] R SET UP GLOBAL DEFINING THE LENGTH OF A BITMAP ROW
[14] LINELENGTH←4*(R[BIWIDTH]*R[BIBITCOUNT])+32
V

V BI WRITEBITMAPINFOHEADER TIE;IO;BUFFER
[1]  R WRITE OUT BIT MAP INFO HEADER BI TO FILE TIED TO TIE
[2]  IO←0
[3]  BUFFER←,4IAV[4 256 256 256 256 TBI]
[4]  BO←(12p1), 1 1 0 0 1 1 0 0 ,24p1
[5]  BUFFER←BO/BUFFER
[6]  BUFFER INAPPEND TIE
V

```



```

V R+READBITMAPPALETTE TIE;NOC;MAX;DATA;IO
[1] RETURN THE PALETTE READ FROM BM FILE TIED, USE BI-GLOBALS
[2] IO+0
[3] R+ 0 0 0
[4] -(BI[BIBITCOUNT]=24)/0 R 24 BIT COLOURS, DONT USE PALETTE
[5] NOC+2*BI[BIBITCOUNT]
[6] MAX+1/NOC, (BI[BICLRUSED]*0)/BI[BICLRUSED]
[7] DATA+INREAD TIE,82,(MAX*4),BI[BISIZE]+FILEHEADSIZE
[8] R+(NOC,4)+(MAX,4)p[AV;DATA]
V

V BP WRITEBITMAPPALETTE TIE;IO;BUFFER
[1] WRITE OUT PALETTE BP TO TIED FILE
[2] IO+0
[3] BUFFER+,[AV[255[BP]
[4] BUFFER [NAPPEND TIE
V

```

Pocket APL has a very limited workspace size so I have arranged for each line of the bitmap to be read in and processed separately.

```

V R+N READBITMAPLINE TIE;IO
[1] READ LINE NUMBER N FROM BITMAP FILE TIED TO TIE
[2] IO+0
[3] R+INREAD TIE,82,LINELENGTH,BF[BFOFFBITS]+LINELENGTH*N-1
[4] R-BI[BIBITCOUNT]UNPACK [AV;R
[5] R+BP[R;0] RETURN GRAY COLOUR VALUE
V

V LINE WRITEBITMAPLINE TIE
[1] WRITE A SCAN LINE OUT TO TIED FILE
[2] LINE+CLRUED PACKLINE LINE
[3] LINE [NAPPEND TIE
V

V R+BC UNPACK DATA;IO
[1] UNPACK MULTIPLE PIXELS PER BYTE IF DATA IS PACKED
[2] +(BC= 1 4 8)/L1,L4,L8
[3] IO+0
[4] L1:R+,[ 2 2 2 2 2 2 2 DATA 0 +0
[5] L4:R+,[ 16 16 DATA 0 +0
[6] L8:R+DATA
V

V R+CLRS PACKLINE LINE;IO
[1] PACK MULTIPLE PIXELS PER BYTE
[2] IO+0
[3] +('M'=1+TYPE)/L1
[4] LINE+PALETTEOUT[0[255[LINE;CLRCOL]
[5] +(CLRS= 2 16 64 256)/L2,L16,L64,L256
[6] [ERROR(vCLRS),' COLOURS NOT SUPPORTED'
[7] L1:R+16,[((pLINE)+2),2)pLINE 0 +ALL
[8] L2:R+21,[((pLINE)+8),8)pLINE 0 +ALL
[9] L16:R+16,[((pLINE)+2),2)pLINE 0 +ALL
[10] L64:
[11] L256:R+LINE
[12] ALL:R+OLINELENGTH+[AV[R, 0 0 0]
V

```

Grey Scaling

The hardware in the standard VGA card can convert coloured images using a process known as "sum to grey" for display on a monochrome screen. However when sending an image to an ordinary printer, there is no hardware which is equivalent to the VGA card. Windows has to perform this task for us, which it does via the printer device driver.

Under some circumstances, it may be useful for us to do our own conversion to grey scales; for instance, processing a grey image requires one third of the work that processing a full colour image would require.

Converting a set of RGB intensities into a grey scale has to take into account the fact that the human eye's colour response is not linear. Each colour component must be multiplied by a weight before being added together to produce the appropriate grey level. This colour weighting I have stored in the global *RGBRATIO*.

The function *PALETTE2GRAY* (below) will do this task. Note that with 2, 16 and 256-colour bitmaps, only the palette needs to be converted. With 24-bit bitmaps, each pixel in the bitmap requires conversion.

```

V R+PALETTE2GRAY OLD;NOC;IO
[1]  R CONVERT OLD COLOUR PALETTE TO GRAY PALETTE
[2]  IO+0
[3]  OLD+(256/3)*OLD R      +10 CONTRAST OLD
[4]  OLD*CONTRAST ADJCONTRAST OLD
[5]  NOC+1+OLD R NUMBER OF PALETTE ENTRIES (COLOURS)
[6]  R+(0.5+1/OLD)*(OLD)*RGBRATIO
[7]  R+(OLD)+(NOC/3)*R(NOC/3)/R
V

```

A pretty coloured picture of a bridge at dusk can be copied from a disk distributed with "The Programmer's Introduction to Windows 3.1" by Brian Myers and Chris Doner (Sybex Inc 1992). The 256 colours in the palette consist of a series of sepia tones ranging from dark brown to sunset reds. This DIB will be used as the source image in many of the examples that follow.

Printing the original bitmap on an HP LaserJet direct from Windows Paintbrush, and printing the bitmap after *PALETTE2GRAY* conversion produced an apparently identical result.



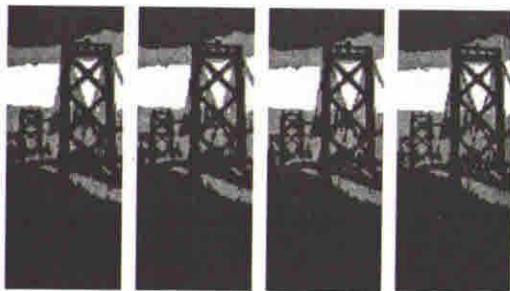
Simplifying Over-complex Images

It may be necessary to simplify overly complex images to help the problem of edge detection, for example when converting an image into a map. Also, bitmaps using "hard coded" dithered colours do not scale well, and it can help to replace dithered colours by absolute colours.

Data averaging is the easiest simplifying method. This normally involves replacing a point with the weighted average of the point in the surrounding area (often 3×3). The weights are given in a 3×3 mask, and a variety of masks can be used to gain different effects.

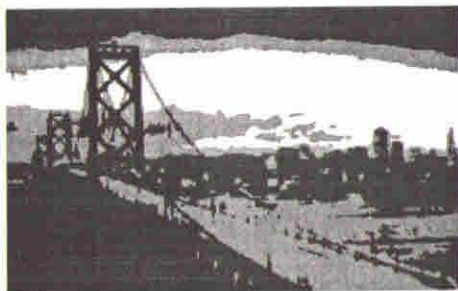
In the function *RUN*, the function *PROCESSLINE* performs the process one scan line at a time. For an environment with large workspaces the whole bitmap could be processed in one go.

Bitmaps AV0.BMP, AV1.BMP, AV4.BMP, AV8.BMP use 3×3 reshape of masks 1 1 1 0 1 1 1 1, 1 1 1 1 1 1 1 1, 1 1 1 1 4 1 1 1, and 1 1 1 1 8 1 1 1 respectively.

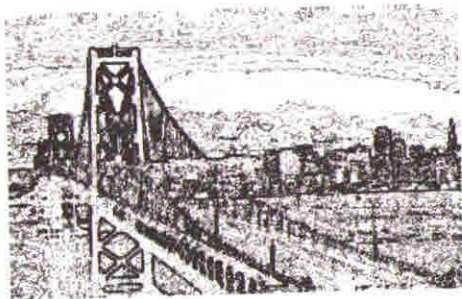


Low and High Pass Filters

The more complex "Low pass filter mask" 1 2 1 2 4 2 1 2 1 is said to produce a good approximation of "two-dimensional Gaussian weighting". This is very useful for "noise smoothing and interpolation of missing or damaged data elements".



The "High pass filter mask" -1 -2 -1 -2 12 -2 -1 -2 -1 has the opposite effect and might be used for enhancing contrast and detecting edges. In the example I have also inverted the image.



There are several other high pass filter masks which are worth trying, for example -1 -2 -1 -2 16 -2 -1 -2 -1 and -2 -3 -2 -3 20 -3 -2 -3 -2.

Converting Images to Maps

The process of transforming an image into a map is primarily one of edge detection. Adrian Smith used this process in building a TrueType APL font, see Vector 9.4 page 138.

One technique for edge detection is similar to the above filter, but rather than using one symmetrical mask, 4 highly directional masks are used, and the results are then combined, as shown in *PROCESSMAP*. The final image is converted into a monochrome bitmap, using function *CTOM*, by comparing the result of the transformation against the original value, rather than comparing against an absolute value.

```

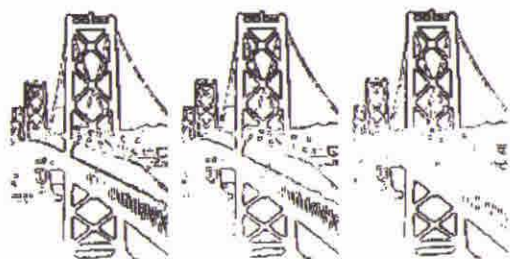
V R←PROCESSMAP MAP;□IO
[1]  #FIND BOUNDIRES
[2]  □IO←1
[3]  R←0 PROCESSLINE MAP[1 2 3 4 5 6 7 8 9]
[4]  R←R[0 PROCESSLINE MAP[4 1 2 7 5 3 8 9 6]
[5]  R←R[0 PROCESSLINE MAP[7 4 1 8 5 2 9 6 3]
[6]  R←R[0 PROCESSLINE MAP[8 7 4 9 5 1 6 3 2]
[7]  R←LTHIS CTOM R
V

```

The following examples show the effects of two transformation masks. *SMOOTH.BMP* was produced with the mask -1 -1 -1 0 0 0 1 1 1, and *SOBEL.BMP* with -1 -2 -1 0 0 0 1 2 1.

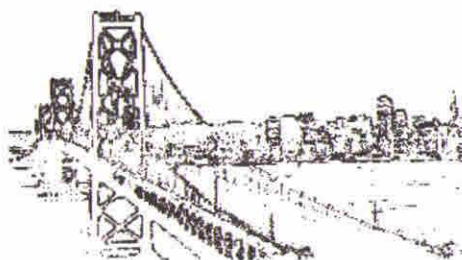


Both transformations have lost the right hand side of the bridge. If you look back at *GRAY.BMP*, you can see why; the river and road are almost the same grey in this area. In an attempt to recover this information, I then tried running the "smooth" mask on individual colour components to produce the bitmaps *RED.BMP*, *GREEN.BMP* and *BLUE.BMP*. This involved resetting the global *RGBRATIO*. *BLUE.BMP* picks up this edge the best.



An alternative mapping algorithm (the Isotropic transform) uses a similar set of masks to the SOBEL, but with the 2 replaced by the square root of 2. However since floating-point operations are so slow compared to integer ones, integer masks with 1 replaced by 128 and the square root of 2 by 181 can be used with the final result divided by 128.

Finally I've included the Laplace transformation. Unlike the Smooth, Sobel and Isotropic, this uses a single symmetrical mask of the form 0 -1 0 -1 4 -1 0 -1 0, or -1 -1 -1 -1 8 -1 -1 -1 -1, or 1 -2 1 -2 4 -2 1 -2 1. Note that the mask's values sum to zero, unlike the High pass filter, which it resembles. The transformation is said to be good at outlining areas of constant intensity in photographs. The example LAPLACE.BMP uses the 2nd of these masks.



Grey Scales to Monochrome

Some printers do not support grey-scale well, so it may be necessary to convert the image to monochrome. A 16 grey-scale bitmap can be converted into a monochrome image, with each grey pixel in the original replaced by a 4x4

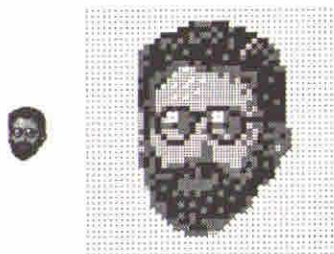
matrix (cell) of bits in the output. By varying the ratio of black dots to white dots in each cell, the grey scale can be reproduced, albeit in a much larger bitmap.

The spatial resolution of a graphics printer is normally much higher than that of a monitor. For example, a 14-inch screen is approximately 10 by 6 inches with 640 by 480 pixels in standard VGA. On a 300 dots per inch printer this results in an image of 2.1 by 1.6 inches. Thus it follows that a 16 grey-scale 640 by 480 bitmap could be converted into a monochrome 2560 by 1920 image and printed, resulting in an 8.5 by 6.4 inch picture.

Two new process are involved here, first generating a set of grey masks, and second repacking the expanded bits into a monochrome bitmap.

Now comes the question: how do you select the black dots for each level of grey? Selecting the bits in a too-regular fashion may produce unintended patterns. Most systems use hard-coded patterns. I feel that an algorithm could be written to produce an expansion mask suitable for the resolution and aspect ratio of any printer, on demand.

The function *GRAYSCALE* was my first attempt at producing such a monochrome expansion mask, and its results can be seen in the conversion of *AUTHOR.BMP* into *GRAYME.BMP*.



This algorithm is not entirely satisfactory, and I am sure that there are better methods. One possibility might be to use the famous N-Queens problem as a pattern to be used in spreading out the dots, but I have not yet had time to investigate this.

Graham Parkhouse in his article "APL Graphics – First Principles" (Vector Vol.7 No.4 page 83) describes a method to produce a 64-level greyscale via an 8×8 mask of the values 1 to 64. *DOUBLE* is a function that can be used to reproduce equivalent masks for 4, 16, 64, 256, 1024 ... levels of grey:

```

V R←DOUBLE MASK;ROWS;COLS
[1]  ⍺Expand MASK by factor of 2
[2]  ROWS←2×1+⍺MASK
[3]  COLS←2×1+⍺MASK
[4]  R←(COLS⍥1 0)∖(ROWS⍥1 0)∖MASK
[5]  R←R+1⍥1⍥R+(R≠0)×[/,R
[6]  R←R+(ROWS⍥1 1)⍥R+(R≠0)×[/,R
V

```

DOUBLE 1 1⍥1

```

1 3
4 2

```

DOUBLE DOUBLE 1 1⍥1

```

1 9 3 11
13 5 15 7
4 12 2 10
16 8 14 6

```

DOUBLE DOUBLE DOUBLE 1 1⍥1

```

1 33 9 41 3 35 11 43
49 17 57 25 51 19 59 27
13 45 5 37 15 47 7 39
61 29 53 21 63 31 55 23
4 36 12 44 2 34 10 42
52 20 60 28 50 18 58 26
16 48 8 40 14 46 6 38
64 32 56 24 62 30 54 22

```

The resulting 16-level greyscale has been used in producing the image MONO.BMP.



Brightness and Contrast

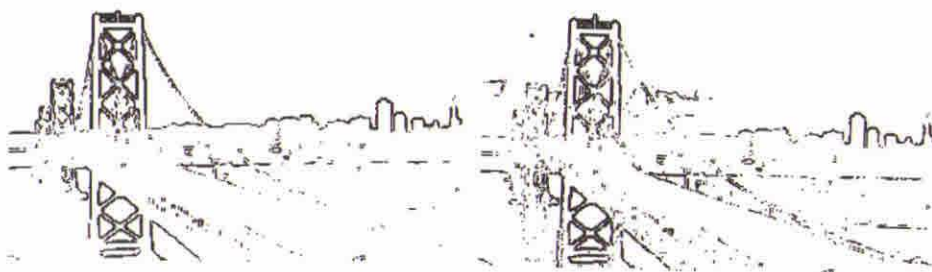
With 16 and 256 colour bitmaps, the brightness and colour contrast can be easily adjusted by varying the colour palette. The brightness of an image can be increased by increasing the palette colour values. The ratio of red to green to blue in any one colour should remain the same. Colour contrast on the other hand can be changed by varying the ratios of red, green and blue in each RGB quad entry to emphasise the predominant primary colour.

```

V R←PC ADJCONTRAST RGB;RGBMIN;RGBMAX
[1]  Adjust colour contrast in palette RGB by PC percent
[2]  R←RGB
[3]  →(PC=0)/0  Exit if zero adjustment requested
[4]  RGB←(256 3 ⌊pRGB)+RGB
[5]  PC←⌊0.5+PC×RGB÷100
[6]  RGBMIN←PC×RGB=(pRGB)ρ3/⌊/RGB+256×RGB=0
[7]  RGBMAX←PC×RGB=(pRGB)ρ3/⌈/RGB+256×RGB=0
[8]  R←(pR)+0⌈255⌊RGB+RGBMAX-RGBMIN
V

```

Due to images in VECTOR articles being printed in grey, I am a bit restricted in showing examples; however compare bitmaps BLUE.BMP and CONTRAST.BMP. They differ only in that in CONTRAST.BMP a 15% contrast has been applied to the incoming bitmap.



Notes on the APL

The code was written using STSC Pocket APL but should run on almost any APL if the workspace size is big enough. The native file quad-functions will need to be replaced with the equivalent functions which can read and write ordinary DOS files. The code does not need to run under Windows, but a bitmap viewer is required to see the results.

A function *GO* was written to test the code and produce all the bitmaps shown in this article, and is not meant to be an "end user" function. With Pocket APL on a 20MHz 386, *GO* takes about 40 minutes to run. In the text I have talked about and shown 3x3 masks, but in the code I have actually used the masks in the ravelled form as a nine-element vector.

Although not used for the examples above, the function *ROTATE* will rotate a SQUARE matrix through multiples of 45 degrees. *ROTATE* is a bit of an over-kill for rotating a 3x3 mask, but more advanced processes require 5x5, 7x7 or even larger masks to be rotated. Applying *ROTATE* on the bits of a square bitmap can also produce interesting effects.

```

      V R←T ROTATE MAT;N;I;MAX;N1;S
[1]  R←ROTATE SQUARE MATRIX BY T×45 DEGREES
[2]  R←MAT
[3]  N←(ρMAT)[1]
[4]  →(N<3)/0
[5]  N1←N-1
[6]  S←-{0.5×N1
[7]  M4←(4,N)ρMAT[1;],MAT[;N],(ϕMAT[N;]),ϕMAT[;1]
[8]  M4← 0 1 ↗M4
[9]  M4←(4,N1)ρ(S×T)ϕ,M4
[10] M4←(¬1ϕ(ϕM4)[;1]),M4
[11] R[1;]←M4[1;]
[12] R[;N]←M4[2;]
[13] R[N;]←ϕM4[3;]
[14] R[;1]←ϕM4[4;]
[15] S←1+1N1
[16] R[S;S]←T ROTATE R[S;S]
      V

```

ROTATE is quite crude compared with Morten Kromberg's winning entry from the APL86 matrix rotation competition, which was published in Vector 3.2 page 100. This solution did 90-degree rotations on square and non-square matrices, and each shell could be rotated by different amounts and in different directions! I have not yet tried to amend this to suit my requirement for 45-degree rotations on a square matrix, but I feel sure that it should be possible. Can anyone help?

Conclusion

Un-compressed Microsoft device-independent bitmaps are relatively simple and can be usefully and easily manipulated within APL workspaces. APL's interactive environment offers an excellent platform which allows the programmer to experiment with different image-enhancement algorithms.

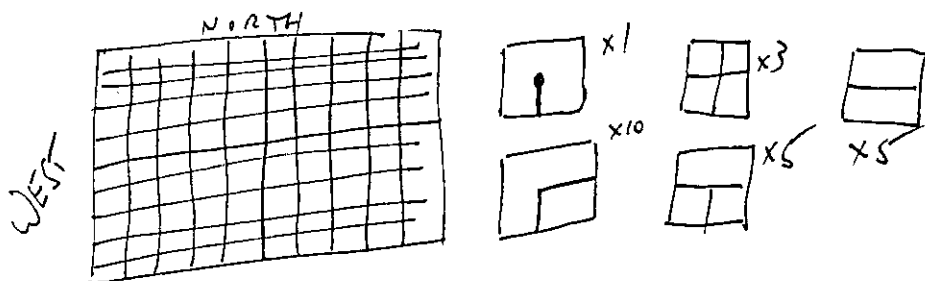
Coast-to-Coast — Designing with Objects

by Adrian Smith

Historical Note

Once upon a time when the world was young, I shared a house with a mad computer programmer called Dai Lowe. Dai was (among other things) a games fanatic, and had once come across a board game called 'Terminator'; this was no longer obtainable in the shops, but we made ourselves a set and had a good deal of fun trying to understand the strategy and tactics.

I lost touch with Dai some 10 years ago, but never totally forgot the principles of the game ... accordingly when I was looking for something 'a little different' to test out the more obscure corners of the Dyalog APL GUI, I re-activated some long-dormant brain cells and quickly sketched out the basic design:



As you can see, the game is played on a 10-by-10 board (although other variants would be just as good, and clearly with Windows a 'soft' design would be preferable) with a set of tiles, each of which has drawn on it a 'path' linking one or more edges. Again, the number of tiles could be varied, although the piece distribution shown here seems to give a nicely balanced game.

The game is for two players, one of whom plays North-South and the other East-West. The winner is the first to complete a continuous path joining his two opposite edges — of course once a piece is played the path on it counts for both players! Games can be drawn, as you could easily complete your opponent's path in the very act of completing your own.

Prologue

I am getting increasingly bothered by the growing conflict between the existing APL design paradigm (based firmly on the *Workspace*, containing *functions* and *arrays*) and the newer object models (as partially implemented in Visual Basic, PowerBuilder, and the Dyalog *WDESIGN* workspace). I instinctively shy away from anything which buries code where it is hard to find it, and which makes the 'structure' of the application less than obvious to anyone other than the developer. The great advantage of the APL workspace is that everything you need to know about the application is there ... and if you can't find it quickly you can write yourself some utilities which can!

Under the Visual Basic design paradigm, code is attached to objects and the application structure is implied by the events which trigger that code. As Duncan Pearson put it in Vector 9.4 "How do you, as a developer starting to maintain someone else's broken system, know what the hell does what, with what, when?". I like my application structure explicit, my code where I can find it, and I want my events logged where I can see who fired them, with what, when! Applying the first rule of amateur astronomy:

"It takes less time to grind a 3" mirror, then a 6" mirror, than it takes to grind a 6" mirror"

... I started work on a 3" mirror. The game described above seemed ideal as:

- it clearly lends itself well to an object-based design (each tile is an object, with certain obvious properties such as its rotation, topology etc.).
- the whole problem is well-bounded, and could produce an interesting end-product in a short space of time.
- I wanted to have some fun with the visual design — for example I was keen to make a nice 3D board with etched grooves, and I wanted to make the tiles stand well, snap nicely into place, and show the partially completed path as clearly as possible.
- there is just enough complexity to make it a worthwhile test of my (at this point very sketchy) ideas for managing objects within the traditional APL workspace.

I hope that along the way I can leave behind some useful code which may help you to:

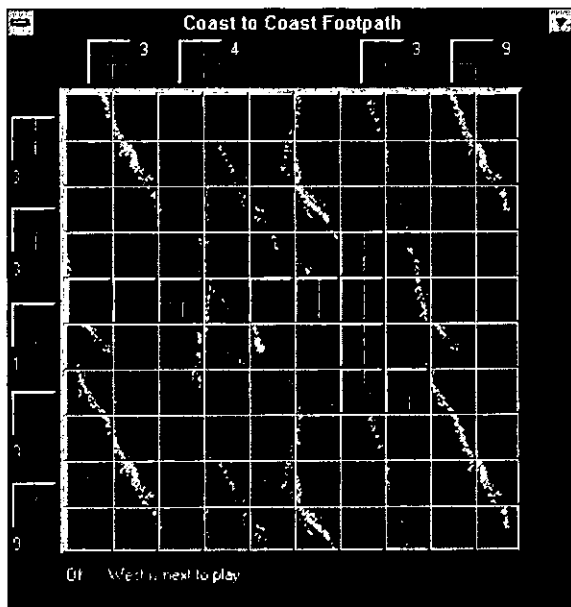
- make nice 3D bitmaps 'on the fly' without the need to draw them by hand in Paintbrush, and waste vast amounts of workspace storing them as bit-patterns.

- manage the Dyalog 'DATA' property in a simple and transparent way.
- manage and trace the relationship between objects, events and code in a highly visible and effective way.

All suggestions and comments welcome ... it is my intention to get this code 'up to scratch' in time for the November BAA meeting, and any input from the Windows APL community will be of enormous help to me in achieving this.

The Basics — Setting up the Board

On the time-honoured principle of "here is one I prepared earlier", here is a sample game with a few pieces in play:



The top-level function is so trivial as to be hardly worth showing you:

```

V start
[1]  A Set up and play game
[2]  'bcg' \WC\BITMAP\Aboard
[3]  'ssf' \WC\FONT\ 'MS Sans Serif' 8
[4]  init_board
[5]  init_icons
[6]  init_game
[7]  set_events
[8]  \DQ'. '
V

Aboard+ 'c:\windows\marble'  A or some such

```

So to the playing surface, set up by *init_board* as follows:

```

V init_board; hgrd; vgrd; ttl
[1]  A Set up playing area ...
[2]  ttl+ 'Coast to Coast Footpath'
[3]  'BD' \WC\FORM\ ttl(50 20)(400 400)
      ('COORD' 'PIXEL') ('SIZEABLE' 0) ('MAXBUTTON' 0)
      ('BCOL' 0 64 64)
[4]  'BD' plinth(40 40)(320 320)4
[5]  'BD' bdr' \WC\RECT'(40 40)(320 320)
      ('FCOL' 255 255 255) ('FSTYLE' 'bcg')
[6]  'BD' msg' \WC\TEXT' ''(370 40)(0 320) ('FONT' 'ssf')
      ('FCOL' 255 255 255)
[7]  A Horizontal grid ....
[8]  hgrd+(18p1 0) c[1](2/40+32*19), [1.5] 18p40 360
[9]  'BD.g1' \WC\POLY\ hgrd('FCOL'(c0 0 0))
[10] A Vertical grid ....
[11] vgrd+(18p1 0) c[1](18p40 360), [1.5](2/40+32*19)
[12] 'BD.g3' \WC\POLY\ vgrd('FCOL'(c0 0 0))
[13] hgrd+hgrd+ "c2 2p1 0 1 0
[14] 'BD.g2' \WC\POLY\ hgrd('FCOL'(c255 255 255))
[15] vgrd+vgrd+ "c2 2p0 1 0 1
[16] 'BD.g4' \WC\POLY\ vgrd('FCOL'(c255 255 255))
V

```

The one non-obvious trick worth pointing out is the use of a partitioned *enclose* on lines [8] and [11] to make each grid line into a separate 2-by-2 array; note also the requirement to *enclose* the colour specification when drawing multiple objects in the same *\WC* call. In passing, it is worth noting just how effective this simple trick — drawing a black line one pixel above a white line — really is. Incidentally in the world as defined by Microsoft, the sun is always shining from the top left!

Now for some tiles ...

```

v init_icons;cm;bmp;map
[1]  a Make tile icons
[2]  cm+acmap  a Since default colour map is suspect:
[3]
[4]  a make_xxx requires orientation on left, colour on right.
[5]  a DATA is (rotation)(function to make one)(bits map)
[6]
[7]  bmp map+2 make_term 3
[8]  'ttmk'WC'BITMAP'  'bmp cm
[9]  'ttmk'set_data('rtn' 2)('fn' 'make_term 3')('map'map)
[10]
[11]  bmp map+3 make_term 1
[12]  'ttmr'WC'BITMAP'  'bmp cm
[13]  'ttmr'set_data('rtn' 3)('fn' 'make_term 1')('map'map)
[14]
[15]  bmp map+3 make_line 3
[16]  'ttlk'WC'BITMAP'  'bmp cm
[17]  'ttlk'set_data('rtn' 3)('fn' 'make_line 3')('map'map)
[18]
[19]  bmp map+2 make_line 1
[20]  'ttlrl'WC'BITMAP'  'bmp cm
[21]  'ttlrl'set_data('rtn' 2)('fn' 'make_line 1')('map'map)
[22]
[23]  bmp map+2 make_tee 3
[24]  'ttek'WC'BITMAP'  'bmp cm
[25]  'ttek'set_data('rtn' 2)('fn' 'make_tee 3')('map'map)
[26]
[27]  bmp map+3 make_tee 1
[28]  'tter'WC'BITMAP'  'bmp cm
[29]  'tter'set_data('rtn' 3)('fn' 'make_tee 1')('map'map)
[30]
[31]  bmp map+2 make_cross 3
[32]  'tcrk'WC'BITMAP'  'bmp cm
[33]  'tcrk'set_data('rtn' 2)('fn' 'make_cross 3')('map'map)
[34]
[35]  bmp map+3 make_cross 1
[36]  'tcrr'WC'BITMAP'  'bmp cm
[37]  'tcrr'set_data('rtn' 3)('fn' 'make_cross 1')('map'map)
[38]
[39]  bmp map+2 make_corner 3
[40]  'tcnk'WC'BITMAP'  'bmp cm
[41]  'tcnk'set_data('rtn' 2)('fn' 'make_corner 3')('map'map)
[42]
[43]  bmp map+3 make_corner 1
[44]  'tcnr'WC'BITMAP'  'bmp cm
[45]  'tcnr'set_data('rtn' 3)('fn' 'make_corner 1')('map'map)

```

v

It really doesn't matter very much which *make_* function we look at ... so let's go with *make_tee* as the most complex example:

```

▽ tm+orient make_tee clr;msk;bm;cen;rtn;bc;lc;map
[1]    A Set up new tee tile from plain bitmap
[2]    bm+init_tile Δtile ◊ msk+bm=3 ◊ bm+(msk×clr)+bm×~msk
[3]    lc+Δline ◊ tm+bm ◊ cen+bm[1+130;1+130] ◊ bc+cen[2;2]
        ◊ map+3 3p0
[4]    cen[14;15 16]+lc ◊ map[1 2;2]+1
[5]    cen[15 16;]+lc ◊ map[2;]+1
[6]    cen map+orient spin cen map
[7]    cen+shade_3D cen
[8]    A Drop it back on top of full tile ...
[9]    tm[1+130;1+130]+cen
[10]   tm+tm map
▽

▽ tile+init_tile sz
[1]    A Make a plain square tile of the right size
[2]    tile+sz szp3
[3]    A Basic 3D effect ...
[4]    tile[1;]+15 ◊ tile[;1]+15
[5]    tile[1+ptile;]+0 ◊ tile[;1+ptile]+0
▽

```

The globals *Δtile* and *Δline* just give the standard tile size (32 by 32) and footpath colour (10 = bright green works well). Note from the hefty function above that not all the tiles start in the same orientation (the West-East player naturally prefers his 'straights' to start horizontally), so we need a little utility to rotate square bitmaps ...

```

▽ new+orient spin mat;rtn
[1]    A Bitmap flipper to orient correctly ...
[2]    A See 'dance of rounds' ... Benkhard 1991
[3]    rtn+(orient+1)◊'  '◊'◊'◊'◊'◊'
[4]    ±'new+',rtn,'mat'
▽

```

... for convenience the function which makes the visible tile also records the topology in the most convenient way I could think of ... as a 3 by 3 boolean:

```

  map
0 0 0
1 1 1
0 1 0

```


... which will come in really handy when checking if the player has dropped his tile on the board so that all boundaries match! Finally, a look at the tricks needed to give that nice 3D effect:

```

      ▽ new+shade_3D cen;msk
[1]  A Now the tricky bit ... shade it 3-D ...
[2]  A Uses lc and bc from make_... etc
[3]  msk+(lc=10cen)^cen=bc ◊ cen+(7*msk)+cen*-msk      A Shadow
[4]  msk+(-10cen=lc)^cen=bc ◊ cen+(8*msk)+cen*-msk      A Highlight
[5]  msk+(10cen=lc)^cen=bc ◊ cen+(7*msk)+cen*-msk      A Shadow
[6]  msk+(-10cen=lc)^cen=bc ◊ cen+(8*msk)+cen*-msk      A Highlight
[7]  A Tidy edges (leave where line colour)
[8]  msk+Δline=cen[1 30;] ◊ cen[1 30;]+(msk*30 2p8 0)+
      cen[1 30;]*~msk
[9]  msk+Δline=cen[;1 30] ◊ cen[;1 30]+(msk*30 2p8 0)+
      cen[;1 30]*~msk
[10] new+cen
      ▽

```

This applies only to the middle of the tile (the edges being already accounted for) and uses colours 7 and 8 (the light and dark greys) rather than black and white, which are much too stark for this job. Ignore the fussy bit about the edges; the essential trick is well illustrated by lines [3] — [6] where the bitmap is shifted and compared with itself to find the boundaries. Anyway, enough of these gritty details ... now for something seriously useful

Managing Object Data

Here is a wholly unexplained line of code from *init_icons* ...

```
[9] 'ttmk'set_data('rtn' 2)('fn' 'make_term 3')('map'map)
```

This is where you need to think hard about your design, in particular you need to be very clear about what data is logically associated with particular objects. It makes obvious sense to keep all the data about an object with that object (which is where the Dyalog 'DATA' property comes in so handy), but the last thing an application should need to know is anything about the internal structure of the DATA (which can after all be any APL array).

In the above example, we want to record: how the bitmap was rotated (so that when a player asks to rotate it we know where it is starting from); what code we used to make it (so we can quickly make another one); what its topology is (for edge checking). The *set_data* utility is intended to take the pain out of managing the data property, and to insulate the application entirely from its

internals: essentially it implements a simple named component file system on each object (rather like APL2/PC's AP 211):

```

V obj set_data arg;prop;inx;dt;prp
[1]  A Set data property sub-element prop[1] to prop[2]
[2]  A Add property to table if new, else update inplace
[3]  A e.g. 'ff' set_data 'usage' (12p0)
[4]  A May be used to set multiple properties like WC ...
[5]  A e.g. ('p1' 12)('p2' 'hello')('p3' ('Hi' 1 2 3))
[6]  A
[7]  dt+obj [WG'data'
[8]  + (0<pdtd)+Exists
[9]  A Initialise a new list as (dir)(contents) ...
[10] dt+2p<0
[11] Exists:→(0 1 2 3 4=4[|=arg)/0 0 Encl Next 0
[12] A Only one pair ... so enclose it to loop round!
[13] Encl:arg+arg
[14] A Find 'item' in property list
[15] Next:prop+arg
[16] A Must be 2 elements ... so fix ('prop' 2 3 4) ...
[17] + (2=p,prop)+Fixed
[18] prop+(>prop)(1+prop)
[19] Fixed:prp+dt & inx+prp:prop[1]
[20] + (inx<p:prp)+Fnd
[21] A Add new property ...
[22] dt+dt, < "prop & →Done
[23] Fnd:((2,inx)>dt)+2>prop
[24] Done:arg+1+arg & →(p:arg)+Next
[25] A Finally replace the updated DATA ...
[26] obj [WS'data'dt
V

```

Simply to be able to associate named properties with each and any GUI object, and to retrieve and remove them by name, seems to me to be a huge step forward in design, and it fits so well with all the APL style we already know. To illustrate the point, here is the function to rotate a tile (well actually all it rotates is the bitmap) on its starting block:

```

V r+rotate_tile msg;tile;bmp;rtn;fn;new;map
[1]  A Rotate tile on starting block ...
[2]  A
[3]  tile+1>msg & bmp+tile [WG 'BITMAP'
[4]  rtn fn+bmp get_data'rtn' 'fn'
[5]  rtn+4|1+rtn
[6]  new map+1(▼rtn), ' ',fn
[7]  bmp [WS'BITS'new
[8]  bmp set_data('rtn'rtn)('map'map)
[9]  r+1
V

```

This gets fired when a player clicks on the tile, so it gets the whole event message as left argument ... it throws away everything except the tile name and uses this to find out which bitmap was stuck on it. Now we can simply ask the bitmap what its rotation was, and how it was made:

```
[4]      rtn fn+bmp get_data'rtn' 'fn'
```

... and a suitably trivial piece of APL spins it around, and then records the new information where it belongs ... back with the bitmap! Back to the game

```

      ▽ start
[1]      A Set up and play game
[2]      'bcg'⎕WC'BITMAP'Δboard
[3]      'ssf'⎕WC'FONT' 'MS Sans Serif' 8
[4]      init_board
[5]      init_icons
[6]      init_game
[7]      set_events
[8]      ⎕DQ'.'
      ▽

```

... we have drawn the board and made lots of handy bitmaps ... now to generate the starting position and let the players get going:

```

      ▽ init_game
[1]      A Set up game
[2]      A Record starting state and some useful diagnostics
[3]      'BD'set_data'Next' 1      A Next available tile
[4]      Δmap+10 10p←0      A Connections map
[5]      A Now make initial tile collection ....
[6]      A Right arg is (bitmap name) (number in pot)
[7]      56 2 make_tile'tcrk' 3
[8]      2 56 make_tile'tcrr' 3
[9]
[10]     120 2 make_tile'ttek' 5
[11]     2 120 make_tile'tter' 5
[12]
[13]     186 2 make_tile'ttmk' 1
[14]     2 186 make_tile'ttmr' 1
[15]
[16]     248 2 make_tile'ttlk' 5
[17]     2 248 make_tile'ttlr' 5
[18]
[19]     312 2 make_tile'tcnk' 10
[20]     2 312 make_tile'tcnr' 10
[21]
[22]     A Next free tile number should be 11!
[23]
[24]     'BD'set_data'ToPlay' 'WN'      A Either player!
      ▽

```

```

▽ pos make_tile arg;bmp;rm;nxt;nm
[1]  A Use given bitmap <bmp> to make tile at <pos>
[2]  A Gets next available tile name (held at board) ...
[3]  bmp rm←arg
[4]  nxt←'BD'get_data'Next' ◊ nm←'BD.t',▽nxt
[5]  nm [WC'IMAGE'pos('DRAGABLE' 1)('BITMAP'bmp)]
[6]  nm [WS('EVENT' 5 'rotate_tile')]
[7]  nm set_data'rm'rm
[8]  A Label it with no. of tiles remaining in set <rm> ...
[9]  pos←pos+('r'='1+bm')$36 0
[10] ('BD.1',▽nxt)[WC'TEXT'('rm')pos
      ('FCOL' 255 255 255)('FONT' 'ssf')]
[11] A Click tile counter on 1 ...
[12] 'BD'set_data'Next'(nxt+1)
▽

```

Surprisingly little data is relevant at the board level ... it is a handy place to record the next free tile name, and who is next to play. This is recovered from the objects by `get_data` as follows:

```

▽ prp←obj get_data prop;inx;dt;prp
[1]  A Get data property sub-element prop[1] from obj
[2]  A e.g. 'ff' get_data 'prop1'
[3]  A or 'ff' get_data 'p1' 'p2' .... 'pn'
[4]  prp←dt←obj [WG'data' ◊ →(p,prop)+0 ◊ →(0←pdt)+0
[5]  A Check if only one item, and enclose if so ...
[6]  ±(2>|=prop)/'prop'←prop
[7]  prp←dt ◊ inx←prp|prop
[8]  A Weed out any mishits ...
[9]  inx←(inx<prp)/inx ◊ prp+' ' ◊ →(pinx)+0
[10] prp←(2,inx)←dt
[11] A Strip outer case if only one element ...
[12] ±(1=prp)/'prp'←prp
▽

```

When I get around to finishing the program, I will probably store `Δmap` with the board as well — but while I am trying to work out nifty ways of detecting winning lines (and maybe of getting the computer to play the game) it is much handier to leave this in the workspace as a global. The only obvious tile property is the number of siblings remaining in the pot; this is used when a player 'clones' a tile by dragging it and letting go:

```

▽ pos clone_tile bmp;txt;nxt;nm;bits;cm
[1]  A Repeat tile using bitmap <bmp> on board at <pos>
[2]  A Uses next available tile name (at board level) ...
[3]  nxt←'BD'get_data'Next' ◊ nm←'BD.t',▽nxt
[4]  A Make a new bitmap, in case the parent is rotated ...
[5]  bits cm←bmp [WG'BITS' 'CMAP'
[6]  bmp←nm,'x'

```

```

[7]  bmp [WC'BITMAP' ''bits cm
[8]  nm [WC'IMAGE'pos('BITMAP'bmp)
[9]  A Click tile counter on 1 ...
[10] 'BD'set_data'Next'(nxt+1)

```

▽

... and also to update the label (and maybe kill the tile altogether) when a new instance is made:

```

▽ r←place_tile msg;tile;lbl;rc;pos;bmp;rm;nxt;map
[1]  A Drop tile on board (snapped to grid) ...
[2]  A Reject drop if tile is dropped illegally
[3]  A Otherwise make a new tile at the correct spot
[4]  A and reject the event anyway!
[5]  r←0 + tile+3>msg + rc+16+msg[4 5]
[6]  A Snap to nearest square and check it ...
[7]  rc←41+32×[32+~rc-40
[8]  +(v/(rc<40)v(rc>40+10×32))+0
[9]  pos←c1+[32+~rc-41 + (2=pppos→Δmap)+Inuse
[10] A Position is Ok ... check legal neighbours ...
[11] bmp←tile [WC'BITMAP'
[12] A Are we trying to play out of turn?
[13] nxt←('Kr'['HW'BD'get_data'ToPlay']
[14] →((-1+bmp)←nxt)+OutofTurn
[15] map←bmp get_data'map'
[16] A map can be checked for edge fit with adjacent maps
[17] →(chk_fit map pos)→Illegal
[18] A ===== all OK =====
[19] rm←tile get_data'rm'
[20] tile set_data'rm'(rm-1)
[21] A Redo label (and possibly kill tile if none left) ...
[22] label_tile tile
[23] A Update diagnostic data
[24] (pos→Δmap)←map
[25] A Copy parent tile to right place on board ...
[26] rc clone_tile bmp
[27] nxt←('North' 'West')[1+'r'=-1+bmp]
[28] 'BD'set_data'ToPlay'(>nxt)
[29] log'OK ... ',nxt,' is next to play'
[30] →0
[31] A ===== Errors =====
[32] Inuse:log'Silly billy' +0
[33] Illegal:log'Piece must match on all boundaries' +0
[34] OutofTurn:log'It's not your turn!!' +0

```

▽

```

▽ label_tile tile;rm
[1]  A Update tile label with number remaining
[2]  rm←tile get_data'rm'
[3]  lbl←tile + lbl[4]+1 + lbl [WS'text'(vrm)
[4]  A Count remaining tiles; kill parent if none ...
[5]  →(rm>0)+0 + [EX"tile lbl

```

▽

So far, I have never needed to remove data from an object, but just for completeness:

```

∇ obj rmv_data prop;inx;dt;prp;msk
[1]  A Remove named data property from obj
[2]  A e.g. 'ff' rmv_data 'usage'
[3]  A May be used to kill multiple properties ...
[4]  A e.g. 'ff' rmv_data 'p1' 'p2'
[5]  A
[6]  dt←obj □WG'data' ∘ →(0≤pdt)+0
[7]  ±(2>|=prop)/'prop'←prop
[8]  A Find 'items' in property list
[9]  prp←pdt ∘ inx+prp\prop
[10] inx←(inx$prp)/inx
[11] A Kill em from catalogue and value list ....
[12] msk←~(1p prp)∈inx
[13] dt←(msk/prp)(msk/2>dt)
[14] obj □WS'data'dt
∇

```

All that is needed to run the finished game (in human vs human) mode is a couple of extra events and a □DQ ...

```

∇ set_events
[1]  A Set up event table
[2]  'BD'□WS'EVENT' 11 'place_tile'
[3]  'BD.msg'□WS'EVENT' 1001 'show_msg'
∇

```

... the strategy for displaying messages is quite novel:

```

∇ log txt
[1]  A Send message to message area to get txt displayed
[2]  □NQ'BD.msg' 1001 txt
∇

∇ show_msg msg
[1]  'BD.msg'□WS'text'(3>msg)
∇

```

Rather than allowing functions to update the message text directly, I have defined 1001 as a 'message' event and sent it to the 'message object' along with the appropriate text. Comments please — this is 'getting a little too 'object-oriented' for my taste.

Tracking Events

By now, the astute reader will have noticed that the code totally fails to achieve the other objective I set myself ... making visible the relationship between objects, events and the blocks of code which they fire. As always, I wrote the workspace first and only later did I realise the problems I was setting for myself in debugging and maintaining it. What I think I need is something like this:

DISP Δevent

BD	11	<i>place_tile msg</i>	<i>Basic drag-drop of tile on board</i>	BD	INDEX ERROR
BD.t*	5	<i>rotate_tile >msg</i>	<i>Double-click spins tile on base</i>	BD.t23	VALUE ERROR
BD.msg	1001	<i>show_msg >msg</i>	<i>Our own message to show caption</i>		

... and a little function to maintain it (why should I have to remember that event 11 is a 'drag-drop' ?)...

```

V obj set_event action;pos;eno
[1]  A Update event table Δevent with new action
[2]  A Monadic call initialises table ...
[3]  A (obj)(eno)(action)(comment)(who)(what happened)
[4]  A Null right arg kills all events for that object
[5]  A Null action kills specific event only.
[6]  A
[7]  →(0=NC'obj')+Update ◊ Δevent+0 6p0 ◊ +0
[8]  A Map text event mnemonics to dyalog event numbers
[9]  Update:→(paction)+Set
[10] pos←Δevent[;1]◊obj
[12] Set:⋆(2>|=action)/'action+',c'caption'
[13] eno+5 11 1001 9999['dclk' 'drdp' 'msg'\action[1]]
[14] pos←(Δevent[;2]=eno)^(Δevent[;1]◊obj)/⋆pΔevent
[15] →(eno=9999)+0 ◊ →(1=paction)+Kill
[16] →(ppos)+Replace
[17] Δevent;←c0 ◊ pos←pΔevent
[18] Replace:Δevent[pos;]+obj eno,(2+1+action),2p0 ◊ +0
V

```

```
'BD' set_event 'drdp' 'place_tile msg' 'etc'
```

... obviously to do this for real you need a global table of event names and internal codes to replace the subset required for line[13]. I have been wondering about inheritance (should the function that searches this table check the parent if it fails to find an event defined on a given object?) and provisionally come to the view that wild-cards are more useful — and a lot less dangerous! Accordingly:

```

V r←list Δwcm tgt;hits
[1]  A Wild Card Match of <tgt> to word list
[2]  A '?' is don't care about this char.
[3]  A '*' is forget anything to the right of me.
[4]  A
[5]  A e.g. 'fred' 'ted*' 'bill?' wcm "" 'teddy' 'billy'
[6]  A matches 0 1 0 0 0 1
[7]  tgt←tgt, 'I' A ensure no false hits on leading part!
[8]  list←(ptgt)↑"list," 'I'
[9]  hits←(list='?')∨list=ctgt
[10] r←^/"(1+list:"*")+hits
V

```

and to manage the whole complex web of objects and events, I think that all I need is a 'standard' event which fires on all interesting events on any object:

```

V Δfire msg;tgt;ex;result;⌈TRAP
[1]  A Field all tracked events and fire associated
[2]  A code from global event table Δevent.
[3]  A N.B. wild card matches on objects are allowed.
[4]  A Updates event table to log who fired it, and when.
[5]  tgt←i→pΔevent
[6]  tgt←(Δevent[;2]=msg[2])/tgt ∘ +(ptgt)+0
[7]  A We are interested, so check the object ...
[8]  tgt←(Δevent[;1]Δwcm>msg)/tgt ∘ +(ptgt)+0
[9]  A If more than one hit, just fire the higher ...
[10] tgt→tgt
[11] A Extract code and run it ...
[12] ex←tgt>Δevent[;3]
[13] ⌈TRAP←0 'C' i→Event_Error'
[14] result←'ok' ∘ Δex ∘ →Log
[15] Event_Error:⌈TRAP←0p⌈TRAP
[16] result→⌈DM
[17] A Log success or trapped failure in event table
[18] Δevent[tgt;5 6]←(→msg)result
V

```

In this rather simple case, with only three interesting classes of object, there is no real problem, but I get the strong feeling that I am going to need something like the above very badly indeed when I start to work on anything at all heavyweight. The advantages are fairly obvious:

- because all events are sent through the same funnel, it is very easy indeed to build in a 'debug' mode which tracks 'who fired what' in a little window out to one side.
- you can attach any executable APL expression to an event, not just a single function which gets force-fed the event message as one argument!
- this should be the only place you need an error trap, and again it can be a lot more sophisticated, for example it might dump a lot of useful diagnostics to file when a triggered event fails.

- the bane of the product-support technician is the un-reproducible bug — and under Windows the situation is a hundred times worse than in an old-style menu-driven system. At least with a global event log, he can ask to see exactly what events were triggered, and get some clue what the results were!

All of which had better be left for the next Vector!

Appendix — Just for the Record

```

V fm plinth arg;rc;hw;lw;off;ct;crd
[1]  A Make a plinth around rectangle defined by arg
[2]  A ... white top left, dark lower right
[3]  rc hw lw+arg 0 ct+1
[4]  Loop:→(ct>lw)+Tl
[5]  off+ct
[6]  crd+3 2p(rc+(hw[1]+off),-off),(rc-off),
    (rc+(-off),hw[2]+off)
[7]  (fm,'.pl1',vct)[]WC'POLY'crd('FCOL' 192 192 192)
[8]  crd+3 2p(rc+(hw[1]+off-1),-off),(rc+hw+off-1),
    (rc+(-off),hw[2]+off-1)
[9]  (fm,'.pl2',vct)[]WC'POLY'crd('FCOL' 128 128 128)
[10] ct+1 0 →Loop
[11] A Mitre top corner nicely ...
[12] Tl:(fm,'.pl3')[]WC'POLY'(2 2prc,rc-ct)('FCOL' 255 255 255)
[13] (fm,'.pl4')[]WC'POLY'(2 2p(rc+hw),rc+hw+ct-1)
    ('FCOL' 192 192 192)
V

V ok+chk_fit arg;map;pos;nesw;fit;chk
[1]  A Check for fit with neighbours.
[2]  A <map> is the square to be verified, placed at <pos>
[3]  map pos+arg 0 ok+1
[4]  nesw+(-1 0)(0 1)(1 0)(0 -1)+pos
[5]  fit+((1 2)(3 2))((2 3)(2 1))((3 2)(1 2))((2 1)(2 3))
[6]  chk+(/"nesw>0)^"/"nesw<11
[7]  nesw+chk/nesw 0 fit+chk/fit
[8]  chk+2=0 0 Δmap[nesw] 0 nesw+chk/nesw 0 fit+chk/fit
    0 →(pfit)+0
[9]  A now check each adjacent map from Δmap
[10] chk+Δmap[nesw]
[11] ok+^/fit mat_edge"chk
V

V ok+fit mat_edge chk
[1]  A check each required edge (uses map from calling fn)
[2]  ok+(fit[1]>map)=fit[2]>chk
V

```

If that isn't all the code, I'm sorry. As usual with APL-385 experiments, the workspace is 'free to good home', but please wait until November for the finished version. If anyone has any bright ideas about a computable strategy for actually playing the damn' game, I should be delighted to hear from them.

ROWFIND

for APL2 (Mainframe)

Do you have performance problems with your search routines in APL2 mainframe? No? Are you sure? Look at these figures:

	CPU time in ms			
	l=A	l=A	l=A	
	r=B	r=A	r=C	
x ← (⌈ [2] 1) ⌈ (⌈ [2] r)	35	337	6	
x ← 1 Δ_MMI r	150	1880	1.5	from APE2 sort method Interprocess
x ← 1 GRADETIOTA r	33	66	27	
x ← 1 ILOOKUP r	19	224	0.4	
x ← 1 ROWFIND r	2.4	5	0.8	

Arguments

A - character matrix 1155 17	rows sorted
B - character matrix 100 17	random rows from A
C - character vector 17	random row from A
(or matrix 1 17)	

Note that ROWFIND is very good for almost ALL sizes of the arguments.

This routine alone saves £ 50,000 a year in one big APL2 system at Volvo Car Corporation. Can YOU afford not to use it?

ROWFIND for APL2 mainframe is very similar to ROWFIND in APL*PLUS II/PC. ROWFIND from CYBEX comes with a general package of utility functions. The price is £ 1,900.



CYBEX AB

Gruvgatan 35B, 421 30 V Frölunda, Sweden
Phone +46 31453740, Fax +46 31452423

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Technical Correspondence		114
A Standard Font Dialogue Box from Dyalog APL/W using □WA	Duncan Pearson	117
The MAGIC goes away - Opening the Boxes	Mike Day	125
The Fractal MAGIC Universe	Gérard Langlet	137

TECHNICAL CORRESPONDENCE

A Windows GOTCHER for APLIWIN

From: Anthony Camacho

15 May 1993

I have been using APLIWIN to provide a customer with a PC version of a system originally written in Sharp APL in 1977. I have been very impressed with APLIWIN even though my stupidity and the lack of a system manual for this specific implementation has caused me some head-banging (for example it took me ages before I found out how to fill a combo box with 20 lines of alternatives).

This letter is being written in WinWord. I have been using WinWord to make notes on the conversion for colleagues and have been delighted to be able to cut functions from the workspace and paste them into WinWord (or Write) and then by selecting the pasted section and changing font to ISIAPL's TrueType font the correct APL characters are both displayed and printed. Marvellous.

The other evening I had some work to do which required a PostScript font. I set WinWord to use the PostScript printer, did the job (printing to file and using GoScript to emulate the printer) and thought no more about it. I little realised what I had done.

The next time I did a cut and paste from ISIAPL to WinWord the characters were displayed in some default font or other. The ISIAPL font was there in the font list and could be applied to selected text but it appeared not to work: the text obstinately refused to turn into APL.

So of course I tried all the obvious things; I deleted and re-installed the font; I did something which I had intended to do ever since I mistakenly allowed CorelDraw to install hundreds of rarely needed fonts - I deleted about a hundred fonts; I edited WIN.INI to remove the section which specified font substitutions; I rang the Microsoft help line (a mistake - see below); I even bought a book Windows 3.1 Insider (£24.00!) which had a large section on fonts.

The Microsoft help line was worse than useless. An authoritative man told me to do this and that and the other and, like a fool, I did what he said. When we had got as far as to 'remove' all fonts and then 'add' all fonts I realised that my adviser knew less than I did. The effort of selecting a hundred fonts to delete and waiting for the Windows longwinded deletion process was undone. Then he said he was going to delete and re-install all my drivers and I said that plainly he was unable to see what was wrong and that I might as well re-install windows from scratch as it would take less time than his diagnosis.

The GOTCHER is the shibboleth WYSIWYG [What You See Is What You Get]. Microsoft have programmed WinWord and Write so that if a printer is selected that cannot print TrueType the display will not show TrueType. They could easily have indicated that the TrueType fonts were not available or disabled by removing them from the list of fonts or by some warning or other. But they didn't; you can select from all the fonts and the substitution is automatic and outside your control and you are not told it is being done to you.

The Insider book (and, to be fair, the Microsoft help line) told me that the printer driver determined what could be displayed. The Microsoft help line did not tell me to check what printer was selected. They didn't realise the significance of this. They didn't suggest I tried selecting other printers. All I had to do to recover the ability to cut and paste APL was to select the DeskJet or Epson or any printer that can do TrueType.

Try it! Put something on the screen in an out-of-the-way font. Set up a PostScript printer (you don't need to have one) or a plotter or daisywheel and select it; the screen miraculously changes its display to the nearest default font. Select a dot matrix printer that can do TrueType and the screen changes back to show the font as you expected.

I have written to Microsoft saying that I think they have made a mistake. If my text will not print as I intend, please warn me but don't in the name of WYSIWYG convert my intentions to something else. And don't call it a printer driver if it is really a display and printer driver. Write the message loud and clear **WHAT IS SHOWN ON THE SCREEN IS LIMITED TO WHAT YOUR PRINTER CAN PRINT.**

Apart from the waste of effort it does make me feel so stupid.

Anthony Camacho

Errors in "Integrating with Insight"

From: John R. Clark

June 1993

The article "Integrating with Insight" by Norman Thomson, pages 113 to 116 of Vector Vol.9 No.2 Jan 1993, has some typographical errors and one logical error.

Logical Error:

Page 115 line [2] of *METHOD* should end with $\lceil L+Q \times \lceil L \div Q$. For Simpson the number of subdivisions must be even.

Typographical errors:

Page 113. $1/4096$ should be $1/4095$

Page 115. Line [4] of *ROMBERG*. *EPSL* is an undefined variable.

Page 115. Line [3] of *ADAPT*. *EPSn* is an undefined variable.

I am not sure what the undefined variables should be. If the L of *ESPL* is $\leq$ then *ROMBERG* works. If the n of *EPSn* is $\geq$ then *ADAPT* works. Perhaps you have a problem with your typesetting software. I believe you need magnitude in both tests.

I receive my copy of Vector through membership in Los Angeles SOCAL SIGAPL and Roy Sykes of Sykes Systems — I do enjoy your publication.

As a past algorithm editor of QuoteQuad I am aware errors do creep into publications.

Keep up the good work

John R Clark
P.O. Box 821
Lowell Ar 72745
USA

*Ed: I created a translate table to convert Norman Thomson's original text from APL2 □AV into APL*PLUS/PC □AV, which is used for typesetting. Unfortunately I omitted $\leq$ and $\geq$ from the translate table, hence the EPSL and EPSn.*

A Standard Font Dialogue Box from Dyalog APL/W using `□NA`

by Duncan Pearson

The following is a description of the use from Dyalog APL/W of the common dialogue box for choosing fonts within Windows 3.1. Dyalog APL/W does not provide a means of doing this within its GUI interface, but with `□NA` it is possible to call any function in a dynamic link library. The sets of functions which drive Windows are all located in DLLs and so with access to the Software Developer's Kit manuals, or their equivalent, we can use parts of the Windows GUI that Dyadic did not include in Dyalog APL, perhaps for reasons of compatibility with Motif under UNIX.

The process of using a standard font dialogue box is in five parts:

- 1) Get a block of memory in which to put the current font definition. We use the `GlobalAlloc` function to get the memory and the `GlobalLock` function to lock it into place and tell us where it is.
- 2) Put the current font data into the block using the `hmemcpy` function that copies data from one location to another.
- 3) Pass the address of the block to the `ChooseFont` function as part of a complicated data structure.
- 4) Read the changed data from the block into APL using the same `hmemcpy` function that we used to copy it into the block.
- 5) Free the memory for use by other applications, using the `GlobalFree` function which takes the identifier (returned by `GlobalAlloc`) and returns the memory to the pool.

All the functions used are in the `KERNEL.EXE` dynamic link library except `ChooseFont` which is in `COMMDLG.DLL`. Both of these DLLs are part of Windows.

The font data structure, in which the font specification is passed to and from the `ChooseFont` function, is listed in the appendix but it is basically five integers (two bytes each) followed by eight single byte values followed by a null terminated string. The first and last of the integers are the size and weight of the font, the first three of the byte values, if they are non-zero, indicate bold, underline and strikeout and the null terminated string is the font name.

The way that we use these C functions is to 'associate' them with function names in the workspace. We use the `⌈NA` function which associates a name in the APL workspace (the left argument) with a function in a DLL, specified in the right argument. That name becomes of class 3 (a function) and whenever we use that name in an APL expression the function in the DLL is called.

The way that the arguments to the APL function are translated into the arguments of the DLL function and that the result of the DLL function is translated into an APL object is determined by the first and last parts of the right argument to `⌈NA`.

Memory Allocation

For example:

```
'alloc'⌈NA'U2 kernel.exe.P16|GlobalAlloc U2 U4'
```

associates the name 'alloc' in the workspace with the DLL function `GlobalAlloc`. If we call `alloc` with two integer arguments

```
handle←alloc 0 50
```

the first will be converted into a two-byte unsigned number (the memory type) and the second into a four-byte unsigned number (how many bytes we want). The two-byte result from `GlobalAlloc` will be converted back into a positive integer.

The effect of the function is to reserve 50 contiguous bytes of global memory for our use, the returned result being an identifier for that memory. We need to fix the location of that memory so that we can write data into it. `GlobalLock` is the Windows function that fixes memory and it returns the address at which it has fixed it. We pass the two-byte positive integer identifier and get back a four-byte integer pointer.

```
'lock'⌈NA'I4 kernel.exe.P16|GlobalLock U2'  
ptr←lock handle
```

Copying the Font Data

To read from and write to our little block we use a function called `hmemcpy`. It is a very simple function but it is very versatile. It takes two pointers, a source and a destination, and a number of bytes to copy. It copies the specified number of bytes from the source to the destination. The versatility comes in the way we associate the function.

Here we will copy the integers to the memory area.

```
'putints'␣NA 'kernel.exe.P16|hmemcpy I4 <I2[] I4'
putints ptr(-36 0 0 0 400)10
```

The arguments to the function are, from left to right, the destination pointer, the source pointer and the number of bytes to copy. In the first association the source is defined as an input array of two byte integers. On execution the interpreter will take the integer vector passed, create a C array of two-byte integers and pass a pointer to that array to the DLL function. The third argument should be twice the length of the vector, as that is the actual number of bytes. The negative number given as the size is a value in points.

Then we copy the byte values:

```
'putbytes'␣NA 'kernel.exe.P16|hmemcpy I4 <U1[] I4'
putbytes(ptr+10)(1 1 0 0 0 0 0 0) 8
```

Here we are saying that the second element of the argument, the source data, should be translated as an array of bytes. In APL parlance they are indices into $\square AV$. Notice that when we call the function we pass not the address of the memory but the address of the eleventh byte of the memory, where we want it to put the byte values.

Lastly we insert the name of the font:

```
'putstring'␣NA 'kernel.exe.P16|hmemcpy I4 <OT I4'
putstring(ptr+18)'Times New Roman' 32
```

The interesting feature of this association is the $<OT$ which specifies a translated null terminated string. When the function is called we pass a text vector which is translated from the $\square AV$ to the normal ASCII character set and copied at the pointer. After the last character of the string the interpreter places a 0. This is included in the number of bytes to copy so our name must be no more than 31 characters long.

Running the ChooseFont function

We are now ready to call the ChooseFont function, well, nearly ready. All we need to do is associate the function and run it. Unfortunately, while the function only takes one argument, it is a pointer to a horrendous C data structure, one part of which is the pointer to our font data. However, if we define the structure when we associate the function, we can pass a nested array corresponding to it. As with the simple arrays, the interpreter will pass a pointer to it. The official

structure definition is part of the appendix but the line below defines it well enough for the association.

```
choose_struct+ '{I4 I2 I2 I4 I2 I4 U1[4] I4 I4 I4 I2 I4 U2 I2 I2}'
'changeFont' \NA'U commdlg.dll.P16|ChooseFont =',choose_struct
```

Things to note are that the U1[4] is an array of red, green and blue intensities for the display, the second I4 is the pointer to our font data and the third I4 is the combination of some Boolean values to tell the function what fonts to allow and whether there is meaningful data in the font structure. The "=" before the font structure in the association is an indication to the interpreter that when the function has run we would like to see what it has done to the data in the structure. We are most interested here in the colour part.

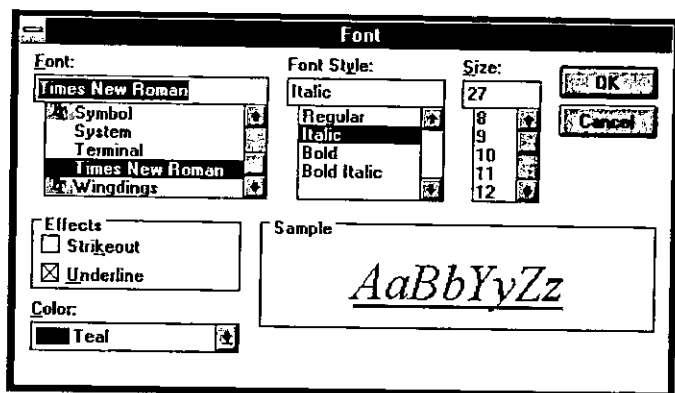
The definition of the individual bits is included in the appendix as a set of "#define CF_*** 0x00***" statements that are pretty self-explanatory. We are going to use CF_SCREENFONTS, CF_INITTOLOGFONTSTRUCT and CF_EFFECTS, the last of these to enable the user to choose underline and colour.

```
flags+211 0 1 0 0 0 0 0 1
```

The first element of the structure is its length in bytes and everything else in it that I haven't mentioned can be set to 0. Thus the function call looks like this.

```
changeFont<15 46 0 0 ptr 0 flags(0 128 128 0)
```

We get a dialogue box that looks like this:



and the result:

```
1 46 0 0 137822208 360 321 128 0 128 0 0 0 0 0 8452 0 0
```

Because we specified one of the arguments as "=" the data for that argument is returned with the result of the function in a nested array, the first element being the result of the function and the second, the contents of the structure. The result of the function is non-zero, indicating that the user pressed OK, and the seventh element of the structure is 128 0 128 0, indicating that he changed the colour from 'teal' to 'purple' (takes all sorts to make a world). We can use this indication to set the colour of any text associated with this font.

Retrieving the Font Data

We now need to query the font structure to find what font attributes the user has changed. To get data out of our block of memory we use the same function that we used to get data in. This time we specify an array as the destination (first argument) and the pointer to the memory block as the source (second argument).

```
'getints' NA 'kernel.exe.P16|hmemcpy' >I2[] I4 I4'
'getbytes' NA 'kernel.exe.P16|hmemcpy' >U1[] I4 I4'
'getstring' NA 'kernel.exe.P16|hmemcpy' >OT I4 I4'
```

Instead of <I2[] (corresponding to an integer vector argument) which means

- take a vector
- turn it into integers
- pass its address to the function
- then forget it

we have >I2[] (corresponding to an integer argument) which means

- allocate space for the specified number of integers
- pass its address to the function
- convert what the function leaves there into an int. vector, 2 bytes per number
- return the vector

What we pass as this argument is the expected length of the result.

```
getints 5 ptr 10
-48 0 0 0 700
```

In the case of the integers we expected five integers (first argument) which meant that it had to copy ten bytes (third argument). The first integer is the new point size and the fifth is the weight.

We then get the attributes. We can only set the underline and italic but we might want to know if the user has set strikeout in order to warn him that it will not be implemented.

```
getbytes 3(ptr+10)3
0 0 1
```

Finally we get the name:

```
getstring 32(ptr+18)32
Courier New
```

Cleaning Up

It is good practice to give the memory back to Windows even if it is only 50 bytes, and for this we use the `GlobalFree` function.

```
'free'NA'U2 kernel.exe.F16|GlobalFree U2'
free handle
0
```

As you can see it takes a memory identifier and returns 0 if it managed to free the memory. If it had failed it would have returned the handle.

So that was easy wasn't it? You would be surprised what you can do with the DLL functions. You can be serious and write a low-level interface to Oracle using the functions in the `ORA6WIN.DLL` supplied with EXCEL or you can be flippant and start writing raw graphics all over other applications' windows, whatever flips your switch.

Font Sizes

A short note about font sizes. You pass a point size to the font structure and get a point size out. Font sizes in Dyalog APL/W are specified in pixels. There are 72 points to the inch and you can calculate the number of pixels to the inch from the `DEVcaps` property of the root object. This is unfortunately not all. For some reason, best known to the morons at Microsoft, when you specify 36 point in the font structure it shows the font in 36 point (according to the above definition) but says that it is in 27 point. Likewise, if the user chooses 36 point in the box, the number in the font structure is -48 and the user sees 48 point. Don't ask me to explain it.

Appendix

The full APL session transcript:

```
'alloc'⎕NA'U2 kernel.exe.P16|GlobalAlloc U2 U4'
handle←alloc 0 50
'lock'⎕NA'I4 kernel.exe.P16|GlobalLock U2'
ptr←lock handle
'putints'⎕NA 'kernel.exe.P16|hmemcpy I4 <I2[] I4'
putints ptr(⌊36 0 0 0 400)10
'putbytes'⎕NA 'kernel.exe.P16|hmemcpy I4 <U1[] I4'
putbytes(ptr+10)(1 1 0 0 0 0 0) 8
'putstring'⎕NA 'kernel.exe.P16|hmemcpy I4 <OT I4'
putstring(ptr+18)'Times New Roman' 32
choose_struct+{(I4 I2 I2 I4 I2 I4 U1[4] I4 I4 I4 I2 I4 U2 I2 I2)'}
'changeFont'⎕NA'U cmdldlg.dll.P16|ChooseFont =' ,choose_struct
flags+211 0 1 0 0 0 0 0 1
changeFont←15 46 0 0 ptr 0 flags(0 128 128 0)
1 46 0 0 137822208 360 321 128 0 128 0 0 0 0 0 8452 0 0
'getints'⎕NA 'kernel.exe.P16|hmemcpy >I2[] I4 I4'
'getbytes'⎕NA 'kernel.exe.P16|hmemcpy >U1[] I4 I4'
'getString'⎕NA 'kernel.exe.P16|hmemcpy >OT I4 I4'
getints 5 ptr 10
-48 0 0 0 700
getbytes 3(ptr+10)3
0 0 1
getString 32(ptr+18)32
Courier New
'free'⎕NA'U2 kernel.exe.P16|GlobalFree U2'
free handle
0
```

The font data structure — from WINDOWS.H

```
/* Logical Font */
#define LF_FACESIZE32
typedef struct tagLOGFONT
{
    int         lfHeight;
    int         lfWidth;
    int         lfEscapement;
    int         lfOrientation;
    int         lfWeight;
    BYTE        lfItalic;
    BYTE        lfUnderline;
    BYTE        lfStrikeOut;
    BYTE        lfCharSet;
    BYTE        lfOutPrecision;
    BYTE        lfClipPrecision;
    BYTE        lfQuality;
    BYTE        lfPitchAndFamily;
    char        lfFaceName[LF_FACESIZE];
} LOGFONT;
```

The data structure to pass to the ChooseFont function — from COMMDLG.H

```
typedef struct tagCHOOSEFONT
{
    DWORD      lStructSize;          /* */
    HWND        hwndOwner;           /* caller's window handle */
    HDC          hdc;                 /* printer DC/IC or NULL */
    LOGFONT FAR* lpLogFont;          /* ptr. to a LOGFONT struct */
    int          iPointSize;         /* 10 * size in points of selected
font */
    DWORD      Flags;                 /* enum. type flags */
    COLORREF    rgbColors;           /* returned text color */
    LPARAM      lCustData;           /* data passed to hook fn. */
    UINT        (CALLBACK* lpfnHook)(HWND, UINT, WPARAM, LPARAM); /* ptr. to hook function */
    LPCSTR      lpTemplateName;     /* custom template name */
    HINSTANCE    hInstance;         /* instance handle of .EXE that
    * contains cust. dlg. template
    */
    LPSTR        lpzStyle;           /* return the style field here
    * must be LF_FACESIZE or bigger */
    UINT         nFontType;          /* same value reported to the
EnumFonts
    * call back with the extra
FONTTYPE_
    * bits added */
    int          nSizeMin;           /* minimum pt size allowed & */
    int          nSizeMax;          /* max pt size allowed if */
    /* CF_LIMITSIZE is used */
} CHOOSEFONT;
```

The settings for the flags in the CHOOSEFONT structure - from COMMDLG.H

```
#define CF_SCREENFONTS          0x00000001
#define CF_PRINTERFONTS        0x00000002
#define CF_BOTH                  (CF_SCREENFONTS | CF_PRINTERFONTS)
#define CF_SHOWHELP              0x00000004L
#define CF_ENABLEHOOK            0x00000008L
#define CF_ENABLETEMPLATE        0x00000010L
#define CF_ENABLETEMPLATEHANDLE  0x00000020L
#define CF_INITTOLOGFONTSTRUCT  0x00000040L
#define CF_USESTYLE              0x00000080L
#define CF_EFFECTS               0x00000100L
#define CF_APPLY                 0x00000200L
#define CF_ANSIONLY              0x00000400L
#define CF_NOVECTORFONTS         0x00000800L
#define CF_NOOEMFONTS            CF_NOVECTORFONTS
#define CF_NOSIMULATIONS         0x00001000L
#define CF_LIMITSIZE             0x00002000L
#define CF_FIXEDPITCHONLY        0x00004000L
#define CF_WYSIWYG               0x00008000L /* must also have
CF_SCREENFONTS
    * & CF_PRINTERFONTS */
#define CF_FORCEFONTEXIST        0x00010000L
#define CF_SCALABLEONLY          0x00020000L
#define CF_TTONLY                0x00040000L
#define CF_NOFACESEL             0x00080000L
#define CF_NOSTYLESEL            0x00100000L
#define CF NOSIZESEL             0x00200000L
```

The MAGIC goes away - Opening the Boxes

by Mike Day

Introduction

A year ago Bill McLean brought the game of MAGIC to the pages of Education Vector, Vector Vol.9, No.1 (pp.25-29). In the next issue, 9.2, Dave Ziemann discussed the game, and presented a method for solution and a set of APL*PLUS II functions to do the job. It was a quite brilliant treatment, but as Dave said, "I am left with the feeling that there's probably a very simple way of knowing what key to press from any given state, but it eludes me for the moment." Here is one way of doing just that. I want to share it with you because it throws light on some aspects of A-level-ish Mathematics. The APL is quite neat as well.

In case you don't remember, in MAGIC you are given a starting 3×3 matrix or "box" of random 0s and 1s. You are asked to pick one or more of nine given standard boxes — I'll call them *MagicBoxes* — to complement the starting box to achieve the target box, which we call *win*.

Bill's article described a program to interact with the player, presenting the random starting box and showing the current position. Dave responded to his invitation to readers to provide solution algorithms.

Like Dave Ziemann, I've developed my solution mostly in terms of direct definition, but using first generation APL*PLUS version 8. I've also developed an analogous solution in J(4.1) rather to my surprise.

Representation

As in Dave's treatment, I take a global zero index origin. I don't want to rewrite all those parts of Dave's article that apply here as well. He showed that it's useful to be able to enumerate the 512 different possible states in a natural way as either decimal numbers 0 to 511 or as binary numbers 0 to 11111111 using some simple functions.

I've cribbed his functions *DB* and *BD* :

```
DB:1=(9p2)T w
BD:2I w

BD  0 0 1 1 1 1 0 1 1
123

DB  123
0 0 1 1 1 1 0 1 1
```

The functions *SHOWBool* and *SHOWDec* show states as 3×3 boxes:

```
sym ← '01' ⋄ blk ← ' '

SHOWBool: 3 0 {blk:0**/p w :sym[3 3 p w ],blk,SHOWBool 0 1 + w
SHOWDec:SHOWBool DB, w

SHOWBool 9 1 p 0 0 1 1 1 1 0 1 1 A FN expects a matrix argument
001
111
011

SHOWDec 123
001
111
011
```

But I'm more interested in the method of solution than in presentation.

Why use McLean's MagicBoxes?

The specified set of building blocks or *MagicBoxes* is

```
SHOWBool MagicBoxes
110 111 011 100 010 001 000 000 000
110 000 011 100 111 001 110 000 011
000 000 000 100 010 001 110 111 011
```

Dave Ziemann presents a "short MAGIC game". Firstly, the function *SetVars* sets the scene, including choosing which boxes to play with — in this case *MagicBoxes*.


```
SetVars: key+BD+boxes+w ◊ sym+'01',blk+' '◊ win+495
        ◊ w+0+□EX'BaseToKey'
```

```
SetVars 'MagicBoxes' ◊ 32 SHOWPATH win
```

```
0    5    6
```

```
110 001 000
```

```
110 001 110
```

```
000 001 110
```

```
000 110 111 111
```

```
100 010 011 101
```

```
000 000 001 111
```

BaseBoxes

But suppose we had a different set of boxes; one I call *BaseBoxes*:

```
SHOWBool BaseBoxes
```

```
100 010 001 000 000 000 000 000 000
```

```
000 000 000 100 010 001 000 000 000
```

```
000 000 000 000 000 000 100 010 001
```

We can easily see that the same problem can be solved — in more steps in this case:

```
SetVars 'BaseBoxes' ◊ 32 SHOWPATH win
```

```
0    1    2    5    6    7    8
```

```
100 010 001 000 000 000 000
```

```
000 000 000 001 000 000 000
```

```
000 000 000 000 100 010 001
```

```
000 100 110 111 111 111 111 111
```

```
100 100 100 100 101 101 101 101
```

```
000 000 000 000 000 100 110 111
```

It is pretty obvious with this set of building blocks that we never need use more than nine moves. You can probably see quite easily that there is only one starting state for which nine is the minimum number of boxes, namely the complement of *win*,

```
SetVars 'BaseBoxes' o 16 SHOWPATH win
```

```

0   1   2   3   4   5   6   7   8
100 010 001 000 000 000 000 000 000
000 000 000 100 010 001 000 000 000
000 000 000 000 000 000 100 010 001

000 100 110 111 111 111 111 111 111
010 010 010 010 110 100 101 101 101
000 000 000 000 000 000 000 100 110 111

```

We can use more than the minimum number of moves for any solution, but only by adding some box 2 or 4 or 6 or any positive even number of times more than necessary.

Solution with BaseBoxes

The solution is intuitively obvious when we use *BaseBoxes* to provide the building blocks. We can also express it mathematically.

Given:

```

BD START ← 0 0 0 1 0 0 0 0 0
32
BD WIN ← 1 1 1 1 0 1 1 1 1
495

+BD □← DIFF ← ADD START, [0.5] WIN
1 1 1 0 0 1 1 1 1
463

```

where

$ADD: * / w$

So we can easily solve a MAGIC puzzle using the unofficial set called *BaseBoxes*. But how do we solve a real puzzle with Bill McLean's set, *MagicBoxes*?

Plus and Times for Boolean Numbers

We can think of $\neq$ for "Booleans" as equivalent to $+$ (and $\wedge$ as well) for ordinary decimal numbers, since (for $\Leftrightarrow$ read 'is equivalent to'):

$$\begin{aligned} x \neq y &\Leftrightarrow 2 \mid x + y \\ &\Leftrightarrow 2 \mid x - y \end{aligned}$$

The dyadic operation $\neq$ is the "sum" for the "Vector Space" to which the Binary representations of puzzle states belong.

It is obvious with either *BaseBoxes* or the original *MagicBoxes* that you can add them together any number of times, but that all even numbers are equivalent to doing nothing, while all odd numbers are just the same as only adding once. So we are discovering a system for "scaling" Boolean vectors x with integer numbers k such that

$$\begin{aligned} x + k y &\Leftrightarrow x && \text{if } 0 = 2 \mid k \\ &\Leftrightarrow x \neq y && \text{if } 1 = 2 \mid k \end{aligned}$$

Suppose we know x and k and that

$$x \Leftrightarrow k z$$

What is z ?

If k is odd, i.e. $1 \Leftrightarrow 2 \mid k$, then

$$z \Leftrightarrow x$$

However, if k is even, both x and z must be null vectors. k cannot be even for non-zero x . This is not quite the same as when

$$X = 0 \text{ Z implies that Z is infinite unless } X = 0.$$

We can in fact extend our system of scalar multipliers to include "rational numbers" of the form (p/q) where p and q are integers. If p/q is reduced to its lowest terms, $p/q \Leftrightarrow 1$ if both p and q are odd, $\Leftrightarrow$ infinity if either is even.

Using BaseBoxes as a Basis

Let's now return to our two sets of boxes. When we don't use the *SHOWBool* function they appear as two 9×9 Boolean matrices:

```
(▼MagicBoxes), (9 10p ' '), ▼BaseBoxes
1 1 0 1 0 0 0 0 0 0      1 0 0 0 0 0 0 0 0 0
1 1 1 0 1 0 0 0 0 0      0 1 0 0 0 0 0 0 0 0
0 1 1 0 0 1 0 0 0 0      0 0 1 0 0 0 0 0 0 0
1 0 0 1 1 0 1 0 0 0      0 0 0 1 0 0 0 0 0 0
1 0 1 0 1 0 1 0 1 0      0 0 0 0 1 0 0 0 0 0
0 0 1 0 1 1 0 0 1 0      0 0 0 0 0 1 0 0 0 0
0 0 0 1 0 0 1 1 0 0      0 0 0 0 0 0 1 0 0 0
0 0 0 0 1 0 1 1 1 0      0 0 0 0 0 0 0 1 0 0
0 0 0 0 1 0 1 1 1 1      0 0 0 0 0 0 0 0 1 0
0 0 0 0 0 1 0 1 1 1      0 0 0 0 0 0 0 0 0 1
```

Consider the third column of *MagicBoxes*

```
SHOWBool 9 1p [] ← MagicBoxes[:,2]
0 1 1 0 1 1 0 0 0
011
011
000
```

This Boolean vector may be seen as signifying that the third Magic Box consists of the following elements of *BaseBoxes*:

```
0 1 1 0 1 1 0 0 0 / 1 9
1 2 4 5

SHOWBool 9 1p * / BaseBoxes[:, 1 2 4 5]
011
011
000
```

So we now know — trivially really — how to express *MagicBoxes* in terms of sums of *BaseBoxes*. And if we have a number of *MagicBoxes* — e.g. 0 5 6 as in Dave's first example game — we can take their sum

```
* / MagicBoxes[:, 0 5 6]
1 1 1 0 0 1 1 1 1
```

It is equivalent to the sum of *BaseBoxes* numbers 0 1 2 5 6 7 8 as we saw earlier:

```
* / BaseBoxes[:, 0 1 2 5 6 7 8]
1 1 1 0 0 1 1 1 1
```

Using MagicBoxes as a Basis

We now need to be able to INVERT the process. Instead of expressing combinations of *MagicBoxes* in terms of *BaseBoxes*, we need to express *BaseBoxes* in terms of *MagicBoxes*. For example, starting from the knowledge that the set 0 1 2 5 6 7 8 of *BaseBoxes* solves a problem, how can we deduce that the set 0 5 6 of *MagicBoxes* does the same job?

Well, the 9×9 matrix *MagicBoxes* is really a transformation matrix from a representation with respect to the basis *MagicBoxes* to one in terms of *BaseBoxes*. For example, the vector

```
m ← 1 0 0 0 0 1 1 0 0 A <=> take MagicBoxes 0 5 6
```

is transformed to the vector

```
b ← 1 1 1 0 0 1 1 1 1 A <=> take BaseBoxes 0 1 2 5 6 7 8
```

using a Boolean Matrix multiplication function *MULT*

```
MagicBoxes MULT m
1 1 1 0 0 1 1 1 1
```

We can define *MULT* either by

```
MULT: α * . ^ ω A using Boolean operations
```

or by

```
MULT: 2 | α + . * ω A using numeric add and multiply
```

Boolean Matrix Inversion

So let's find a Matrix Inverse for the array *MagicBoxes* — the usual APL $\boxtimes$ almost does the trick, but not quite.

The second form of definition for *MULT* suggests that if

$x = M y$, for Boolean x and y and using Boolean matrix multiplication

then $X = M Y$, for numeric X, Y , $2 \mid X \leq \Rightarrow x, 2 \mid Y \leq \Rightarrow y$ and ordinary matrix multiplication, $(+ . \times)$.

If there is a Boolean inverse *IM* for *M*, then $Y = IM X$

so $y = 2 \mid (IM M)(y + 2W)$ for some integer W

i.e. $y + 2Z = (IM\ M)(y + 2W)$ for some integer Z .

But if $IM = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} M$ then this condition is met as $(IM\ M)$ is an identity matrix. However IM can contain non-integer rationals:

$$+IM + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} M + MagicBoxes$$

-0.2	0.4	-0.2	0.4	0.4	-0.6	-0.2	-0.6	0.8
0.6	-0.2	0.6	-0.2	-0.2	-0.2	-0.4	0.8	-0.4
-0.2	0.4	-0.2	-0.6	0.4	0.4	0.8	-0.6	-0.2
0.6	-0.2	-0.4	-0.2	-0.2	0.8	0.6	-0.2	-0.4
-0.2	0.4	-0.2	0.4	-0.6	0.4	-0.2	0.4	-0.2
-0.4	-0.2	0.6	0.8	-0.2	-0.2	-0.4	-0.2	0.6
-0.2	-0.6	0.8	0.4	0.4	-0.6	-0.2	0.4	-0.2
-0.4	0.8	-0.4	-0.2	-0.2	-0.2	0.6	-0.2	0.6
0.8	-0.6	-0.2	-0.6	0.4	0.4	-0.2	0.4	-0.2

We now recall that any odd scalar multiple of a Boolean vector is equivalent to the vector itself in our nine-dimensional vector space. The same applies to matrices:

Let:

$$+IM1 + \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} MagicBoxes \div 5$$

-1	2	-1	2	2	-3	-1	-3	4
3	-1	3	-1	-1	-1	-2	4	-2
-1	2	-1	-3	2	2	4	-3	-1
3	-1	-2	-1	-1	4	3	-1	-2
-1	2	-1	2	-3	2	-1	2	-1
-2	-1	3	4	-1	-1	-2	-1	3
-1	-3	4	2	2	-3	-1	2	-1
-2	4	-2	-1	-1	-1	3	-1	3
4	-3	-1	-3	2	2	-1	2	-1

The divisor 5 is in fact the determinant of the matrix *MagicBoxes*!

$$b$$

1	1	1	0	0	1	1	1	1
---	---	---	---	---	---	---	---	---

$$2 \mid IM1 + . \times b \text{ a which just happens to be the same as}$$

1	0	0	0	0	1	1	0	0
---	---	---	---	---	---	---	---	---

$$m \text{ a !}$$

1	0	0	0	0	1	1	0	0
---	---	---	---	---	---	---	---	---

Unfortunately I've cheated. While we could in general take the ordinary inverse and try different odd integer scalars until we light on the right one, we strictly ought to calculate either the determinant of *MagicBoxes*:

DET MagicBoxes

5

(It is interesting that the J 4.1 determinant (-/ . *) threatens to take over four hours on a portable PC (Toshiba 1850) for this problem.)

... or to gain perhaps even more insight by using a special-purpose function

BINV MagicBoxes

```

1 0 1 0 0 1 1 1 0
1 1 1 1 1 1 0 0 0
1 0 1 1 0 0 0 1 1
1 1 0 1 1 0 1 1 0
1 0 1 0 1 0 1 0 1
0 1 1 0 1 1 0 1 1
1 1 0 0 0 1 1 0 1
0 0 0 1 1 1 1 1 1
0 1 1 1 0 0 1 0 1

```

I cobbled *BINV* together from Keith Smillie's *INV* function in STATPACK2

```

VBINV[ ]
[0] b←BINV a;k;s;p;i;is;t;u;QIO
[1]  a INVERT a SQUARE BINARY MATRIX... AFTER KW SMILLIE'S STATPACK2
[2]  a NB REAL ADD/MINUS BOTH BECOME *
[3]  a REAL TIMES BECOMES ^
[4]  a REAL DIVIDE SHOULDN'T BE NECESSARY AS DIVISOR is 0 OR 1
[5]  a
[6]  a AN ALTERNATIVE METHOD is TO TAKE 2 | (DETERMINANT a) × a
[7]  a BUT THAT NEEDS a DETERMINANT FUNCTION OF SIMILAR COMPLEXITY
[8]  a
[9]  QIO+1 0 +((x/x/b=a+2|b+a)^(2=ppa)^=/pa)/OK a CHECK a is OK
[10]
[11] NOINV:'NO INVERSE!' 0 +-b+1
[12]
[13] OK:p+is+is+k+(/pa 0 a+a[:p,1]
[14]
[15] LOOP:a[:1+s]+2>is 0 i+t:/t+a[i,k;1] a FIND NEXT ROW
[16] p[t+1,i]+p[u+1,1] 0 a[t;is]+a[u;is] a SWAP ROWS
[17] +(0=a[1;1])/NOINV a SINGULAR MATRIX
[18] a DONT NEED ... a[1;]+a[1;]+a[1;1] DIVISOR ALWAYS UNITY IF REACH HERE
[19] a+a*(a[1;1]^1<is)Δ.a[a[1;]] a HAS a - (a[1;] × 1 < is)Δ.* a[1;]
[20] a+a[t+1+s|is;1+is,0] 0 p+p[t] a SHIFT ROWS AND COLUMNS
[21]
[22] +(0<k+k-1)/LOOP a
[23] b+a[:p;is] a REORDER FOR ANSWER

```

It is of course very similar to Smillie's original, just replacing all '×' by '^' and both '+' and '-' by '≠'. Note that I disabled the line involving division by the pivot cell, since we have seen that Boolean "division" yields either no change or an infinity — and we expect the argument to be non-singular as we are dealing with basis representations.

Solution

It just remains to fit the various elements together. The function *PATH* does the job.

```
PATH:(KEY ADD DB w , a )/!pkey
```

The arguments α and w are the decimal representations of the start and target boxes. *ADD DB w , a* finds the solution in terms of *BaseBoxes*.

```
ADD DB win, 32
1 1 1 0 0 1 1 1 1
```

The function *KEY* multiplies this by the Boolean Inverse of *MagicBoxes*, *BaseToKey*, calculating it in the function *BaseRepToKeyRep* if it does not exist. (We see now why *SetVars* expunges *BaseToKey* — it is in case a new set of *MagicBoxes* is invoked as the building blocks.)

```
BaseRepToKeyRep:BaseToKey:0=[NC'BaseToKey':BaseToKey+BINV
DB key
```

```
KEY:BaseRepToKeyRep MULT w
```

```
SetVars 'MagicBoxes' o +m + KEY ADD DB win, 32
1 0 0 0 0 1 1 0 0
```

Finally, the resulting Boolean vector is used to select the *MagicBoxes*,

```
m/!pkey
0 5 6
32 PATH win
0 5 6
```

The remaining functions are just utilities or display routines.

Conclusions

The game of *MAGIC* as presented confuses the issue (probably intentionally!) in being apparently a two-dimensional problem with 512 states. However, as Dave Ziemann showed, we really have a 9-dimensional set of Boolean vectors. The decimal representation is helpful to my mind only to make the notation more concise. It does not add to our understanding of the transitions as Dave found. I suspect that instructive analogies might be drawn with other systems.

For example, theoretical physicists describe representations of microscopic space-time in terms of multiple (11?) dimensional strings embedded in 4 "real"

dimensions. A Boolean number is a numerical realisation of the smallest possible non-trivial commutative "group" of order 2, so that our MAGIC game problem is a little like a simple physics with 9 different quantum properties, each with 2 alternative states. I gather that elementary particle theory teems with interacting "symmetry" groups, mostly built up from simple subgroups of order 2 or 3.

Casting aside speculation, we have found a concise, quick method of solution for MAGIC puzzles. There are symmetries which I have not pursued — the two-dimensional view may after all be useful! — e.g. the solutions for decimal 1, 4, 64 and 256 are all identical apart from rotation. (There are four corner boxes and four edge boxes and one middle. Various symmetry groups — order four and order two — relate corners together and edges together.) It's possible that consideration of these symmetries might lead to an improvement in performance of Dave's algorithm, but my method is so quick and simple that I haven't followed up this particular line.

Duncan Pearson wondered if a similar vector space treatment involving inversion of a transformation matrix could be applied to the N-Queens problem. I don't think so — at least I haven't found it — the reason probably being that you can't realise a presumably N-dimensional basis corresponding to the operation of "adding" a Queen.

However, the ubiquitous not-equal scan does make an appearance, even if not providing as much of the solution as Gérard Langlet's views on the power of this construction would lead us to believe (e.g. Vector 9, No 3).

APL Functions

Here is a listing of all the APL functions involved. It's only those few listed under "SolutionFunctions" that actually find the answer. The others are to do with setting up the problem (*SetVars*) and displaying one or more states (*SHOWBool* and *SHOWDec*) or the whole problem (*SHOWPATH*).

My version of Direct Definition — it's not part of APL*PLUS as supplied — allows local variables, *x*, *y* and *z*, in addition to the arguments *w* (and *α* if the function is dyadic). The direct definition interpreter expects its functions to produce results.

Utility Functions

```
LAM: α .[[IO-0.5] w ⍝ LAMINATE IN NEW 1ST DIMENSION
SetVars: key←BD ⍝ boxes←w ⍝ sym←'01',blk←' ' ⍝ win←495 ⍝ w←0+⍳EX
'BaseToKey'
```

Solution Functions

```

ADD:*/ w
BaseRepToKeyRep:BaseToKey:0=NC 'BaseToKey':BaseToKey+BINV DB key

BD:21 w
DB:1=(9p2)τ w

KEY:BaseRepToKeyRep MULT w
MULT: α ≠.^ w
PATH:(KEY ADD DB w , α )/1pkey

```

Display Functions

```

KEYPath:(0, w )LAM α ,key[w + α PATH w ]
SHOWBool: 3 0 +blk:0≠*/p w :sym[3 3 ρq w ],blk,SHOWBool 0 1 + w
SHOWDec:SHOWBool DB, w
SHOWPATH:(SHP0 w [1 1 ρIO;])τSHP1(w + α KEYPath w )[1+IO;]
SHP0:1φ[1+IO]2φ(- 0 2 4 +p w )† w + 4 0 ▽ 0 0 1 + w

SHP1:((3 4 ρblk),SHOWBool DB,1+ w )LAM SHOWBool scanADD DB w
scanADD:≠\ w

```

```

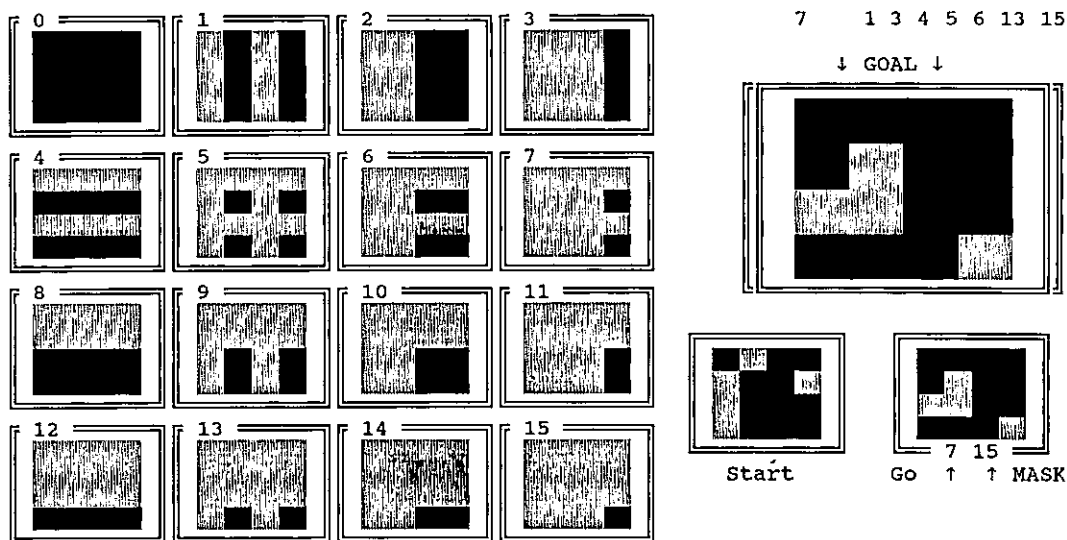
VDET[ ]V
[0] z←DET m;i;j;n;a;b;IO
[1] A FIND THE DETERMINANT OF m
[2] A A METHOD THAT WORKS ... NOT OPTIMAL BUT >> THAN RECURSION!
[3] A DET m EQUIV DET m' WHERE m'[i;] = m[i;] - k × m[j;]
[4] A TRIES TO GET ZERO'S IN LOWER TRIANGLE (OFF DIAGONAL)
[5]
[6] A DET OF A TRIANGULAR MATRIX IS JUST THE PRODUCT OF THE
[7] A DIAGONAL ELEMENTS
[8]
[9] →(∧/∧/ m=m+.×qm)/ENDI,IO+1
[10] i←(0,n+1+pm)[1]
[11] LOOPI:→(n≤i+1)/ENDI
[12] j←i+1n[1]
[13] →(1E-20≥|m[i;]|)/ZERO
[14] m[j;]←m[j;]-m[a;]×m[j;b]÷m[a+i+b;b+n[1]ρi]
[15] →LOOPI
[16] ZERO:→(∧/b+0=i+m[;])/LOOPI
[17] m[i;]←m[i;]+,(<\~b)/[1](i,0)+m
[18] i←i-1
[19] →LOOPI
[20] ENDI:
[21] A m SHOULD BE IN UPPER TRIANGULAR FORM
[22] z←x/ 1 1 qm

```

The Fractal MAGIC Universe

by Gérard Langlet

This is an extended version of the puzzle which has been called "MAGIC" in VECTOR (cf. [1], [2]); recently, a short paper by J.J. Girardot [3] also proposed an APL90 version of the 3×3 magic game, which was given as an exercise to students in computer science at the Ecole des Mines in St. Etienne, to make them progress with APL array capability.



The "Fractalissime" (c) G. Langlet "the fastest APL game in the world"

The goal of the present paper is to bring an ultimate solution to this class of puzzles; then, if one thinks of very complex ones, such as Queens and even chess games, a new conjecture will emerge: π would be able to solve ALL puzzles which have a unique solution... (and physical problems ARE indeed such puzzles for physicists). So, a game is not for fun only; like Cellular Automata (e.g. Conway's Life Game), deep game study may enlighten "General Problem Solving".

Here is a screendump of a PC at the end of a computer demo (which is available in a small WS which also plays classical "Magic" and may be easily extended to play — and teach — much more complex puzzles).

A randomly chosen binary 4-by-4 matrix (among 2×16 different possibilities) is named "GOAL" and displayed, enlarged, on the right of the screen. 1s appear as black squares on the paper, 0s as grey squares (on the screen, 1s appear as a bright foreground, in colour, on a darker background of 0s).

Another 4-by-4 matrix, different from GOAL, is also randomly chosen and is displayed as the initial configuration (on the Figure it is labelled "Start"). The computer also displays in the top left hand corner of the screen a kind of menu with 16 different masks, numbered from 0 to 15.

When a player clicks one of the masks, the bits of the current configuration in correspondence with the 1-bits of the masks are inverted, i.e. negated: 1 becomes 0 while 0 becomes 1. The rule is to determine the shortest sequence of masks which must be successively applied to the initial configuration (Start) in order to reach the GOAL.

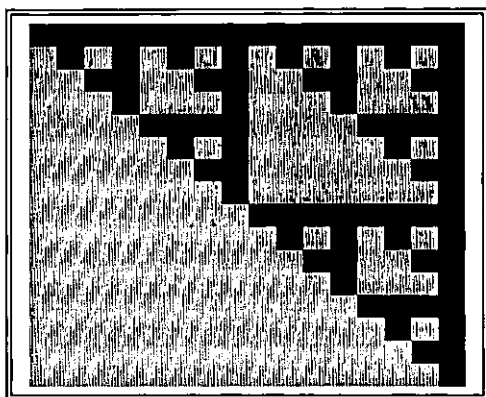
In this demo, the computer plays by itself. At the same time it displays a new GOAL and a new Start configuration; it also tells the user what it has found as the shortest way, i.e. the *exact solution*. For more complex puzzles (which may also be based on frames, i.e. matrices with any shape, even irregular), instantaneous solving of the "best-path" problem by the computer is possible; so, when YOU play against the computer (or, rather, against the APL program), and ignore that the computer has found the best path, you will lose, although you play first — as the computer's guest — after having made just ONE mistake, i.e. one bad choice from the menu.

In the present case, 7 masks are necessary to go from "Start" to "GOAL". These are numbers: 1 3 4 5 6 13 15. The computer has just "won", using mask 15 after six other displays of its clever approach, on the same frame in the bottom right corner (there is not much room on the PC screen) ... with semi-graphics.

What is the Reason for the "Fractal" Qualifier?

Indeed, all games are fractals. If this does not immediately appear in your mind, it comes from the fact that fractality is somehow hidden under the rules (very often, rules are numeric), but appears again as soon as one examines these rules with a *binary* approach.

In the present example, please ravel (unwind) every mask into a 16-bit vector, and form matrix M which will contain one mask per row. Plot M in the same way as masks, i.e. using semi-graphic characters on the PC (see also [4]). You will obtain:



This is a Sierpinski matrix, a ϕ -mirrored "geniton" (cf. [4] to [9]). The corresponding binary matrix M , now considered no more as a Boolean, but rather as a numeric matrix in modulo-2 algebra, an algebra in which all "numbers" $\in \{0, 1\}$, 0 meaning "nothing", 1 meaning "everything", has a nice magic property.

M is its own matrix inverse, so that $2 | M + . \times M$ or, using logical functions, $M \neq . \wedge M$ returns a

unitary matrix, i.e. a matrix with the same shape as M , and in which only the main (or first) diagonal contains 1s. The property is very general, e.g. $9 \ 9 \ + \ M$ is also its own matrix inverse in modulo-2 algebra. With any number instead of 9 in order to take the same number of first rows and columns of the "infinite ϕ -mirrored (or ϕ -mirrored) geniton", the property will hold.

So, the rules of this puzzle were chosen so that matrix inversion becomes unnecessary (although classical Magic involves 3-by-3 matrices, the Fractal-Magic puzzles, even extended to huge arrays, are simpler ...).

If the *GOAL* matrix is ravelled into vector G , and if the *Start* matrix is ravelled into vector S , so that D is the binary difference $G \neq S$ or $S \neq G$, then:

D modulo-2-matrix-divided-by M

will solve a pretty linear system of (here 16) equations, producing X , a modulo-2 vector $\in \{0, 1\}$ which is THE MASK of the (unknown?) necessary transformations. In the example, mask X is:

```

      X
0 1 0 1 1 1 1 0 0 0 0 0 0 1 0 1
      X/1pX
1 3 4 5 6 13 15

```

A in 0-origin
A 7=+/X

APL has no "modulo-2 matrix divide". There are several solutions which could fill the gap made by the lack of such a (useful?) extension:

1. write the modulo-2-matrix divide using the domino;

2. write the modulo-2-matrix divide NOT using the domino;
3. do *nothing but thinking*.

The best way is to choose as a rule a *self-inverse matrix* so that the missing function becomes *useless*; hence the use of the ϕ -mirrored geniton coming from the APL-Theory of Everything [4].

If M is such a matrix, the same mask X will be the result of

$$D \neq .\wedge M \text{ or of } 2 | D + .\times M$$

which is very fast, at least for "small" matrices.

Mask inversion in APL is also performed by $\neq$, the difference primitive, which corresponds to PLUS modulo-2 and to MINUS modulo-2 in modulo-2 algebra at the same time. The expression

$$C \leftarrow \neq \backslash D, X \neq M$$

will produce successive rows in a row-"difference-scanned" matrix (successive Current Configurations), so that, if N is $\begin{smallmatrix} -1 & \phi & 4 & 4 \\ , & 1+ & +/ & X \end{smallmatrix}$ then:

$$(N \rho G) \neq N \rho C$$

will return the successive current 4-by-4 matrices which display the path from Start to Goal (including both).

The conjecture that $\neq$ alone or in association with "scan" could model everything else (i.e. all *clever* computing *viz.* Nature's) is reinforced by the fact that the result of the binary matrix product of a binary vector by a conforming ϕ -mirrored geniton, i.e. result X ,

- a. is one of the Fast Fourier Transforms in modulo-2 algebra (named "helix transform" in [4] to [7]), then bound to the properties of information in helices, the clue to future understanding of DNA genetic patrimony;
- b. can also be obtained as ϕV , the vector of the successive defined integrals of $K \leftarrow \phi D$ by iteration of $V \leftarrow V, \quad -1 \uparrow K \leftarrow \neq \backslash K$ 16 times in the example, starting from an empty vector;
- c. can be computed using a fast algorithm in which only $\neq$ is used (the AND $\wedge$ primitive of the matrix product disappears in the holes of the Gruyère cheese of the fractal matrix!) — cf [5], [8] and refs therein.

Solving the original Magic this way can now be undertaken by pure thinking on the *symmetry* of the problem: the 9 matrices which ARE the rules for Magic have to respect the symmetry of the square, i.e. a 4-fold symmetry; then, the 9-by-9 matrix M in which each row is the ravelled bit-sequence of each 3-by-3 square transformer, has to be, in modulo-2 algebra, a 4th root of the conforming unitary matrix U (readers may easily verify this property, even with a free version of APL such as I-APL or TRYAPL2):

$$U \equiv M \times . \wedge M \times . \wedge M \times . \wedge M$$

The same expression may be re-written as:

$$U \equiv M \times . \wedge (M \times . \wedge M \times . \wedge M)$$

in order to show (although parentheses have no other utility within this APL identity), that $I \leftarrow M \times . \wedge M \times . \wedge M$ HAS TO BE the modulo-2 matrix-inverse of M . I can be pre-computed ONCE and memorised as 81 bits (less than 11 bytes).

Then, the short APL expression (in which $/[\square IO] \setminus [\square IO]$ and $, [\square IO]$ may replace $\neq$ $\backslash$ and $\neq$ respectively in APL):

$$C \leftarrow \neq D \neq X \neq I$$

produces the successive Configurations for Magic as well, at the speed of light, given any difference vector D as $S \neq G$ (*Start* and *Goal* ravelled vectors with shape 9).

Then, the 9 matrices of Magic may be replaced by any linear combination of themselves in order to form a coherent system of rules, so that the number of NEW puzzles even for shape 3 3 will become quite large...

And the APL community can now become a puzzle generator:

Given a thick book with 1024 pages, with an image on each page as a 1024 1024 pixel binary matrix, such a book will become the "Puzzle Solver/User's Manual"; then, on a high-definition screen, with a 1024 by 1024 image, an initial Start configuration will appear with 1 kilopixel size, as well as the Goal, e.g. in another colour if the screen allows RGB output; the user will have to choose a minimum of successive transforms from the atlas-book; even for billions of pixels and images in a collection of video-discs, the same magic formula (a tiny APL line) will hold... , as it WILL for the giant Sierpinski puzzle of our fractal universe of which any problem is a sub-problem, even tomorrow's weather forecasting...

$\neq \backslash$ is the absolute integrator at the quantum level. Many of its properties were found and proven using APL. Now, *natural* shapes are deduced from $\neq \backslash$ algorithms [10].

Then, is there STILL a reason why most physicists ignore Iverson's powerful notation for mathematics and programming, although parity and symmetry are their most basic concepts, while integration, Fourier transforms, spinors or twistors (matrix theory) are their favourite mathematical tools, and in spite of the fact that they are trying to understand what hides under the fashionable keyword *fractal* coined by Benoit Mandelbrot, which will NEVER be explained by continuity?

References

- [1] Ziemann D, "*Exploring Magic*", Vector Vol.9 no.2 pp. 16-27, October 1992.
- [2] McLean W W, "*MAGIC*", Vector Vol.9 no.1 pp. 25-29, July 1992.
- [3] Girardot J J, "*L'inversion Diabolique*", Les Nouvelles d'APL no. 6 pp. 54-55, March 1993.
- [4] Langlet G A, "*Towards the Ultimate APL-TOE (Theory of Everything)*", APL92 Conference Proceedings pp.118-135.
- [5] Langlet G A, "*Un Concours Paritonesque*", APL-CAM Journal, Vol.14 no.3, pp.412-416, July 1992.
- [6] Langlet G A, "*From the Vital Execute to Fractals and 5-Fold Symmetry*", Vector Vol.9 no.3, pp. 91-96, January 1993.
- [7] Langlet G A, "*New Mathematics for the Computer*", Tool of Thought VIII, pp.1-31, January 1993.
- [8] Langlet G A, "*Symétries, Force et Phénomènes*", APL-CAM Journal Vol.15 no.1, pp. 57-80, January 1993.
- [9] Langlet G A, "*From Risky Programming to the Baby Computer*", APL-CAM Journal Vol.15 no.1, pp.81-92, January 1993.
- [10] Langlet G A, "*Building the APL Atlas of Natural Shapes*", APL93 (in press).

$\neq \backslash$

Index to Advertisers

Cybex	112
Dyadic Systems Ltd	58
Greymantle Associates	8
Lingo Allegro Inc	24
Manugistics	18
MicroAPL	2
Soliton	52

All queries regarding advertising in VECTOR should be made to Gill Smith, at 04393-385.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue n+1 and discuss themes for issues n+2 onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Anthony Camacho,
11 Auburn Road, Redland,
BRISTOL, BS6 6LS
Tel: 0272-730036

Authors wishing to use Windows Write should contact Vector Production for a copy of the Vector APL TrueType font and Vector APL typebox.

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members' news) should be sent to Vector Production, Brook House, Gilling East, YORK. Tel: 04393-385 (any time).

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL.
The membership year runs from 1st May to 30th April.

Name: _____
 Address Line 1: _____
 Address Line 2: _____
 Address Line 3: _____
 Post or zip code: _____
 Country: _____
 Telephone Number: _____

Membership category (please tick box): 93/94

(these rates also apply to renewals)

UK private membership	£12	☐
Overseas private membership	£20	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£100	☐
Corporate membership overseas	£155	☐
Sustaining membership	£430	☐
Non-voting UK member (student/OAP/unemployed only) £6		☐

PAYMENT — in Sterling only

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Mastercard or Visa number.

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____ | ____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

*Data Protection Act:
 The information supplied may be
 stored on computer and processed
 in accordance with the registration
 of the British Computer Society.*

Signature: _____ Send the completed form to:

British APL Association, c/o Rowena Small, 8 Cardigan Road, LONDON, E3 5HU

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1993/94 Committee

Chairman:	David Eastwood 071-922 8866	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Secretary:	Duncan Pearson 0904-603510	143 Hull Road, YORK YO1 3JX
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Anthony Camacho 0272-730036	11 Auburn Road, Redland, BRISTOL, BS6 6LS
Activities:	Richard Weber 0302-539761	10 Hakehill Close, Bessacarr, DONCASTER, DN4 7EY
Education:	Dr Alan Mayer 0792-295296	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Technical:	Jonathan Barman 0488-648575	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Projects:	George MacLeod 0442-878065	Greymantle Associates Ltd., Bartrum House, Ravens Lane, BERKHAMSTED, Herts HP4 2DY
Publicity:	Misha Jovanovic 0753-853141	99 Oxford Road, WINDSOR, Berks SL4 5DX
Recruitment:	Mark Harris 0438-310155	Kestrel Consulting, Business & Technology Centre, Bessemer Drive, STEVENAGE, Herts SG1 2DX
Administration:	Rowena Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU

Journal Working Group

Editor:	Anthony Camacho	0272-730036
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Advertising:	Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	Jonathan & Bridget Barman (0488648-575), Ray Cannon (0252-874697), Richard Weber, Sylvia Camacho, Duncan Pearson, John Searle, David Ziemann (071-267 8032), Jon Sandles	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly journal of the British AFL Association and is distributed to Association members in the UK and overseas. The British AFL Association is a Specialist Group of the British Computer Society. AFL stands for 'A-Programming Language' - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframes, workstations and personal computers.

SUSTAINING MEMBERS

The Committee of the British AFL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

Compass R&D Ltd
15 Frederick Sanger Road
Surrey Research Park
Guildford, Surrey GU2 5YD
Tel: 0483-302249
Fax: 0483-302279

HMV Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON E6Y 0HA
Tel: 071-459-4212
Fax: 071-459-4326

MicroAFL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel: 071-922 8866
Fax: 071-928 1006

Peter Cybex Systems
22 Herford Road
LONDON W2 4AA
Tel: 071-229 5844

Manjalis
219 Eastclifferson St
Rockville
MARYLAND 20852 USA
Tel: (301) 984-6412
Fax: (301) 984-5094

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE
Hants, RG23 0AL
Tel: 0256-811125
Fax: 0256-811130

Cooking & Bury Ltd
130 Tottenham Court Road
LONDON, W1P 0UE
Tel: 071-486 9481
Fax: 071-486 0524

Sollon Associates Ltd
Groen Bankenberg 33
1082 AC Amsterdam
Netherlands
Tel: +31 20 670 8739
Fax: +31 20 670 6753
Email: h.j.n.sollon.com

Dutch AFL Association
Postbus 1841
3430 BH Nieuwegein
Netherlands
Tel: 0374-2537

Kestrel Consulting
Business & Technology Centre
Bossamer Drive
STEVENAGE, Herts SG1 2DX
Tel: 0438-310165
Fax: 0438-310161
Email: k.kestrel@apl.demon.co.uk